NUCLEAR OPTION · FLIGHT CONTROL

NO FCS Kit

A measured-state fly-by-wire for Nuclear Option aircraft, packaged so you can put it on your own.

Reference jet	FCS version	What the aircraft is
F-22E Strike Raptor (Aryx's mod)	4.3.0	twin canted fins + rudders, all-moving stabs, flaperons, **pitch-only TVC**; since 3.2.0 a **relaxed-to-neutral airframe** (airframe X2e)
KR-67 Ifrit (stock)	1.3.0	canted all-moving tails that are also the **yaw axis** (no rudders), split decelerons, **pitch + roll TVC**
FS-12 Revoker (stock)	1.4.1	**canards**, single fin, delta elevons, **pitch + yaw TVC**
T/A-30 Compass (stock trainer)	0.5.0	conventional tail slabs, wing-root strakes, ailerons + flaps, **no TVC**, fixed-nozzle twin
FS-3 Ternion (Blueprinter mod)	0.4.0	**two pitch pairs** (canards + aft elevators), twin rudders, flaperons + rollervons, **no TVC**
Su-30VXN Gandiva (kyleisred's Blueprinter mod)	0.6.0	canards + stabs + rudders, flaperons, **canted single-axis nozzles** (each nozzle one axis, 32° outboard)

Status in one line: the source here rebuilds all six shipped release DLLs byte for byte (Rust core included), and all six benches build, compile their native core and fly. The bench is an approximation of the game and may not match it exactly, but the in-game behaviour of every release in this kit has been **flown and passed by the author (Tom, 12 Nov 2026)**. Details in §17.

Kit 3.1 (12 Nov 2026) adds Part IV: a control theory primer for modders coming from the stock fly-by-wire (§21), the workflow and tools behind the kit and why an LLM agent fit it (§22), what to expect doing it by hand (§23), and airframe aerodynamics recommendations (§24).
New in kit 3.0 (12 Nov 2026). The heavy maths of every jet now runs in a **native Rust core** carried inside the plugin DLL, bit-identical to the C# on the bench and about 4× faster in game (§2.18); an **asymmetric-load AoA limit from a coupled check** of every effector replaces the one-axis check on five jets (§8.23); loaded roll reversals no longer stop and then spike (§8.25); **flying backwards**, stick and pedals turn the nose the way they point (§8.24); the main brakes help steer on the ground (§8.26). New bench sets and game-runtime checks are in §10.8, building the core in §15.3, and eleven new failure modes in §13 (F29–F39). Kit 2.0 (10 Nov) added the Compass, Ternion and Su-30, multi-core ticks, the F-22E's relaxed-to-neutral airframe and the push-over work. **Every problem and solution since the first kit (5 Nov) is in §19**; §19.8–19.14 cover 10–12 Nov.

FCS by **Tom** (GitHub `duythinh2022-ui`) · developed with Claude, Sept–Nov 2026
Manual built 12 November 2026 from `README.md`

Start here

If you want to...	Read
learn the flight-control basics first (AoA, static margin, velocity-vector roll, INDI, allocation)	§21 Control theory primer
know how the work was done, with which tools, and what to expect doing it by hand	§22 Workflow and tools, §23 Doing it by hand
shape your airframe so the FCS can fly it well (CG, margin, surfaces, strakes, thrust line)	§24 Airframe aerodynamics
know why a replacement FCS is needed at all	§1 Why an FCS mod
understand how this one works	§2 How it works, then §3 the six jets
fly one of the six jets	§6 Using it in game
put it on your own aircraft	§4 Quick start → §7 Porting → §8 Solution library → §9 Models → §10 Bench → §11 Tuning
fix something that flies wrong	§12 Troubleshooting, §13 Failure modes, §19 Problem and solution log
see what changed since the first kit	§19 Problem and solution log
look up a word	§18 Glossary
work on the code as an AI model	`AGENTS.md` first: the hard rules and procedures in compact form

Every technical term is explained in plain English where it first appears, and again in the glossary. The same manual is also in this folder as a formatted PDF, **`NO-FCS-Kit-Manual.pdf`**, which is easier to read than this file.

Contents

Part I · Understand it

1. Why an FCS mod: the vanilla flight control and its limits
2. How it works
3. The six reference jets side by side

Part II · Use it and build with it

4. Quick start
5. What is in the folder
6. Using it in game
7. Porting it to your aircraft
8. Solution library: aircraft configurations we have solved
9. Adding the models
10. The bench
11. Tuning

Part III · Reference

12. Troubleshooting
13. Failure modes we have hit
14. Design rules
15. Building, versioning and releasing

16. Telemetry (flight logs)
17. Verification status
18. Glossary
19. Problem and solution log
20. History and further reading

Part IV · Background and practice

21. Control theory primer for this FCS
22. Workflow and tools
23. Doing it by hand: caveats
24. Airframe aerodynamics: recommendations for modders

PART I

Understand it

1. Why an FCS mod: the vanilla flight control and its limits

1.1 What the vanilla fly-by-wire actually does

Every Nuclear Option jet flies through the same stock fly-by-wire: `ControlsFilter.FlyByWire.Filter` in the game's `Assembly-CSharp.dll`. Each aircraft prefab only supplies its own parameter values. What follows was read from the disassembled method in the game build of late October 2026 (the bench's `bench/sim/StockFbw.cs` is a port of the same method). The chain is:

```
stick, pedals ──► FlyByWire.Filter ──► normalised pitch / roll / yaw inputs (−1..1) ──► every ControlSurface deflects
                  (PID on body pitch rate,                                              input × its fixed pitch/roll/yaw range
                   P loops on roll and yaw rate)                                        (nozzles: pitch input × fixed vectoring
```

In plain terms:

- **Pitch:** the stick asks for a body pitch rate. Below the corner speed the target is stick × gLimit × g / V, scaled down by the dynamic-pressure ratio (no lower than 30 %). That is a steady-turn guess of the g the rate will give, n ≈ V·q/g. A PID on the pitch-rate error is integrated into the stick output (`pitchAdjuster`). Above the corner speed the PID output is divided by the dynamic-pressure ratio (`remapFactor` = 1/max(q̄/q̄_corner, 1)).
- **AoA limiter:** only below the corner speed, and only when the AoA is past `alphaLimiter` (25–31°) on the same side as the command. It then scales the pitch-rate *target* down by `alphaLimiterStrength` per degree over at most 10°.
- **Roll:** a proportional roll-rate loop. The target is stick × `maxRollAngularVel` × clamp(q̄/q̄_corner ÷ `maxRollSpeed`, 0.5, 1), plus a slow roll-trim integrator while the pitch rate is small. Above the corner speed the roll output is blended back toward the raw stick by `remapFactor`.
- **Yaw:** `yawTightness` × `remapFactor` × (body yaw rate − pedal): a yaw-rate damper, with the pedal as a yaw-rate command.
- **Stability Assist off:** below 1.2× the corner dynamic pressure, the pitch target fades (with a time constant of about 1 s) to stick × `maxPitchAngularVel`: a plain rate command with no g limit.

Parameters of the three reference jets' stock FBW (from their prefabs):

Parameter	F-22E (Aryx)	Ifrit	Revoker
`gLimitPositive`	9.5	9.0	9.0
`cornerSpeed` (m/s)	140	160	160
`alphaLimiter` (°) / `alphaLimiterStrength`	31 / 0.05	27 / 0.05	27 / 0.1
`maxPitchAngularVel` (rad/s, rate law)	1.25	0.75	0.9
`maxRollAngularVel` (rad/s)	10	10	6
`maxRollSpeed`	210	300	300
`pFactorFast` / `iFactor` / `dFactorFast`	7 / 0.01 / 1.22	6 / 0 / 1.5	10 / 0 / 1.0
`yawTightness` / `rollTightness`	2 / 1.1	1 / 0.2	4 / 0.75

The prefabs also carry `postStallManeuverSpeed`, `pidTransitionSpeed`, `pFactorSlow` and `dFactorSlow`, but `Filter` does not read them in the build checked.

1.2 The limitations, and what this FCS does instead

Ten things the vanilla FBW cannot do. Each one: what is missing, what it causes in flight, and what replaces it here.

1. No real g or AoA command. The g limit is open-loop: a pitch rate computed from n ≈ V·q/g, with no measured-g feedback. At high AoA the body pitch rate is not the flight-path rate.

- *What it causes:* g over- and undershoot; the stick doesn't mean the same thing across the envelope
- *What this FCS does instead:* G + pitch-rate command on **measured** nz; reachability blend (§2.8)

2. The AoA limiter is soft and partial. It only reduces the command (by at most 50 % at strength 0.05), only below the corner speed, and never pushes the nose down. Above the corner speed there is no AoA limit at all.

- *What it causes:* the jet can be pulled into a deep stall or a departure; nothing recovers it
- *What this FCS does instead:* hard AoA limits with kinematic capture at the *real* nose-down authority, protection, a low-thrust cap and an asymmetric-load cap (§2.8, §2.11)

3. The roll speed schedule never acts. The speed term (q̄ ratio ÷ `maxRollSpeed`, about 0.005) always clamps to its 0.5 floor. The target is therefore a fixed 0.5 × `maxRollAngularVel`: **286°/s** on the F-22E and Ifrit, 172°/s on the Revoker, at every speed. The high-speed roll reduction (q̄ ratio > `maxRollSpeed`) can never trigger.

- *What it causes:* roll rate demand unrelated to what the jet can do or stop
- *What this FCS does instead:* roll ceiling from the live roll damping and authority; governor caps the commanded rate to what the yaw can follow (§2.9, §2.10)

4. Body-axis roll, separate roll and yaw axes at every AoA. Yaw is only a yaw-rate damper.

- *What it causes:* at AoA a body roll turns AoA into sideslip; at high AoA it departs (on the bench the stock FBW departs the Ifrit in a loaded roll at the 65° limit)
- *What this FCS does instead:* velocity-vector roll with a sideslip loop; one coupled lateral axis past ~36° (§2.9)

5. One crude gain schedule. All loop outputs are divided by the dynamic-pressure ratio above the corner speed, and the roll output blends back toward the raw stick.

- *What it causes:* response changes with speed; at high speed roll is partly open-loop
- *What this FCS does instead:* INDI: the loops act on measured acceleration, so they need no speed schedule (§2.5)

6. Fixed surface mixing. Every surface moves input × its fixed ranges, whatever its local AoA, stall or reversed flow, damage, or what the other surfaces are doing. The nozzles follow the pitch input whether or not there is thrust.

- *What it causes:* stalled surfaces keep being driven the wrong way; nothing redistributes when a surface stalls, saturates or is shot off; no use of TVC to free the tails for roll/yaw
- *What this FCS does instead:* nonlinear allocator over stall- and reversal-aware models of every effector at its local AoA (§2.6, §2.7)

7. No damage or failure logic. Nothing for asymmetric stores, lost fins or wing sections, or an engine out.

- *What it causes:* rolls into the asymmetry, departs, no VMC protection
- *What this FCS does instead:* asymmetric-load cap, yaw-health scaling, decelerons, fin-loss cap, thrust match, VMC limiter, fuel balance and trim (§2.11)

8. No authority or saturation awareness. The only protection against windup is clamps (integrator ±0.2, output ±1).

- *What it causes:* commands the jet can't follow; overshoot after saturation
- *What this FCS does instead:* capacity estimator and governor; the allocator reports unmet moment (§2.10)

9. Stability Assist off has no g limit. Below 1.2× the corner dynamic pressure the stick is a plain rate command.

- *What it causes:* over-g possible
- *What this FCS does instead:* MPO keeps g limits (+12.5 / −3.5)

10. Idle throttle deploys the airbrake. (the stock idle-brake rule).

- *What it causes:* unwanted braking at idle; brake tied to throttle
- *What this FCS does instead:* brake control only, at any throttle; fades with its drag share at high AoA (§2.12)

1.3 Why a new FCS instead of retuning the stock parameters

The parameters only scale a structure that is missing pieces. No set of values adds an AoA protection that pushes the nose down, a sideslip loop, a velocity-vector roll, a surface allocation that knows when a surface has stalled, thrust vectoring used on purpose, or anything for damage. Each of those needs code that reads the live airframe.

That is what this FCS adds. The cost:

- **CPU:** single-threaded it was about 0.7–2.4 ms of main-thread time per physics tick per jet on the bench (F-22E 0.7, Ifrit 1.9, Revoker 2.3, Su-30 2.05), mostly allocator solves and the capacity estimator's probes. Since F-22E 3.0.4–3.1.3 (ported to every jet on 9 Nov) most of that work runs on other cores: the main thread drops to about 0.43–0.5 ms (F-22E) and 0.8–1.1 ms (the stock jets) on a 2-core test machine (§2.16). Other players' jets run on their own machines.
- **Multiplayer:** it only works where the host allows it (§2.14).
- **Complexity:** every aircraft needs its effectors described and a bench to check them (§7, §9 and §10).

2. How it works

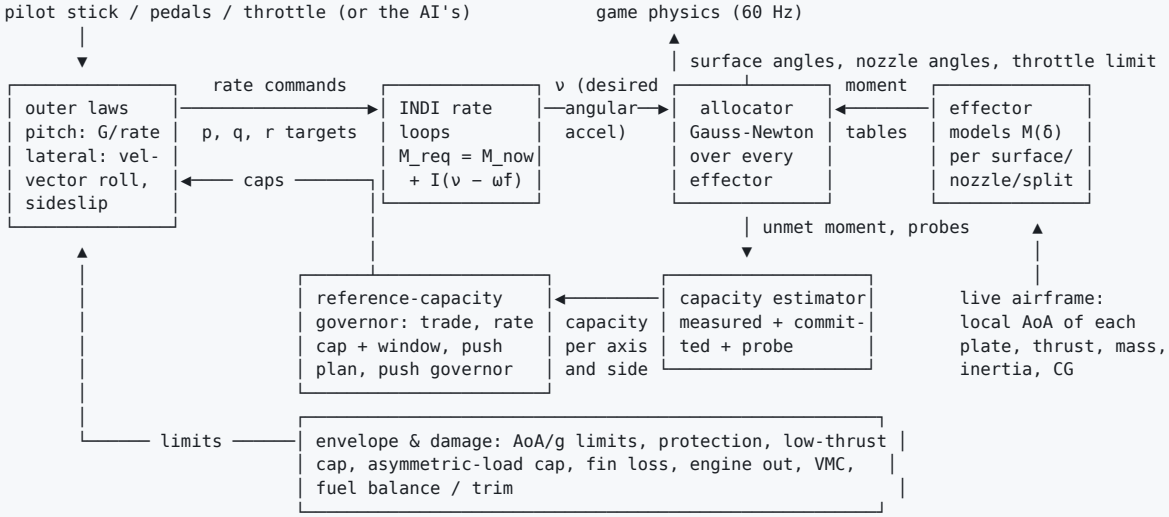
2.1 The idea in one paragraph

Nuclear Option (NO) simulates an aircraft as a set of rigid parts, each carrying an independent aerodynamic **plate** (a flat lifting surface with an airfoil lookup table, a wing area and a centre of lift). There is no Mach effect on the airfoils, no tail blanking (one surface shadowing another) and no camber coupling (a deflected flap only changes its own plate). The FCS works *in the game's own terms*: it re-evaluates every control surface's plate at that surface's real local angle of attack to know what moment each deflection will make, and it reads the airframe's actual angular acceleration every tick instead of trusting a model of the airframe. Concretely it is three layers:

1. **Outer laws** turn the pilot's stick into a *body-rate command* (how fast to pitch, roll and yaw), with every limit (AoA, g, rates, sideslip) computed from the live airframe rather than from a schedule.
2. **INDI** (incremental nonlinear dynamic inversion; plain English: "measure how the jet is accelerating right now, and ask the control surfaces only for the *difference* needed") turns that rate command into a *moment request*.
3. A **nonlinear control allocator** (plain English: a small optimiser that decides how far to move each surface and nozzle so that, together, they produce the requested moment) solves for every effector's deflection, using stall- and reversal-aware models of each one.

Around that core sit a **reference-capacity governor** (it slows a command when one axis can't keep up with the others), envelope protection, damage tolerance, ground handling and aircraft systems.

2.2 The loop



2.3 One physics tick, in order

The FCS runs once per physics step from a Harmony prefix on the game's `ControlsFilter.Filter` (`Hooks.FilterPrefix`), which replaces the stock fly-by-wire for that aircraft and calls `FcsController.Tick(inputs, flightAssist)` then `VmcApply`. Method names below are in `FcsController.cs` unless stated; line numbers drift between versions, so search by name.

#	Step	Where	Plain English
1	Mass properties	`UpdateMassProps`	total mass, CG and inertia tensor from the live rigidbodies (stores, fuel, lost parts included)

#	Step	Where	Plain English
2	Air data at the CG	`Tick` start (`AirDataAtCg` , `FlightPathAtCg`)	TAS, IAS, AoA (α), sideslip (β), load factor (nz), flight-path rotation (ωV), all at the CG, not the nose
3	Flaps / slats / rudder fade	`flaps.Update` , `rudderAuth`	high-lift schedule; how much the rudders are still a yaw axis
4	Effector tables	`eff.Evaluate(ref ctx)` for every effector (since 3.1.0: `EvalGather` on the main thread, `EvalCompute` on several cores)	rebuild each effector's moment-vs-deflection table at this tick's flow (~240 plate evaluations on the F-22E)
5	INDI filters + hybrid lag model	`omegaDotF` , `mNowF` , `ComputeHybridSlopes` , distortion lead	measured angular acceleration (filtered, τ = 0.04 s) and a model of the filter's lag at high speed
6	Authority reads	`aUp` , `aDn` , `aRoll` , `aYaw`	how much nose-up/down, roll and yaw acceleration the effectors can make right now
7	Allocator weights and costs	`alloc.axisWeight` , fight costs, pitch priority, reach pass, stall windows	what the optimiser should care about this tick (e.g. pitch first after a stick release)
8	Envelope and damage sweeps	`AsymStep` , `AuthCapStep` , `YawHealthStep` , `ComputeEnvelope`	asymmetric-load cap, low-thrust AoA cap, fin health, reachable AoA/g (some every 0.1–0.2 s)
9	Mode	`Mode` (`FcsMode`)	Ground, GearDownRate, AoAG (normal), Rate, MPO (Stability Assist off), AIRate
10	Stick shaping	`ShapeCommands` , `StickSmoothing`	smooth and shape the pilot input
11	**Pitch law**	`PitchLaw`	pitch-rate target qT from G/rate command, envelope, capture, protection
12	**Lateral law**	`LateralLaw` (+ `PushPlanStep`)	roll / yaw targets: velocity-vector roll, sideslip loop, governor
13	INDI moment request + clamps	after the laws	M_req = M_now + I(v − ώf); yaw target clamped to reach; split-flaperon / deceleron yaw past the rudders
14	TVC budget, branch passes	`TvcBudgetStep` , `RollPitchPass` , `TvcNeutralHold`	keep the nozzles near trim during rolls, stab "branch" switching
15	**Allocate**	`alloc.Solve(effs, target)` (one to a few passes)	solve every effector's command
16	Capacity estimator	`capEst.Step(...)` , since 3.0.4 kicked onto a worker thread (`EstimatorJob`) and joined at the top of the next tick	measure what each axis can still do; read by the laws on the **next** tick
17	Telemetry	`CaptureRow` (main thread) → background writer (`TelemetryRow`)	one CSV row when logging is on
18	VMC / thrust match	`VmcApply` , `ThrustMatchApply` (after `Tick`)	limit throttle when an engine-out yaw can't be held

Steps 0 and 16 bracket the tick: the estimator job started at step 16 of tick N is collected at the very top of tick N+1, before anything reads it, so threading never changes what the laws see (§2.16).

The surface commands are written into the game by `Hooks.SurfacePostfix` (a postfix on `ControlSurface.UpdateJobFields` , which sets the control-surface job's target angle), and the nozzle angles by transpiler/prefix hooks on the engine's `FixedUpdate` (stock jets: `Turbojet` , `Hooks.TurbojetTranspiler` and `TvcPitch` / `TvcRoll` / `TvcYaw` ; the F-22E mod: `Turbofan` , `Hooks.TurbofanTranspiler`). The game's own servos still move the surfaces at their servo speed, and the FCS models that lag.

2.4 Air data and state

- **Reference point.** All air data (velocity, AoA, sideslip, flight-path acceleration) is taken at the centre of mass (`AirDataAtCg`, F-22E 2.30.0). The game's root rigidbody is not at the CG: 2.48 m ahead on the F-22E, 0.62 m on the Ifrit, 0.95 m on the Revoker. Reading at the root made a yaw rate look like sideslip.
- **Unwrapped AoA** (`alphaU`) so a tail slide through ±180° is not read as a −45° violation.
- **Mass properties** come from the live rigidbodies every tick (`UpdateMassProps`), so stores, fuel burn, fuel transfer and shot-off parts change inertia and CG immediately. The inertia inverse `IaInv` is what INDI and the allocator use.

2.5 INDI: the inner rate loops

The outer laws give a body-rate target ω_cmd. Each axis has a first-order rate loop (gains `PitchRateGain` 5, `RollRateGain` 6, `YawRateGain` 2.5 per second) that turns the rate error into a desired angular acceleration ν. INDI then asks for

M_req = M_ctrl(δ_now) + I · (ν − ω̇_f)

- M_ctrl(δ_now) is the moment the effectors make *right now* according to their models (`Mnow`, summed);
- ω̇_f is the measured angular acceleration, filtered (τ = 0.04 s, `IndiTau`);
- I is the live inertia tensor.

Plain English: whatever the airframe is doing that the FCS did not model (its own aero, inertial coupling, damage, gusts) shows up in ω̇_f, so it cancels out. The FCS only has to model the *control effectors* well, not the whole aircraft. This is why an explicit inertial-coupling feed-forward was found harmful and turned off (1.8.14): INDI already measures the coupling as it develops.

Additions on top of plain INDI:

- **Hybrid lag model** (`HybridIndiGain`, 1.9.0): at high speed the airframe's own pitch moment changes fast with α and q, and the 0.04 s filter lag let q overshoot. The lag is predicted from model slopes of the non-control parts (∂M/∂α·(α − α_f) + ∂M/∂q·(q − q_f)), faded out between 25° and 40° AoA.
- **Disturbance-rate lead** at high AoA (2.4.7, `HybridIndiHighAoALead`).
- **Live-node anchoring** (`AnchorLateral`, F-22E 3.0.1 / Ifrit 0.8.3 / Revoker 0.8.2): the allocator answers from tables but INDI's target is built on the live moment; where the two differed at a surface's current deflection, the difference acted like a standing pilot input and made the jet creep in roll with an asymmetric trim. Each surface's live point is now a node of its own roll and yaw table (`Effector.SetLiveNode`).

2.6 The allocator

`Allocator.Solve(effs, target)` finds the commands δ for every active effector that minimise

$$J(\delta) = W\cdot\left\|\Sigma_i M_i(\delta_i) - M_{req}\right\|^2 + s(\bar{q})\cdot\Sigma_i \mu_i(\delta_i/N_i)^2 + \lambda\cdot\Sigma_i(\delta_i - \delta_i^{-})^2 + J_{pair} + J_{fight} + J_{budget}$$

with box constraints (travel limits, and how far each servo can move this frame). Plain English: get as close as possible to the requested moment; among the ways to do that, prefer small deflections (μ), small changes from last tick (λ), sensible pairings, and avoid surfaces fighting each other.

- **Solver:** Gauss–Newton with Levenberg–Marquardt damping (`LmFactor` 1e-3), a few iterations per call (`gnIterations` 4), numerical Jacobians on each effector's table. It is warm-started from last tick's answer.
- **W** (`axisWeight`): per-axis weights, changed every tick by the laws (e.g. pitch priority ×up to 30 after a stick release, `ReleasedStickPitchPriority`; yaw priority `YawAxisPriority`). With `SlipFrameAlloc` (stock jets) the lateral residual is weighed in the velocity (stability) frame.
- **s(q̄)** (`regScale`, `LowQAllocatorRelief`): the error term scales with dynamic pressure squared, the regularisation does not; at low speed the cheapest answer used to be "stay neutral" (weak roll at 117 kt). The regularisation now scales with (q̄/q̄_ref)², floored at 0.02.

- **J_pair:** each _L/_R pair is priced in its symmetric and antisymmetric parts (`symCost`, `antiCost`), so differential stab or rudder toe-in can be made cheap or expensive on their own. TVC's costs are ~50× below symmetric stab: that is how "TVC unloads the stabs for roll and yaw" is expressed.
- **J_fight:** penalises surface groups whose roll opposes the others (`fightK`, `rollGroup` 1 stabs / 2 outer flaperons / 3 inner flaperons) and stab-vs-nozzle pitch fights (`pitchFightK`, `pitchGroup` 1 stabs / 2 TVC). These break "crossed sets" (§13, F5).
- **J_budget:** nozzle budget (`anchor`, `band`, `anchorK`): beyond ±band of the trim anchor each degree costs extra, so rolls don't swing the nozzles (`TvcBudgetStep`).
- **Stall windows** (`StallWindow`, `SettledWindow`, `StabParkEscape`): limit a stab's travel to the attached side of its lift curve in the states where a stalled "plateau" would trap it (§13, F4).
- **Reach pass + two-horizon pick** (`ReachPass`, `TwoHorizon`, F-22E 2.41/2.43): a slow effector (the stabs) keeps its command, and the fast ones (nozzles, wing surfaces) are placed against where the slow one *will be* within `reachTau` 0.1 s, not where it is sent. Otherwise the slow one is moved only as far as it gets in that time, whichever plan costs less over the transit and at arrival. This replaced the older stab rate bound.
- **Paired stab/TVC search** (`StabTvcPairedSearch`): hands pitch from a stab parked past its useful range to the nozzles so the stab can go back to roll/yaw.
- **Unmet report:** after solving, the allocator reports per axis what it could not make (`unmetRaw` → `allocUnmet`). The governor's pace window reads it.

2.7 Effector models

Every actuator the FCS can move is an `Effector` (see `Effector.cs`): travel `min..max`, current position `now`, command `cmd`, servo `rate`, current reachable box `lo..hi`, cost terms, and a **moment table** `M[N]` over a grid of deflections (roll, pitch, yaw moment at each grid point; linear in between). `Evaluate(ref EvalContext)` rebuilds the table every tick.

Effector class	What it models	Used on
`SurfaceEffector`	a game `ControlSurface`: its plate's force at its **local** AoA (from its own velocity, sideslip and dynamic pressure), with the game's plate equation and airfoil table, across its travel; stall- and reversal-aware; camber and downwash mirrors	all
`TvcEffector`	symmetric (pitch) nozzle angle, from live thrust of each operable engine; zero above the game's vectoring speed limit	all
`TvcDiffEffector`	differential nozzle angle about the symmetric one (roll TVC)	Ifrit
`TvcYawEffector`	nozzle yaw angle about the pitch angle	Revoker
`SplitEffector`	split deceleron: drag on one wing makes yaw ($F = 0.1 \cdot split \cdot \rho v^2$)	Ifrit (the class is in the Revoker too, but its stock surfaces have no split; the F-22E uses opposed flaperon pairs as decelerons)
`HingeFlapEffector`	a hinged trailing-edge flap that is *not* a game `ControlSurface` (driven directly by the FCS)	Revoker inboard elevons; Compass / Ternion / Su-30 inherit the class
`CantedTvc`	one nozzle swivelling about **one** axis in a plane canted 32° outboard (AL-31FP style); the pair's symmetric part is pitch, the differential is proverse roll + yaw	Su-30

Two pitch pairs on one jet (Ternion: canards + aft elevators) are both `Stabilator` pairs told apart by an `isCanard` flag, with their own trim costs (`ElevatorTrimCost`) and a roll-split cost high enough that the two pairs cannot sit in opposing roll splits (`StabilatorRollCost`, Ternion 0.1.6). The Su-30's canards are a pitch-only pair (`CanardSymmetricOnly`: a prohibitive split cost and the command averaged).

Details that matter when you add your own (§9.1):

- **Surface roles** (`SurfaceKind` : Stabilator, Flaperon, Aileron, Rudder, Other) come from `Classify` (name and range based). The role selects costs, fight groups, pitch handling and the estimator group.
- **Monotone pitch model for stabilators** (`linearPitch` , 1.4.0): the stab's pitch column is held monotone so the allocator never uses a stalled stab "backwards" for pitch (it used to park at +20–28° after high-AoA releases). The raw curve is still used for roll/yaw. **Reversed flow** (local AoA past 120°, hysteresis 110°) flips the monotone sense (tail slides).
- **`noPitch`** : flaperons, ailerons and rudders are modelled without their pitch moment unless a feature says otherwise (`RudderPitchAid` , Revoker `ElevonPitch`).
- **Pairs** are found by name (`X_L` ↔ `X_R`) with opposite roll (or yaw, for rudders) sign; that is where pair costs attach.
- **Camber coupling mirror:** where the game gets a camber force on the fixed part ahead of a surface (`CamberForce`), the surface's model adds the same term (`host` , `hostFoil` , K per surface).
- **Staggered stall mirror:** the surface's foil is the same remapped table the game was given (`StallStagger.TableFor`).

2.8 Pitch law (`PitchLaw`)

Modes (`FcsMode`):

Mode	When	Stick means
`Ground`	on the wheels, slow	ground handling; pitch for rotation via `GroundRateLaw`
`GearDownRate`	gear down in the air (and the runway rotation)	pitch-rate command, AoA/g limits
`AoAG`	gear up, Stability Assist on (normal)	the gear-up law, `PitchCommandLaw` (below)
`Rate`	`PitchRateCommandOnly` = true	pitch-rate command everywhere, limits still apply
`MPO`	Stability Assist off ("manual pitch override")	plain pitch-rate command, MPO g limits (+12.5/−3.5), no AoA limit
`AIRate`	AI pilot	pitch-rate command with the stock FBW's stick gain, FCS limits

`PitchCommandLaw` (gear-up, default `GAndPitchRate` since 2.1.0): the stick commands **pitch rate where full g is out of reach (low IAS) and g where it is in reach**, blended by βG (how reachable the g limit is within the pitch-rate budget). AoA is a *limit* only. `GAoAAndPitchRate` restores the older AoA/G law (AoA command at low IAS with a hand-over to rate command past ~36°). Pure AoA command was rejected because it tracks guns badly.

- **Released stick holds zero pitch rate.** Easing the stick targets the pitch rate the jet will sustain at the new state and reaches it through a jerk-limited deceleration, so the nose never reverses.
- **Pitch-rate budget:** `MaxPitchRate` (F-22E 60, stock jets 48 rising to `FullStickPitchRate` 60 at full stick); IAS-based at low AoA and thrust/weight-based otherwise (from the SimplePlanes design the law came from).
- **Command shaping** (`ShapeCommands` , 1.9.0): small AoA/g steps are built gently, large ones fast (pre-filter bandwidth 1.5 s^{-1} for a tiny step to 10 s^{-1} at 15° or 3 g).
- **Envelope:** `AoALimitPositive` (65 / 65 / 70), `AoALimitNegative` (50 / 60 / 55), `GLimitPositive` 9.5, `GLimitNegative` .
- **Kinematic capture:** the approach to a limit is braked using the nose-down authority the jet *really* has at that AoA (nozzles at live thrust), so it arrives without overshoot (`AoACaptureBrakeBlend` , `AoACaptureThrustBrake`).
- **Low-thrust cap** (`AuthCapStep` , every 0.2 s): sweeps the nose-down authority available from full nose-down stab, flaperons at trim and nozzles at present thrust; the AoA limit is set where that authority runs out (with

margin). Without it a full pull then push at idle went to 80° and didn't recover. With no TVC the caps are `NoTvcAoACapPositive/Negative` 45.

- **Protection:** past the limit the nose is driven down at the full rate. **`StickCancelsLimit`** (F-22E 2.44.0, on every jet with pitch TVC; never on the Compass / Ternion, §8.14): full aft stick at the limit holds the AoA with zero pitch rate once zero (or nose-down) is the only rate that can hold it; protection pushes the nose down only once the stick is released. Partial stick scales it by (1 − stick/0.9).
- **Asymmetric-load cap** (`AsymCap`): with uneven stores or damage, the AoA limit is capped where enough roll authority is left, and the stick is remapped onto the cap (§2.11).
- **Measured capture** (Su-30 0.1.15 / 0.2.0, `CaptureMeasured`, `GCaptureMeasured`): the approach to the AoA limit and to the g limit is planned on the **measured** pitch acceleration plus the reachable swing of each effector (with its own swing time and the command-path lag as the delay), instead of a modelled-braking blend with hand factors. `GCaptureReachable` (0.1.9) only brakes toward a g limit the jet can actually reach (`nMaxAero`), which removed a 10°/s pitch-command dip in a 450 km/h pull.
- **Soft AoA limit** (Su-30 0.2.0, `SoftAoALimit`, margin 3°): forced more than 3° past the limit (by a departure or a rolling pull), the jet is flown by the MPO law until it has been back inside for 0.25 s, and full aft stick can't pull it further past.
- **Jets without TVC** (Compass, Ternion): the inherited fixed no-TVC caps (`NoTvcAoACapPositive/Negative` 45) are off on a jet with no vectorable nozzle at all (`noVecJet`, Ternion 0.1.1), so its own AoA limits and the swept authority cap rule. The stick never cancels the limit on such a jet (`StickBypass`, Compass 0.1.2). In MPO it gets the full pitch-rate cap at every AoA (`MpoNoVecFullRate`, Ternion 0.1.8; before, a 40 × T/W term that is 0 without TVC held MPO to 14–24°/s at high AoA).
- **Gyroscopic budget** (Su-30 0.1.11, `GyroBudget` 0.25): below 130–170 km/h past 35–45° AoA the pitch-rate command is held so the gyroscopic roll (Iyy − Izz)/Ixx·q·r stays within a quarter of the roll authority (inverted corkscrews were tumbling). Since 0.3.5 the same budget is also spent on the roll (`GyroRollTrade`).
- **Soft pitch hold past the limit** (Su-30 0.3.10, `SoftPitchHold`): past the AoA limit the AoA law no longer pushes the nose down by itself; a released stick holds zero pitch rate, aft stick can't pull further past, forward stick still pitches. The full hand-over to MPO stays at +3°. Tom: "get rid of the 3 degs after overshooting AoA limit since that cause the jet to nose down". Switching Stability Assist back on past the limit after MPO doesn't trigger a recovery either (`SoftMpoFromMpo`).
- **Pitch-rate ceiling 60°/s** on the F-22E, Revoker (1.4.1) and Su-30 (0.5.7) (`MaxPitchRate`; Ifrit, Compass, Ternion 48 with a full-stick ramp to 60). Raising the ceiling changes partial stick and pushes, not full aft: on the Su-30 bench 0.7 stick at 450 km/h went from 29 to 64° AoA.

2.9 Lateral-directional law (`LateralLaw`)

- **Velocity-vector roll.** The stick commands a roll about the flight path (stability axis, rate pS), not the nose, so the jet rolls around its velocity vector without building sideslip. Coordinating it needs a body yaw rate r = pS·sin α, so above ~30° AoA the roll is mostly a yaw.
- **Coupled axis.** Below `CoupledAxisAoA` (36°) the pedals command sideslip (`PedalSideslip` 15° at full pedal); above it the pedals blend into the stick's velocity-vector roll (fully by +14°): one lateral axis, as on the real F-22 past 36°.
- **Sideslip loop** (`SideslipGain` 1.3 /s): the yaw axis drives sideslip to its command (0 with pedals centred).
- **Roll-rate ceiling:** from an online estimate of roll damping and usable roll moment, times `RollRateGainLowIAS/HighIAS` against the original aircraft's roll rate, capped at `MaxRollRate` 250; yaw rate capped at `MaxYawRate` (75 / 75 / 90).
- **Low-IAS yaw boost** (`LowIasYawBoost`): the falling-leaf yaw-rate requirement (30–36°/s in deep stall).
- **MPO** (Stability Assist off): on the stock jets `MpoRollMode` picks separate roll / yaw axes (default) or velocity-vector roll; the F-22E's MPO is a plain rate command with no AoA limit.
- **Rudder fade** (`RudderFadeStartAoA/EndAoA`, 50–60° on the F-22E): only on jets whose rudders lose yaw purpose at high AoA (the F-22E's canted fins produce inverse roll there). It is *not* tail blanking; NO does not

model blanking.

- **Thrust-borne yaw** (Su-30): canted nozzles keep their yaw as the jet slows, so the high-AoA yaw floor's IAS-decay anticipation scales only the aero share (`YawThrustShare`, 0.1.6); the allocator's velocity-roll weight rises ×4 from 25 to 45° AoA while the nozzles have thrust and the roll stick is in (`HighAoAVvRollWeight`, 0.1.1); `FallingLeafYawBoost` 1.2 raises the yaw-rate limit in the falling leaf (0.2.0).
- **Low-speed TVC roll** (Su-30 0.1.11–0.1.16, `LowSpeedTvcRoll`): when the airframe has almost no sideslip stiffness left (the probe's `slipStiff` under 0.2–0.4 °/s² per degree, past 25–35° of total flow angle, or flying backwards), the sideslip hold and roll governor fade out and the roll stick turns the jet about the canted nozzles' axis directly. Always on in MPO.
- **Stop past the sideslip margin** (Su-30 0.2.0, `StopIgnoresSlipOutsideMargin`): with the stick released and the sideslip already past `RollStopSideslip`, roll and yaw both brake at full; the roll isn't held back for the yaw.
- **Engine-out sideslip trim** (Ternion 0.1.12, `EngineOutSlipTrim`): engine out, controls centred, |AoA| < 25°: the steady sideslip is learned (τ 2 s, max 4°) and removed from the sideslip hold, so the jet flies straight instead of creeping 1–2°/s.
- **Flying backwards** (all six since 12 Nov, `ReversedFlowNoseAxis`): past 90° AoA the velocity-vector roll's body-roll share is |cos α| instead of cos α (yaw stays sin α), so left stick or left pedal always turns the nose left, on one axis blended by AoA: pure yaw at 90°, half and half at 135°, pure roll at 180°. It replaces a sense flip at 100–150° that reversed the yaw with the roll (§8.24).
- **Loaded roll reversals** (all six since 10–11 Nov): with an asymmetric load a reversal flies one roll acceleration from the stick flip through the zero crossing, built over the time the rudders need to cross (§8.25).
- **Hard stop on a reversal** (Su-30 0.5.7, `StopHardOnReversal`): lateral stick against the velocity roll in progress counts as a release for the hard stop: "stop the roll or yaw even if that requires the sideslip is outside the margin".
- **Free roll in MPO** (Ifrit 1.1.2, `MpoFreeRoll`): with Stability Assist off and separate roll / yaw axes, the roll stick's body roll is limited only by what the roll axis can do, not by sideslip.

2.10 The reference-capacity governor

Plain English: **a governor slows a command when one axis can't keep up with the others.** At high AoA a velocity-vector roll is mostly a yaw, so if the yaw can't follow, the jet builds sideslip and the roll "wobbles" or departs. Earlier versions had a dozen special devices for specific manoeuvres. The current design (F-22E 2.27 → 3.0.0, ported to the Ifrit 0.9 and Revoker 0.6) replaces them with one input and a few consumers:

Capacity estimator (`CapacityEstimator.cs`, runs after the allocator; the laws read it on the next tick). For each axis (roll, yaw, the two-axis velocity-roll direction, and on the stock jets the **sideslip axis**) and each side (+/−), the angular acceleration the jet can still produce:

capacity = measured ω̇ (filtered) + committed increment + allocator probe

- *measured*: what the airframe and the effectors are doing now (passive aero included);
- *committed*: the table increment from `now` to this tick's command, counted only as far as each servo gets within `CapHorizon` 0.15 s;
- *probe*: the real `Solve` run again from this tick's commands with a large extra demand (`ProbeZ` 400 deg/s² of yaw, `ProbeX` 1500 of roll, `ProbeS` / `ProbeB` 800 along the two-axis / sideslip directions), read per **effector group**, minus the allocator's unmet; then the commands are restored.
- *groups and gains*: F-22E: rudders / stabs / other surfaces, gains 1.0 / 0.7 / 1.0 (yaw) and 1.0 / 0.65 / 1.0 (roll), identified from flight logs. Stock jets: **six groups, every effector counted** (rudders, stabilators incl. canted tails and canards, other surfaces, TVC of any kind, splits, hinged surfaces), gains 1 until fitted.
- *rudW*: how much of the yaw can be added without spending roll. On the F-22E it is the rudder fade × fin health; on the stock jets it is derived from the yaw-only probe's roll collateral (1 − |roll collateral| / |yaw added|).

Consumers:

- **The trade** (rolling pull): when pull + roll ask the yaw for more than the capacity, 70 % of the shortfall comes off the roll target and the rest off the AoA rate. The trade reads the yaw capacity through a low-pass (`TradeCapTau`): 0.05 s suits the F-22E's rudder transit, but on the Su-30 the effector carrying the yaw is the canted nozzle pair (0.46 s across its travel), so 0.45 s there (0.1.4) removed a pitch-rate hitch in rolling pulls. *Settings:* `PullRollTrade` 0.7, `TradeCapShare` 0.7, `TradeCapTau`
- **Steady roll-rate cap**: caps the *commanded* roll rate where yaw can't stop the roll inside the sideslip margin (reversal swing model). *Settings:* `RollPaceRateBlend` (F-22E 0.408, Ifrit 0.4, Revoker 0.65), `RollStopSideslip` (4 / 6 / 6)
- **Pace window**: holds the roll target back only while the allocator reports it can't deliver the asked-for yaw (latched on unmet, lead 1/KYaw). *Settings:* `PaceSatFrac`, `PaceLeadT`
- **Released-stop yaw brake**: on a released roll, brakes the yaw on a profile shaped to the rudders' unwind time (snap-roll nose wobble). *Settings:* F-22E 2.31.0
- **Push plan**: a push-over from high AoA is planned once at its start: yaw shed rate capPlan = clamp(K0 + K·IAS^1.5, 6, 80) deg/s² (IAS in m/s), push time T = (r − rLeft)/capPlan with rLeft = √(2·capPlan·slip), pitch rate follows the plan; the released roll bleeds over T; under a plan it owns pitch. *Settings:* `PushPlanK0` / `PushPlanK`: F-22E 0 / 0.028 (93 logged push-overs), Ifrit 9.6 / 0.0144, Revoker 40 / 0 (a constant); `PushPlanSlip` / `PushoverSideslip`
- **Push-over rate floor** (F-22E 3.1.4/3.1.5): the rate a planned push is held to is max(plan rate, `PushOverMinRate` 23°/s) above 10° AoA; at very low airspeed the IAS model had planned 9°/s pushes that felt frozen. Tom's rule: "let the yaw/roll do its best during that pitch down, some more sideslip is ok". A sideslip guard (3.1.4) was removed in 3.1.5 because it made the push slow-then-fast. Ported to the Ifrit but **off** there (its rule is "no more sideslip than 0.15.0"). *Settings:* `PushOverMinRate` (CSV `ppFloor`); `PushPlanRise` 40°/s² (3.1.8: the plan's rise happens inside the onset instead of creeping after it)
- **Push governor**: paces the pushes the plan doesn't take (damaged fins, low yaw rate): slows the AoA sweep toward zero to what the yaw sheds; never turns the nose back. Since 3.1.6 its sweep cap **fades in** with the velocity roll (5 → 15°/s) instead of switching at 5°/s, and never holds the nose's body pitch rate under the 23°/s floor (scaled by how hard the pilot pushes). Since 3.1.7 the hand-offs between plan and governor are smooth: the toward-zero side stays open after a plan finishes (`PushPlanDoneOpen`), caps are read for the AoA side they were sized for (`SweepSideKept`), the pace fades in with the excess yaw (`PushGovPaceYaw`), the far-side cap fades in from 20–40°/s of roll (`PushGovFadeGrow`), a held push never loses rate it already has (`PushNoRetreat`), and the plan's ramp-out continues through zero AoA at 30°/s² (`PushPlanThrough`). *Settings:* `PushGov*`, `PushGovFadeLo/Hi`, `SweepFloorPushMin`; the 3.1.7 switches are code statics
- **Fin-loss devices**: scaled by fin (or tail) health, inert on a clean jet. *Settings:* `FinLossAoACap`, `PaceNoRetreat`, …

Governor rules that every change has respected (from Tom's flights; see §14):

- it slows the **commanded rate** only, never the roll acceleration or deceleration;
- at low–mid AoA, where yaw can't catch up with roll, slow the velocity-vector roll; at high AoA, where the airframe's passive lateral stiffness keeps roll and yaw together, slow the **pitch** instead;
- the only acceptable failure is sideslip from gravity pulling the jet sideways.

2.11 Damage and failure tolerance

The FCS assumes nothing about which parts survive. In layers:

1. **Asymmetric loads** (`AsymStep` / `AsymCap`, every 0.2 s): as the measured asymmetry grows, the AoA limit is capped and the stick is remapped onto the cap, keeping enough lateral authority for the pilot. Roll limits use the authority left in each direction after the damage trim (`AsymRollHeadroom`). One-wing-missing departures on the bench went from 29/160 to 0/160.
2. **Lost yaw authority** (`YawHealthStep`): with a fin gone, roll and yaw limits scale with the share of yaw effectors left, and the yaw target is clamped to what the remaining effectors can reach (before that, the

allocator thrashed a stab at 0.6 Hz chasing unreachable yaw).

3. **Decelerons:** past the rudders' reach, opposed flaperon pairs (F-22E) or split decelerons (Ifrit, Revoker) make drag yaw.
4. **Fin-loss AoA cap** (`FinLossAoACap`): the finless F-22E is directionally unstable from ~10° AoA; the best held-roll band was 35–45°.
5. **Engine out / damaged engine:** `EngineOutAoACap`, `EngineDamagedAoACap`; **thrust match** (`EngineAbAoAGate`): with one engine down on thrust, the healthy one is held to the damaged one's thrust until the AoA comes down, pre-emptively and proportionally, aware of the engine-spacing moment arm.
6. **VMC limiter** (`VmcApply`, last resort): when yaw is saturated and sideslip runs away with an engine out, the throttle is pulled back. The clamped-away yaw demand counts as saturation (F7 in §13).
7. **Fuel balance and trim** (`FuelBalance`, `FuelTrimStep`): moves fuel between healthy tanks to hold the CG where the clean jet has it; when yaw/roll authority runs short it moves the CG sideways on purpose (toward the live engine / against the asymmetry). Mass is conserved; destroyed tanks create no fuel. On the Su-30 it also holds the fore/aft CG at the 70 %-fuel point down to about 10 % fuel (`FuelCgHold`, 0.2.1; stock the CG walked ~30 cm aft from full to empty).

Added since the first kit:

8. **Faster asymmetric cap after a hit** (`AsymCapGapTighten`, F-22E 3.0.3 / Compass 0.1.9): the cap's tighten rate (10°/s) is multiplied by gap / 3°, 1–4×, so a pull right after a part is shot off can't outrun it.
9. **No-TVC asymmetric protections** (Compass 0.1.5–0.1.8, carried by the Ternion and Su-30): a roll governor on the stabs' nose-down margin (`StabRollGovernor`); the capture plans its stop on half the modelled nose-down (`AsymCapBrake`); the nose-up the cap's model misses is **measured** in flight and taken off the margins (`AsymPitchBias`, cap floor 7°); these protections open when a control surface is lost, not on a lateral-CG reading that swings with a hanging rudder (`AsymDamageGate`). `LiveAuthCap` (Ternion 0.1.10): the AoA limit sits where the measured nose-down authority extrapolates to 60°/s².
10. **No-fin rules only where they belong** (F-22E 3.0.2): the no-fin stop (`PaceStopNoFinGate`) acts only for a roll being stopped and where the window is on (Unity's `Sign(0)` is +1, so it used to see a right roll in straight flight); the release coordination (`ReleaseCoordGate`) only above ~15° AoA with more than 5°/s of real rotation.
11. **Engine out on fixed-nozzle jets** (Compass 0.1.11 `VmcPredict`; Ternion 0.1.9–0.1.12): the live engine's thrust moment is taken as it is, and the throttle comes back once its yaw passes 70 % of the yaw authority at the predicted q. Above the match AoA (`EngineOutMatchAoA` 35, latched) the live engine is matched to zero thrust (`EngineOutZeroThrust`).
12. **Automatic spin recovery** (Ternion 0.1.9/0.1.12, `SpinRecovery`): AoA > 40°, |r| > 15°/s for 0.8 s with lateral controls centred **and the yaw saturated** → nose-down on a rate target scheduled on AoA, full yaw against the rotation, roll damping, throttle idle; ends at AoA < 30° and |r| < 10°/s.
13. **Parts that leave with the wrong parent** (Ternion 0.1.10, `StructureFix`): the game hangs every part off one transform parent; where a tail section is a child of an engine but jointed to the airframe, a destroyed engine took the tail with it. The part is re-homed to its jointed parent.
14. **The coupled asymmetric-load check** (Ternion 0.2.2 → Compass 0.4.0, Revoker 1.2.0, Su-30 0.5.0, Ifrit 1.2.0; ported to the F-22E 4.2.0 but **off** there): with a lateral CG offset, at each sweep AoA a small linear programme (LP) over every effector's moment points finds the flight-path roll left each way with the load held, pitch held and no sideslip acceleration. The AoA cap is the lower edge of the top over-threshold stretch (`FindTop`) of the demand curve, at `AsymCoupledReserveMax` of the roll range (0.7–0.8, per jet). The nose-down limit reads the same LP's held-pitch figure. It replaced the one-axis check, which froze the surfaces at their present deflection so its answer moved with the stick (Su-30: 12–75° with 42 jumps over 10° in 0.5 s; Compass: 7–12° where the jet held 25–30°). Full story in §8.23.
15. **Roll toward a load** (Su-30 0.5.1–0.5.2): the roll the stick may ask toward each side is what that side's reserve can stop in `AsymStopTime` (0.8 s); the AoA limit sits where AoA × usable roll is largest (`CplKnee`, `AsymAoAWeight`).

2.12 Ground, airbrake and high-lift

- **Ground steering** (`GroundSteer`): stock steering while the pedal is held; when released the FCS stops the yaw and holds a straight line (an INDI wheel law, `GroundIndi`), with a small controlled drift allowed at high speed (`GroundDrift*`). Since 11–12 Nov (Su-30 0.5.3–0.5.4, then all six) the **main brakes help**: the nose tyre takes the wanted yaw up to its grip, the outside main brake takes the excess (`GroundIndiBrake`), and with the pedal in, the main on the pedal's side gets |pedal| × 0.5 (`GroundPedalBrake`) so the jet can be drifted on the runway (§8.26).
- **Runway pitch:** on the runway the gear-down rate law flies pitch (`GroundRateLaw`), no separate rotation law.
- **Airbrake** (`ThrottleIndependentAirbrake`, default on): only the brake control deploys it, at any throttle, in the air and on the ground; idle throttle never deploys anything. Turning it off returns the controls fully to vanilla behaviour. In the air the brake fades with its share of total drag (`AirbrakeAoAFade`, `AirbrakeFadeShare*`) so it does not steal lateral authority at high AoA. On the ground there is a rollout aero-brake configuration per jet (`GroundAeroBrake*`).
- **High-lift** (`FlapSystem.cs`): flap/slat schedule, deceleron droop, and on the stock jets the FCS takes over devices the game's `HighLiftDevice` would move, so they can be flown as part of the allocation. Compass: the inner flaps don't move with the pitch stick (`FlapPitchLock`) and their droop comes off with roll demand (`FlapRollBias`); Su-30: the gear-driven landing flap comes up in the air with lateral stick (`FlaperonRollPriority`). Since Su-30 0.5.5 the Su-30's flaperons run the F-22E's flap schedule at any gear state (`FlaperonFlapSchedule`: the gear no longer decides the flaps), Tom's standing rule for every jet whose flaperons do both jobs: roll priority.
- **Ground TVC** (Su-30 0.2.3, `GroundTvcRollYaw`): on the ground the canted nozzles' differential follows roll stick + pedals as well as the pitch stick.
- **Thrust line** (Su-30 0.1.5, `ThrustLineAtCg`): the Su-30VXN's nozzles sit 0.49 m below the CG, so thrust pitched the jet onto its tail on the take-off roll and changed trim with throttle. The thrust force point is moved to the CG height (and the FCS's nozzle models use the same arm); this replaced a closed-loop nose-wheel hold (0.1.1–0.1.9, deleted in 0.1.10).

2.13 AI aircraft

AI-flown jets get the full FCS with a pitch-rate command law (`AIRate`) using the stock fly-by-wire's stick gain (read from `ControlsFilter.flyByWire`), and the FCS limits. `AIFlightControl` = false gives the AI the stock FBW. **After the pilot ejects** the FCS disengages and the stock FBW takes over (F-22E 3.1.0, all jets since 9 Nov); before, the empty jet looked like an AI jet and the FCS kept flying it. The FCS only attaches to aircraft simulated on this machine (`aircraft.LocalSim`): your jet, and AI jets if you are the host. Other players' jets are run by their own machines.

2.14 Multiplayer policy (`MpPolicy.cs`)

Mods that change a vanilla flight model must not give an unfair advantage in multiplayer (the NOMNOM registry rejected an early version for that). The rules:

- the FCS (and its airframe changes) only work if the **host** runs the plugin and allows it (`HostAllowFcs`, read on the host);
- `HostLockVariant` (default on): every client gets the host's performance profile and countermeasure/ammo capacities;
- a client whose host does not run the plugin flies the stock aircraft. Keep this module in any port you publish.

2.15 FcsApi: the discovery contract

`FcsApi` is a public static class that HUD and visualizer mods find by reflection in *any* loaded plugin, so they don't need an update for every new FCS (contract `ApiId = "duythinh2022.fcs"`, `Version` 4 on the Revoker, 3 on the others). Members: `Owns(Aircraft)`, `GetController(Aircraft)`, `GetPitchCommand(Aircraft, float[10])`

(mode code, AoA/g commands, pitch-rate target, AoA, nz, protection, unload, limits). Keep the class name and members in your port (change `FcsName`), and the Flight Data Readout HUD and FCS Visualizer will show your jet's FCS state.

2.16 Running on several cores

Since F-22E 3.0.4–3.1.3 (5–6 Nov), ported unchanged in behaviour to the Ifrit 1.0.0, Revoker 1.0.0, Compass 0.2.0, Ternion 0.2.0 and Su-30 0.2.0, the heavy parts of a tick run on other CPU cores. **None of it changes what the jet does:** every port was checked byte-identical on its bench (suite files, replays, the game CSV) with all switches on and all off.

Work	Dev switch ("0. Debug", default on)	How it stays identical
Capacity estimator's 6–8 probe solves	`EstimatorThreaded`	runs on copies of the effectors and allocator (`EffectorMirror`), kicked at step 16 of tick N and joined at the top of tick N+1; the laws already read the previous tick's estimate
Effector table rebuild	`ParallelTables`	`EvalGather` reads the game's objects on the main thread; `EvalCompute` (pure maths) runs on helpers; the sum into the total moment keeps its order
Allocator reach plan (third solve of the two-horizon pick)	`ParallelAllocator`	depends only on the commands from before the solve; started on a mirror only if the previous solve needed the pick
Asymmetric-load AoA sweep	`ParallelSweep`	game state read once per sweep tick (`SwGather` ; hinged surfaces via `Snap` / `EvalFlowSnap`), each point writes only its own array slot
Lift envelope and hybrid slopes (every 0.1 s)	`ParallelEnvelope`	one gather pass; per-point sums keep the original part order
CSV formatting and file writes	(always, player jet only)	the main thread boxes the values in the old order; a writer thread formats them

Rules that make several FCS mods and several jets behave together (`ParallelFcs.cs`):

- one shared pool: the process-wide .NET thread pool, no private threads (the CSV writer is the one exception);
- each jet uses at most `FcsPool.Helpers` helpers (a quarter of the logical cores, 1–3);
- the main thread always takes part, and a job no helper has started is run inline by whichever thread needs it, so a busy pool can delay the speed-up but never stalls a tick;
- nothing touches a Unity object off the main thread (the quaternion maths used is marked `IsThreadSafe` in the game's `UnityEngine.CoreModule`).

Measured in game (F-22E 4.1.3, 10 Nov 23:48, switches turned off one by one in one flight): everything on 470 µs per tick (median 444, p95 640); the threads alone saved about 240 µs of it (tables 44, sweep 20, estimator 172). With telemetry off 420 µs. The CSV row was most of the garbage (17 KB → 1 KB per tick) and every 3 ms hitch; telemetry is off by default.

`TickTimerLog` (default on) writes the player jet's time per tick (mean / median / p95 / max), its split by phase, all AI jets together and the airframe force hooks to `LogOutput.log` every 10 s; the CSV carries 17 timer columns (`fcsUs` … `tVmcUs` , §16). Bench timing on a 2-core VM: F-22E 0.72 → 0.43–0.50 ms, Ifrit 1.9 → 0.9–1.1, Revoker 2.3 → 0.9–1.0, Su-30 2.05 → 0.79.

When porting this to a jet of your own, the gap that bit the Ifrit port: **every per-tick allocator setting must be copied to the worker copies** (the stock jets' slip-frame weighting and settled flag); without them the estimator's columns, then the pitch and roll commands, differed.

2.17 Fuel burn and other mods' hooks

The performance profile scales fuel burn (`FuelConsumptionScale` , Buffed 0.65 Ifrit / Revoker / Su-30, 0.75 F-22E / Ternion / Compass). Since 7 Nov (F-22E 3.1.9, Ifrit 0.17.1, Revoker 0.8.3, Compass 0.1.12, Ternion 0.1.7) the fuel hook is **two prefixes** on `Aircraft.UseFuel` :

- `UseFuelScalePrefix` at `Priority.First` scales the amount for this jet only and draws nothing;
- `UseFuelPrefix` at `Priority.Last` draws from the attached tanks, but only if no earlier prefix already drew (`__runOriginal` false).

Why: the Aryx Weaponry Expansion has its own prefix on the same method (drain external tanks first, then skip the original). HarmonyX runs every prefix even after one returns false, so both drew: 1.65× stock burn on the stock jets, 1.5× on the F-22E (§19). Any port that changes fuel burn should keep this two-prefix shape.

2.18 The native (Rust) core

Since F-22E 4.0.0–4.1.4 (10–11 Nov), and on the other five since 11 Nov, the heaviest pure maths runs in a Rust library, `<Jet>_FcsCore` (`F22E_FcsCore` , `Ifrit_FcsCore` , `Revoker_FcsCore` , `Compass_FcsCore` , `Ternion_FcsCore` , `Su30_FcsCore`). **The C# is the reference; the library must return the same bits** wherever C# float arithmetic is strict 32-bit.

What runs in Rust	C# entry point	Dev switch ("0. Debug")
the allocator's solve (main solves and the estimator's probes)	`Allocator.NativeSolve`	`NativeAllocator` (0 C#, 1 only where bit-identical, **2 native**, default)
the surface-table grid kernel: one surface's moment at every deflection in one airflow	`SurfaceEffector.GridMomentsNative`	`NativeTables`
the asymmetric-load sweep's capacity loop (min / max roll, yaw, pitch per point)	`SurfaceEffector.GridRangeNative`	`NativeSweep` (needs `SweepGridCache`)
the coupled check's LP (§2.11 item 14)	`FcsNative.Cpl`	`NativeCoupledCap`

- **What stays in C#:** the control laws, every read and write of the game's objects, and the rotations (Unity's `Quaternion.AngleAxis` / `Inverse` are Unity's own native code; the kernels get pre-rotated geometry). The kernels call nothing from a maths library but `atan2` , written as C# writes it (`(float)Math.Atan2((double)y, (double)x)`).
- **One DLL** (since F-22E 4.1.4, the others 11 Nov): the core is a resource inside the plugin DLL. On first load it is written to `BepInEx/cache/<Lib>/<first 16 hex of its SHA-256>/<Lib>.dll` and loaded by full path (a loose `<Lib>.dll` beside the plugin is the fallback). Windows binds a `DllImport` by module name, which is why every jet's core has its own name.
- **Safety:** at load, a self-test runs the kernels and the LP against C# twins; a wrong ABI number, a wrong structure size, a failed test or an error return switches that part back to C# for the session, with one warning in the log. With the DLL missing the whole FCS runs in C#.
- **The game's float mode.** Unity starts Mono with `-O=-float32` , so in the game every C# `float` expression is evaluated in 64-bit and rounded only when stored. The bench (.NET 8) and Rust round every operation to 32-bit. So in the game the Rust solve gives the *bench's* bits, not the game's C# bits: measured on the game's own Mono under Wine over 26,700 recorded solves, 7.1 % identical, 68.4 % within 0.001°, 24.2 % equal-cost alternatives, 0.3 % different. This was always the gap between bench and game; `NativeAllocator = 2` closes it for the allocator ("what the bench verifies is what flies").
- **Speed** (F-22E): 137.5 µs per C# solve against 16.4 µs native on the game's Mono (8.4×); the grid kernel 4.6–4.7×. In game, everything native + threads 470 µs per tick against 1,862 µs all C# on one thread (4×), same flight.
- **The coupled check's cost**, native LP + per-tick grid geometry + one side solved per sideslip case: the loaded sweep 37–52 % cheaper (e.g. Ifrit 362 → 173 µs per tick on the bench).

Rules for changing it (from `native/README.md` in each tree): any change to `Allocator.cs` (`SolveInner`, `ReachFrom`, `SolveCore`, `PairedSearch`, `SolveLinear`, the cost terms), `Effector.MAt`, `SurfaceEffector.Eval*`, `Camber.Foil`, `AirfoilTable.Get`, `Effector.ToAero` or `CoupledCap.cs`'s solve must be made in `native/fcs_core/src/` as well, expression for expression: same operand types, same order, same float / double conversions, no fused multiply-add, no reordering. A new field goes into the shared structs on both sides and the `ABI` number is bumped on both. Then the bench must show zero differences with `NativeAllocatorCheck = 2` (every solve solved both ways). See §15.3 for building the library.

3. The six reference jets side by side

The jets were built one after another, each forked from the one before it: **F-22E → Ifrit → Revoker → Compass → Ternion → Su-30**. Each fork kept the laws and changed the effector set, the airframe changes and the tuning. So the newer jets carry everything the older ones learned, plus their own pieces.

The three fighters with TVC from the first kit:

	F-22E Strike Raptor (4.3.0)	KR-67 Ifrit (1.3.0)	FS-12 Revoker (1.4.1)
Aircraft match (`Registry.IsF22`)	prefab `Aryx_KingRaptor*` or name contains "Strike Raptor"	prefab `Multirole1*` or "KR-67"/"Ifrit"	prefab `Fighter1*` or "FS-12"/"Revoker"
Pitch effectors	2 all-moving stabilators (`Stabilator`), pitch TVC	2 canted all-moving tails (`Stabilator`, `elevator_L/R`), pitch TVC	2 canards (`Stabilator`), outer elevons (`Aileron`, `ElevonPitch`), inner hinged elevons, pitch TVC
Roll effectors	outer + inner flaperons, stab differential	ailerons, decelerons (`Flaperon`), tail differential, **roll TVC** (`TvcDiffEffector`)	outer elevons, inner hinged elevons (`HingeFlapEffector`), canard differential
Yaw effectors	2 canted rudders, stab differential, opposed flaperons as decelerons (fin loss)	**tail differential is the yaw axis** (no rudders), split decelerons (`SplitEffector`), roll TVC on the sideslip axis	1 rudder, **yaw TVC** (`TvcYawEffector`), canard differential
Engines	2, pitch TVC ±20° (`TvcPitchRange`)	2, 2D nozzles: pitch ±15°, roll (differential) ±15°	1, pitch ±15°, yaw ±15°
Surface ranges (code defaults)	stabs +35/−30 TE, rudders ±35, flaperons ±25	game's own; flaperons ±30	canards +60/−45 (TE up/down), elevons ±30, inner elevons ±25
Servo speed	100°/s (`ActuatorSpeedScale` 1.43 × Aryx's 70)	100°/s (`SurfaceServoSpeed`)	100°/s
AoA limits +/−	65 / 50	65 / 60	70 / 55
Max pitch rate	60°/s; planned push-overs held to ≥ 23°/s	48 → 60 at full stick	60°/s (1.4.1; was 48 → 60)
Max yaw rate	75	75	90
Airframe aero changes	stall stagger, lift reshape (`LiftShape`), body-lift balance **plus the 3.2.x fore/aft lift schedule (relaxed to neutral, §8.15)**, stab lift peak 36° (was 25), stab downwash 0.6, fin + flap camber, effectiveness ×1.2 on flaperons, stabs and LERX	flap/aileron camber (K 0.46 / 0.57)	elevon stagger (inner 26 knee-preserving, outer 45), camber
Airbrake	Aryx's speedbrake split in the air; ground aero-brake configuration	split decelerons	dedicated airbrakes + trailing-edge brake config, canards down on the runway
Governor specifics	estimator: 3 groups, fitted gains; `RollPaceRateBlend` 0.408, `RollStopSideslip` 4, `PushPlanK` 0.028	6 groups, gains 1; rudW from probes; `SlipFrameAlloc`; `TvcYawOffload`; blend 0.4, stop slip 6	6 groups, gains 1; rudW from probes; `SlipFrameAlloc`; blend 0.65, stop slip 6, `PushPlanK0` 40
Asymmetric-load AoA limit	one-axis check (the coupled check is ported but **off**, `AsymCoupledCheck`)	coupled check, share 0.75, replaces the one-axis check (1.2.0)	coupled check with both nozzles counted (1.2.0), also with nozzles locked / low thrust (1.3.0), share 0.7 / 0.8

	F-22E Strike Raptor (4.3.0)	KR-67 Ifrit (1.3.0)	FS-12 Revoker (1.4.1)
Native core	`F22E_FcsCore` (exports `f22e_fcs_*`)	`Ifrit_FcsCore` (`ifr_fcs_*`; two-stage settled solve)	`Revoker_FcsCore` (`rev_fcs_*`)
Unique extras	cosmetic nozzle re-rig and vent doors	`MpoFreeRoll`	render-only hinge visuals

The three added in kit 2.0:

	T/A-30 Compass (0.5.0)	FS-3 Ternion (0.4.0)	Su-30VXN Gandiva (0.6.0)
What it is	stock trainer / light attack; forked from Revoker 0.8.2	Blueprinter mod (prefab `P_Trisurface1`); forked from Compass 0.1.11	kyleisred's Blueprinter mod (`SolarSu30.dll`); forked from Ternion 0.1.8
Aircraft match	prefab exactly `trainer` (or a clone), or "T/A-30"/"Compass"	prefab `P_Trisurface1` (or a clone), or "Ternion"	aircraftName "Su-30VXN" / "Gandiva" only (other Su-30 mods are left alone)
Pitch effectors	tail slabs (`elevator_L/R`, `hstab_L/R`) ±30 on a swept-slab curve (`StabSlabFoil`); inner flaps only when a slab is lost	**canards** (60 LE down / 45 LE up) **and aft elevators** (+35/−30), both `Stabilator` pairs (`isCanard`)	stabs +35/−30 on the F-22E's stab curve (`RaptorStabFoil`) with 0.6 downwash; canards pitch-only (+65/−45); **canted nozzles, symmetric part**
Roll effectors	ailerons, flap differential (droop comes off with roll, `FlapRollBias`), slab differential	flaperons (inboard, stall 26) + rollervons (outboard, 45), staggered, camber-coupled	flaperons ±25, stab differential, canted nozzle differential
Yaw effectors	rudders	twin rudders ±35	rudders ±35, **canted nozzle differential** (proverse roll + yaw)
Engines	2, fixed nozzles (no vectoring transform)	2, no TVC	2 AL-31FP-style single-axis nozzles ±15° canted 32° outboard (`CantedTvc`)
AoA limits +/−	35 / 30	55 / 40	75 / 65 (past the limit: soft pitch hold; +3° → MPO law)
Max pitch rate	48 → 60 at full stick	48 → 60 at full stick	60 (0.5.7)
Max yaw rate	90	90	90 (`FallingLeafYawBoost` 1.2)
Airframe aero changes	strake (LERX) Polhamus vortex lift ×3.4 (`StrakeVortexLift`), CG 0.15 m aft, tail downwash 0.45, camber	canards ×1.1, elevators ×1.2 stalling at 30°, shoulder LERX ×1.13, Revoker-style staggered trailing-edge stall, no downwash	high-AoA drag reshaped to halfway between the Ifrit and Revoker (`HighAoADragShape`), thrust line moved to the CG, CG 0.10 m aft, canards ×1.6, LERX ×1.5, aft body lift ×0.5, the 1.0.0 mass layout restored on 1.0.1 (`Su30CgReference`)
Damage / engine-out specifics	`VmcPredict`, `AsymPitchBias`, `StabRollGovernor`, `AsymDamageGate`	`StructureFix` (tail stays when an engine dies), engine-out zero-thrust match, spin recovery, `EngineOutSlipTrim` (`LiveAuthCap` off since 0.2.2)	inherits the Ternion's; `AsymStopTime`, `CplKnee`
Asymmetric-load AoA limit	coupled check (0.4.0), share 0.75, thrust about the offset CG in the load	coupled check (0.2.2, its origin)	coupled check with each canted nozzle as an effector (0.5.0), share 0.8
Native core	`Compass_FcsCore` (`cmp_fcs_*`)	`Ternion_FcsCore` (`tern_fcs_*`)	`Su30_FcsCore` (`su30_fcs_*`)
Governor specifics	blend 0.65, stop slip 6	blend 0.45, stop slip 6	blend 0.4, stop slip 10, `TradeCapTau` 0.45, low-speed TVC roll, `StopHardOnReversal`

	T/A-30 Compass (0.5.0)	FS-3 Ternion (0.4.0)	Su-30VXN Gandiva (0.6.0)
Unique extras	trailing-edge brake set	trailing-edge brake set; stock split airbrake / split yaw off in flight	ground TVC on roll + pedals, `FuelCgHold`, `GyroBudget`, `SoftPitchHold`, `HighAoAPitchBump`, flaperon flap schedule at any gear state

Performance profile (Buffed): thrust ×1.08 (Su-30 ×1.15), fuel burn ×0.75 (Ifrit / Revoker / Su-30 ×0.65), parasitic drag ×0.885–0.90, flares ×10, gun ×2, EW ×1.5 / ×1.2. Stock = the original aircraft's numbers.

`docs/CONFIG_REFERENCE.md` has every key (492 of them) with all six defaults; `docs/MODULE_MAP.md` lists every file with its size on each jet and how far each jet has moved from the one it was forked from.

Which reference to start from:

Your aircraft has...	Start from	Why
a conventional tail with rudders, all-moving stabs, flaperons, pitch-only or no TVC	**F-22E**	the most-flown law; rudder handling, decelerons from flaperon pairs
no rudders (V-tail, canted ruddervators), or roll/differential TVC, or split decelerons	**Ifrit**	the "tails are the yaw axis" solution; roll TVC; splits; the stock-jet governor
canards, a delta wing with elevons, a single fin, yaw TVC, or hinged surfaces that are not game `ControlSurface`s	**Revoker**	the most general effector set
no TVC at all, a conventional tail, strakes / LERX	**Compass**	the no-TVC damage protections (`AsymPitchBias`, `StabRollGovernor`), fixed-nozzle engine-out (`VmcPredict`), strake vortex lift
two pitch surfaces (canards **and** a tail), or a Blueprinter jet, or a twin with no TVC that must survive an engine out	**Ternion**	two `Stabilator` pairs priced so they don't fight; `StructureFix` ; zero-thrust match; spin recovery; MPO without TVC
single-axis or canted nozzles, thrust line off the CG, or a jet that must roll about its nozzle axis when slow	**Su-30**	`CantedTvc`, `ThrustLineAtCg`, low-speed TVC roll, measured capture, soft AoA limit; also the only jet with every feature above
none of the above / not sure	**Revoker** or a later fork	the stock-jet lineage (Ifrit → Su-30) counts every effector in the estimator; the F-22E's estimator drops non-surface effectors (§9.1.4)

PART II

Use it and build with it

4. Quick start

4.1 I just want to fly one of the six jets

Install the release from the author's GitHub or from NOMM (the Nuclear Option Mod Manager). Everything in §6 applies. The Ternion and Su-30 FCS need their aircraft mods installed (the FS-3 Ternion blueprint, kyleisred's Su-30VXN); the other four fly stock or Aryx's jets.

4.2 I want this FCS on my aircraft

1. Read §2 (how it works) at least down to §2.5, and §3 to pick the reference jet closest to your aircraft.
2. Clone it under your aircraft's identity:

   ```
   python3 tools/new_airframe.py --from <f22e|ifrit|revoker|compass|ternion|su30> --out ~/MyJet-FCS --ns
   MyJet.FCS \
       --title "MyJet FCS" --short "MyJet FCS" --file-prefix MyJet --prefab <your prefab name> --match "
   <part of your aircraftName>"
   ```

   `--prefab` is optional: without it the jet is matched by its aircraftName only (what the Su-30 does).
3. Work through the generated `PORT_TODO.md` with §7 (porting) and §9 (models).
4. Build a bench model of your aircraft (§9.2) and check it against flight logs before trusting it.
5. Build the DLL (§15), fly, log, compare with the bench, iterate (§11).

4.3 I want to run a bench

```
sudo apt install -y dotnet-sdk-8.0 python3 python3-pandas python3-numpy python3-scipy   # Linux or WSL2
cd reference/Revoker-FCS-1.4.1           # or Ifrit-FCS-1.3.0 / Compass-FCS-0.5.0 (extract the airframe
files first: §5)
bash bench.sh build                       # sync the FCS source into bench/fcs, generate the config stub,
compile
bash bench.sh fly ias=400 alt=1500 thr=1 T=6 pitch=0:0,0.2:1 roll=0:0,1.5:0,1.7:1 fcslog=bench/out/x.csv
bash bench.sh suite base                  # the regression suite (~10 min on 2 cores)
```

The F-22E bench uses `bench.sh one ...` instead of `fly`. Every bench builds without its airframe files but needs them to fly (game or third-party data, not published: §5); point `BENCH_DATA` (and `BENCH_DMG`) at your own extract. Details in §10.

5. What is in the folder

```
NO-FCS-Kit/
├── README.md                    this manual
├── NO-FCS-Kit-Manual.pdf        this manual as a formatted PDF
├── AGENTS.md                    rules + procedures for AI models
├── docs/
│   ├── CONFIG_REFERENCE.md      every config key, six jets side by side (generated)
│   ├── MODULE_MAP.md            every source file: layer, role, size, divergence (generated)
│   ├── papers/                  Tom's write-ups (Sept / Oct 2026)
│   └── design-history/          the newest design hand-offs
├── reference/
│   ├── F22E-FCS-4.3.0/          plugin folder Aryx_F22E_StrikeRaptor.FCS/; studies/ (airframe study kit, push scoring,
│   │                            the loaded-reversal port hand-off)
│   ├── Ifrit-FCS-1.3.0/         plugin folder Ifrit_KR67.FCS/
│   ├── Revoker-FCS-1.4.1/       plugin folder Revoker_FS12.FCS/
│   ├── Compass-FCS-0.5.0/       plugin folder Compass_TA30.FCS/
│   ├── Ternion-FCS-0.4.0/       plugin folder Ternion_FS3.FCS/; tools/ (bench analysis scripts)
│   └── Su30-FCS-0.6.0/          plugin folder Su30_VXN.FCS/; tools/
│       ├── <Namespace>.FCS/     the plugin: laws, models, glue (one DLL)
│       ├── native/              the Rust core: fcs_core/ (source), prebuilt/<Lib>.dll (the core embedded in the shipped
│       │                        DLL), build.sh (Windows DLL or the bench's Linux .so), verify/ (game-Mono harness), README.md
│       ├── DevUnlock/           optional plugin that unlocks the Dev config
│       ├── Properties/ ...      assembly attributes (part of the build)
│       ├── build.sh             builds the game DLL
│       ├── bench.sh             bench front end: build, suite, diff, replay, runs
│       ├── bench/               sim/ plant, stubs/ fake Unity + game, an/ calib/
│       │                        scripts/ analysis, extract/ airframe extraction, groundsim/ gear and brake models
│       ├── tom*.cfg             config the bench's Plugin stub is made from
│       ├── CHANGELOG.md         one entry per version (stock-lineage jets)
│       └── release-notes/       per-version notes in its source zip (CHANGELOG / §19 cover the rest)
└── tools/
    ├── new_airframe.py          clone a reference onto a new aircraft
    ├── gen_config_reference.py  regenerate docs/CONFIG_REFERENCE.md
    ├── gen_module_map.py        regenerate docs/MODULE_MAP.md
    └── build_manual_pdf.py      rebuild NO-FCS-Kit-Manual.pdf from README.md
```

Not included, on purpose: airframe data extracted from the game or from other people's mods.

- Ifrit, Revoker, Compass: `bench/extract/<jet>_airframe.json`, `bench/data/<jet>_damage.json` (the stock game's assets);
- F-22E: `bench/data/airframe_cal.json`, `bench/data/f22e_damage.json` (Aryx's asset bundle);
- Ternion: `bench/extract/ternion_airframe.json`, `bench/data/ternion_damage.json` (someone else's Blueprinter blueprint);
- Su-30: `bench/extract/su30_airframe.json`, `su30_101_airframe.json` (kyleisred's mod; its bench borrows `ternion_damage.json`).

The benches build without them but need them to fly; regenerate them from your own copy of the game or the mod with the scripts in `bench/extract/` (§9.2; a note in each folder says which) or ask the author. Some Ternion and Su-30 bench modes also compare against the Compass and Revoker files.

Not included either: the F-22E's suite outputs of single runs, each bench's `bin/`, `obj/`, `out/` and synced `fcs/` copy (rebuilt by `bench.sh build`), and the Rust build folders (`native/fcs_core/target/`). The F-22E's 9 Nov airframe study kit *is* included, as `reference/F22E-FCS-4.3.0/studies/` (§8.15). The prebuilt cores in `native/prebuilt/` are included: they are the exact libraries embedded in the shipped DLLs, so `build.sh` reproduces the release byte for byte without a Rust toolchain.

"Bare" versus complete. Each reference tree is the *complete* plugin, byte-for-byte what ships, because that is the only form the bench and the release checks are valid for. The parts that are not flight control are labelled instead of cut: `docs/MODULE_MAP.md` puts every file in a layer (LAW, MODEL, AERO, SYSTEM, GLUE, EXTRA), and §7.4 says which ones a port can delete and where their call sites are. The flight-control core is the LAW + MODEL layers plus the GLUE that wires them in.

6. Using it in game

6.1 Install

- BepInEx 5 for Nuclear Option. Put the plugin DLL in `BepInEx/plugins/<folder>/` (e.g. `plugins/Revoker_FS12.FCS/Revoker_FS12_FCS.dll`) with its `meta.json` for NOMM.
- **One DLL per jet** (since 11 Nov): the Rust core is inside it. On first launch it is written to `BepInEx/cache/<Jet>_FcsCore/<hash>/` and loaded from there; the log says `native core: ... ready` and, every 10 s, how many solves and kernel calls ran native. Loose `*_FcsCore.dll` files beside the plugins from 4.0–4.1.3-era installs are no longer needed (harmless if left).
- Optional: the Dev DLL (`<GUID>.Dev`, built from `DevUnlock/`) at the plugins root unlocks the Dev settings file.
- Check `BepInEx/LogOutput.log` for the FCS's attach line (e.g. "Revoker FCS attached: 5 surfaces [...]") and the version.
- **If you use a mod manager** (NOMM / NOSMR modpacks): update the FCS version there too. On 7 Nov a modpack that still listed the F-22E FCS at 3.0.3 silently redeployed that old DLL over 3.1.x, and another removed the Compass FCS entirely. When a fix "doesn't work", check the version line in `LogOutput.log` first.

6.2 Config files

- **Player file** `BepInEx/config/<GUID>.cfg`: `Enabled`, `PerformanceProfile` (Stock default / Buffed), `StickSmoothing`, `ThrottleIndependentAirbrake`, `PitchCommandLaw`, the AI toggles, the lateral-governor keys (section 6), multiplayer host keys, per-profile countermeasure/ammo scales.
- **Dev file** `BepInEx/config/<GUID>.Dev.cfg`: everything else (envelope, gains, airframe, ground, debug). Without the Dev DLL these are held at their code defaults in memory and never written.
- Some keys are **migrated** once on update (a changed default is forced onto existing files that still hold the old value); the description says so. Everything is listed in `docs/CONFIG_REFERENCE.md`. The migrations are counted by a revision key in the Dev file (`AirframeRevision` on the F-22E, now 28; `GovernorRevision`; `RevokerConfigRevision` 16, `CompassDevRevision`, `TernionDevRevision`, `Su30DevRevision` 15, `Su30ConfigRevision`). Each logs a line such as `Revoker FCS 1.4.1 defaults: MaxPitchRate 60 (from 48), once.`
- **Performance switches** ("0. Debug", Dev file, all default on, take effect at once): `EstimatorThreaded`, `ParallelTables`, `ParallelAllocator`, `ParallelSweep`, `ParallelEnvelope`, `SweepGridCache`, `NativeTables`, `NativeSweep`, `NativeCoupledCap`, `TickTimerLog` (§2.16, §2.18); `NativeAllocator` 0 / 1 / 2 (default 2) and `NativeAllocatorCheck` 0 / 1 / 2 (default 0). Turning a thread switch off mid-flight is a valid timing A/B: the flying is identical. Turning a native switch off changes the arithmetic to the game's float mode (last digits, §2.18), so it is a valid timing test, not a bit-identity one.

6.3 Flying it

- **Gear up, Stability Assist on:** G + pitch-rate command; release the stick and the pitch rate goes to zero and holds.
- **Gear down:** pitch-rate command with limits; on the runway the same law rotates the jet.
- **Stability Assist off (MPO):** pitch-rate command, MPO g limits, no AoA limit.
- **Full aft stick at the AoA limit** holds the limit; protection only pushes the nose down once you release.
- **Rolling at high AoA** is a velocity-vector roll; past ~36° the pedals blend into it.
- **Brake key** deploys the airbrake at any throttle; idle throttle never does.
- **Push-overs from high AoA** pitch the nose down at no less than 23°/s on the F-22E even at very low airspeed, accepting some extra sideslip (§2.10).
- **Ejecting** hands the jet back to the stock FBW (§2.13).

- **Flying backwards** (tail slides, past 90° AoA), left stick or pedal turns the nose left (§8.24).
- **With an asymmetric load** the AoA limit sits where both sides keep a share of their roll (§8.23); expect a lower limit with heavy loads and roll toward the load limited to what can be stopped.
- **On the ground** the main brakes help the nose wheel stop a turn, and holding a pedal drifts the jet (§8.26).

6.4 Flight logs

`Telemetry` (Debug) writes `BepInEx/<Jet>_FCS_<yyyyMMdd_HHmmss>.csv` at the physics rate: commands, estimates, authorities, every surface and nozzle, governor internals, per-phase timers (§16). Since 3.0.4 the rows are written by a background thread; the last row is written when the controller is disposed. Bench replays and calibration read these files. Since 4.1.3 switching `Telemetry` off in flight pauses the rows. Logging costs about 50 µs and 16 KB of garbage per tick (the boxed CSV row), and it caused every 3 ms hitch measured; it is off by default for players.

6.5 HUD and visualizer

The Flight Data Readout HUD and the FCS Visualizer find the FCS through `FcsApi` (§2.15) and show the command mode (G / rate / AoA / MPO / protection) and the live allocation.

7. Porting it to your aircraft

Porting is mostly **telling the FCS what your aircraft's effectors are** and **checking every place where a reference jet's assumption leaks in**. The control laws themselves are written to compensate from measured state, so most of them transfer. The parts that do not transfer automatically are the ones that name or count specific surfaces.

7.1 Before you start

- You need: .NET 8 SDK and Python 3 (Linux or WSL2 for the bench scripts), the game's `NuclearOption_Data/Managed/` and `BepInEx/core/` folders for the real build, and UnityPy + numpy + scipy for airframe extraction.
- Know your aircraft: prefab (GameObject) name, `AircraftParameters.aircraftName`, every `ControlSurface` (name, pitch/roll/yaw range, servo speed, `maxSplit`, `brakeRange`), engines (`thrustVectoring` vector, nozzles), airfoils, fins. The bench's `info` mode prints all of this once you have an airframe file (§9.2).

7.2 Clone

```
python3 tools/new_airframe.py --from <f22e|ifrit|revoker|compass|ternion|su30> --out <dir> --ns
<YourJet.FCS> \
    --title "<Display FCS>" --short "<Log name>" --file-prefix <CsvPrefix> [--prefab <PrefabPrefix>] \
    --match "<aircraftName part>" [--match ...]
```

It renames the namespace and plugin folder, the BepInEx GUID (= namespace), the Dev GUID, the Harmony id, the display name, the log prefix, the flight-log file prefix and `FcsApi.FcsName`; sets the version to 0.1.0; rewrites the aircraft match in `Registry.IsF22` (the method keeps its historical name: it means "is this one of my aircraft"); and rewires the bench so `bash bench.sh build` compiles the clone. It changes **no** law, limit or tuning value; a clone made with the reference jet's own identity produces identical bench output (checked on 12 Nov: clones of all six under a new identity build the game DLL with the renamed core embedded and the bench with the renamed Linux core; the F-22E, Compass and Su-30 clones fly the same bench case as their reference, every column identical except the wall-clock timer columns).

It also **renames the native core** to `<file-prefix>_FcsCore` (the `Lib` constant in `FcsNative.cs`, Cargo's `[lib]` name, the bench's copy step, `native/prebuilt/<Lib>.dll`). Windows binds a `DllImport` by module name, so a clone that kept the source jet's core name would share whichever of the two loaded first in a game where both mods are installed. The prebuilt core itself is the source jet's (same exports, same ABI), which is right until you change the Rust code; then rebuild it (§15.3).

It writes `PORT_TODO.md` listing the jet-specific places with file:line, including the newer jets' name maps (`Names.cs`), `StructureFix`, `ThrustLine`, `CantedTvc`, `Strake`, the one-time config migrations (`*Revision` keys: reset them for a new jet), the fuel hook, the native core's ABI, the coupled check's reserve share, the reversed-flow axis and the gear / brake hooks.

7.3 Work through the checklist (in this order)

1. **Aircraft match** (`Registry.IsF22`): match only your aircraft. Both the prefab-name prefix and the aircraftName parts are checked; make them specific (a loose "Fighter" would grab other jets).
2. **Surface roles** (`FcsController.Classify`): map your surface names to `Stabilator` (all-moving pitch surfaces, canards, ruddervators), `Flaperon` (inner trailing-edge roll/flap), `Aileron` (outer roll), `Rudder`. Anything with a non-zero pitch range becomes `Stabilator` by default. The role decides costs, fight groups, the monotone pitch model, the estimator group and every `SurfaceKind`-keyed rule in the laws.
3. **Effector list** (`FcsController` constructor): surfaces → `SurfaceEffector`; pairs by `_L` / `_R` name; TVC; then the optional effectors (`SplitEffector` for `maxSplit > 0`, `TvcDiffEffector` for ≥2 engines with roll

vectoring, `TvcYawEffector`, `HingeFlapEffector`, `CantedTvc`). Set the minimum surface count (`expected N control surfaces`). **Blueprinter jets:** parts can be named with a prefix (`Left_stab_struct`) instead of the `_L/_R` suffix the lineage keys on (the Su-30 maps them in `Names.Canon`), split physics bodies can put a part's scripts outside the `Aircraft` hierarchy (the Su-30's canard schedule scripts had to be found in the scene), and a part's transform parent can be an engine (the Ternion's tail: `StructureFix`). Dump the part tree and check all three before trusting the attach line.

4. **Airframe setup** (`Registry.EnsureSetup`): what the plugin changes on the game aircraft once (servo speeds, ranges, TVC range, performance profile thrust/drag, stagger, camber). Keep what you want, delete the rest (§7.4). Fix the completeness check (`num2 >= 5 && num >= 1` = 5 surfaces and 1 engine found).
5. **Envelope defaults** (`Plugin.cs`): `AoALimitPositive/Negative`, `GLimit*`, `MaxPitchRate`, `MaxRollRate`, `MaxYawRate`, surface ranges, `NoTvcAoACap*`. Start conservative (e.g. +35/−20 AoA) until the bench and a flight say more is safe.
6. **Aero mirrors:** `Camber.HostOf` / `KForName` (host part names, K values), `StallStagger.FamilyOf` (families by name), `Downwash`, `BodyLift`, `LiftShape`. If your aircraft does not need them, disable them (their `Enabled` / setting) rather than leaving the reference jet's name lists in place: a wrong host name silently does nothing in game but can still alter the FCS's model.
7. **Capacity estimator groups** (`CapacityEstimator.Group`): every lateral effector must be in a group (§9.1.4). Start the group gains at 1.
8. **Rudder / yaw-axis assumptions:** search `SurfaceKind.Rudder`, `rudderAuth`, `RudderFade`, `yawHealth`, `fin`. On a jet without rudders follow the Ifrit (§8.3).
9. **Engine hooks** (`Hooks.cs`, `Plugin.ApplyPatches`): the stock jets' engines are the game's `Turbojet` (nozzle angles set through `TurbojetTranspiler` and the `Tvc*` hook methods); Aryx's F-22E uses `Turbofan` (`TurbofanTranspiler`). Check which engine component your aircraft has and what its vectoring fields are (`Acc.TF_*`), and start from the reference that matches.
10. **Airbrake / ground configuration** (`GroundAeroBrake*`, `Airbrake*`, `TrailingEdgeBrakeConfig` on the Revoker): match your surfaces and your airbrake parts (the attach log prints an "airbrake inventory").
11. **Per-jet governor constants:** `PushPlanK0`, `PushPlanK`, `PushoverSideslip`, `RollStopSideslip`, `RollPaceRateBlend`, `FinLossAoACap`. The push plan's constants are **fits to one jet's logged push-overs**; until you have yours, expect push-overs to be too slow or too sideslippy (§11.4).
12. **Telemetry columns** that name surfaces (`WriteHeader` / `WriteRow`): rename them so logs read sensibly.
13. **FcsApi**: keep the class and members; `FcsName` is already set by the clone tool.
14. **MpPolicy**: keep it if you publish.

7.4 What a port may delete (the "bare" core)

From `docs/MODULE_MAP.md`, layer by layer:

Layer	Files	Keep?
LAW	`FcsController`, `Allocator`, `CapacityEstimator`, `AsymCap`, `RateTraj`, `Lpf`, `Lpf3`, `FcsMode`	**keep**
MODEL	`Effector`, `SurfaceEffector`, `TvcEffector`, the optional effectors you use, `AirfoilTable`, `EvalContext`, `SurfaceKind`	**keep** (delete unused effector classes and their constructor lines)
AERO	`StallStagger(+Jobs)`, `Camber(Force)`, `Downwash(Force)`, `BodyLift`, `LiftShape`	optional. `StallStagger.TableFor` and `Camber` are called from the models, so disable by setting rather than deleting, or remove the calls together
SYSTEM	`FuelBalance`, `FlapSystem`	`FlapSystem` is referenced by the controller (keep); `FuelBalance` is optional (call sites in `Hooks.FilterPrefix` and `FuelTrimStep`)

Layer	Files	Keep?
GLUE	`Hooks`, `Registry`, `Plugin`, `FcsStandalonePlugin`, `Acc`, `MpPolicy`, `FcsApi`, `Telemetry`	**keep**
EXTRA	`Countermeasures`, `DragRelief`, `NozzleRig/Kinematics/MeshData` (F-22E), `VisualHinge` (Revoker lineage)	delete: each is attached from `Registry` (one call each) and configured in `Plugin`; `Countermeasures` also patches `PowerSupply.ModifyCapacitance` in `Plugin.ApplyPatches`, and `HostLockVariant` locks its capacities
threading	`EstimatorJob`, `ParallelFcs`, `TelemetryRow`, `TickStats`	keep: switching them off (Dev "0. Debug") gives the single-thread path without deleting code; deleting needs the call sites in `Tick`, `Telemetry` and the effectors' `Eval*` split undone
native core	`FcsNative`, `native/`	keep: with the core missing or failing its self-tests everything falls back to C#. To ship without Rust, build from a tree with `native/prebuilt/` empty (`build.sh` then embeds nothing and says so); the C# runs, about 4× slower per tick
coupled check	`CoupledCap`	keep on any jet that carries stores or can lose a wing section; it is the asymmetric-load limit on five of the six

After deleting, build the real DLL (§15) *and* the bench, and run the suite: a deletion must not change bench output (byte-identical, §14) unless you meant it to.

7.5 Bring-up sequence

1. Bench model of your aircraft (§9.2); `info` must show the right mass, inertia, surfaces and engines.
2. Bench: trimmed level flight hands-off, then pitch steps, rolls, a pull to the AoA limit, release. Fix the obvious first (wrong signs, surfaces fighting, a stab parked at a stop).
3. Game: fly with `Telemetry` on, at a safe altitude, Stability Assist on. Compare the log with a bench replay of the same window (§10.4). Where they disagree, calibrate the bench (§9.2.4) before tuning the law.
4. Only then extend the envelope, fit the per-jet constants, and tune (§11).

8. Solution library: aircraft configurations we have solved

Each card: the configuration, the problem it posed, the solution in the code, and where to find it. Use them as patterns. The library is not complete: VTOL / STOVL, swing wings, tailless designs, propeller aircraft and other layouts have not been done yet (§24.10).

8.1 Pitch-only TVC that offloads the stabs (F-22E)

- **Problem:** the stabs are needed for roll and yaw at high AoA; using them for pitch leaves no lateral authority.
- **Solution:** TVC is the cheapest pitch effector in the allocator (costs ~50× below symmetric stab). With lateral stick in, the stabs go to roll/yaw in proportion to the stick (`LateralStabReserve` 0.7) and the nozzles carry what pitch they can; pitch rate is limited rather than nozzle travel increased. During rolls a nozzle budget keeps the nozzles near trim (`TvcBudgetStep`, `TvcNeutralHold`).
- **Also:** the low-thrust AoA cap (no thrust = no TVC pitch authority) and the `NoTvc*` caps (45°).

8.2 Canted rudders that make inverse roll (F-22E)

- **Problem:** canted fins' rudders make roll opposite to the wanted one at high AoA.
- **Solution:** at high AoA the rudders are counted on the *sideslip* axis, where their inverse roll helps ($|\beta|/|\dot{p}W|$ 3.2 at 55°, 6.1 at 75°); the yaw fade moved to 50–60° (`RudderFadeStartAoA/EndAoA`); fin camber coupling ($K \approx 0.5$); rudder toe-in as a pitch aid (`RudderPitchAid`, `RudderToe*`).

8.3 Tails that are also the yaw axis, no rudders (Ifrit)

- **Problem:** canted all-moving tails (~40° dihedral) do pitch together and yaw differentially. A rudder-based governor found "no yaw axis" (the first port read zero capacity and pinned the roll rate at its 10°/s floor).
- **Solution:** the tail pair's differential is a yaw effector (`yawEffs`, a synthetic `tailYaw` effector); the estimator derives rudW from the yaw-only probe's roll collateral instead of a rudder fade; when the allocator reports unmet yaw, yaw outranks pitch and the nozzles take the pitch off the tails (`TvcYawOffload`, `YawNeedK` 6, `YawNeed0` 2, `YawNeedSpan` 10, 0.16.0).
- **Lesson:** count every yaw source on the sideslip axis at every AoA (§14).

8.4 Roll / differential TVC (Ifrit)

- **Effector:** `TvcDiffEffector` (differential angle about the symmetric one), with its own cost and a roll-TVC budget that holds the nozzles near pitch trim during lateral input (opened when yaw is short).
- **Why it matters:** at 150 km/h and 65° AoA roll TVC is ~48 % of the Ifrit's sideslip authority and the only part that does not fade with speed. The asymmetric-load model must use the *symmetric* nozzle moment, or it reads roll TVC as a load asymmetry (a port bug that dropped the AoA cap to 12°).

8.5 Pitch + yaw TVC on a single nozzle (Revoker)

- **Effector:** `TvcYawEffector`: the yaw angle about the current pitch angle; the pitch `TvcEffector` table is taken at yaw 0 so the model is tvc.M(pitch) + tvcy.M(yaw). The game zeroes yaw TVC below 3 m radar altitude and both angles above the vectoring speed; the model does the same.
- **Settings:** `YawTvc`, `YawTvcCost`, `YawTvcNeutralHold`, `SlipYawTvcAssist`.

8.6 Close-coupled canards as 3-axis effectors (Revoker)

- **Classification:** canards are `Stabilator` (pitch range ≠ 0), so they get the stab handling: monotone pitch model, pair costs, reach pass. They are *ahead* of the CG, so a stall acts the opposite way in pitch; check that every `SurfaceKind.Stabilator` rule makes sense for your canards.

- **Travel:** asymmetric +60 / −45 (TE up / down), applied from the geometry sign before the surface has seen flow (`GeoTeUp`), so it is right on the runway at a standstill.
- **Windows:** `CanardAttachedWindow` walks the canards to where their own flow still has lift above 30° AoA with lateral input.

8.7 Delta elevons with real pitch moment and staggered stall (Revoker)

- `ElevonPitch` : the outer elevons keep their true pitch moment in the model (pitch + roll effectors).
- `ElevonStallStagger` : inner pair 26° knee-preserving (curve unchanged below the 17° knee, peak lift kept), outer 45°, so the pairs don't stall together and leave a hole in roll authority.

8.8 Hinged surfaces that are not game ControlSurfaces (Revoker inboard elevons)

- In the stock game the inboard flaps are moved only by the high-lift device. `HingeFlapEffector` drives the flap hinge directly at servo speed, turns the host wing section's lift normal for camber (K 0.46), and turns the high-lift function into a symmetric droop target. `FlapSystem` takes over the `HighLiftDevice` (its `FixedUpdate` is skipped while the FCS is engaged). Use this pattern for any moving part you need that the game does not expose as a control surface.

8.9 Split decelerons for drag yaw (Ifrit) and opposed flaperons as decelerons (F-22E)

- `SplitEffector` : yaw from split drag ($F = 0.1 \cdot \text{split} \cdot \rho v^2$), sharing one travel budget with the surface's deflection (everything within ±30° of the wing plane). With the airbrake out, the variable becomes the retraction from full split, so yaw still works with the brake deployed.
- F-22E: with both fins gone, inner and outer flaperons on one wing open against each other; their rolls cancel and their drag yaws the jet (`SplitFlaperonYaw` , `SplitFlaperonYawUse` 1.0, `SplitFlaperonRollYield`). Finless held-roll sideslip went from 19.8/88.1° to 8.7/16.7° mean/worst on the bench.

8.10 Staggered stall (F-22E, Revoker, Ternion)

- Every horizontal surface sharing one foil stalls at once and leaves a lateral-authority hole. `StallStagger` gives each family its own remapped copy of the foil, injected into the game's aero chart array (`StallStaggerJobs` , a prefix on `AeroJobSettings.Schedule`); the FCS models use the same table (`StallStagger.TableFor`).
- Finding: staggering alone does not add roll authority (a pair parked on its own lift peak still cancels); it improves low-AoA static stability. Keep-peak-CL mode is the default; keep-slope mode lost 15–30 % roll.

8.11 Camber coupling (all six)

- NO flies a deflected surface as its own plate; on a real section the fixed part ahead carries part of the extra force. `CamberForce` adds the correction force at the host's centre of lift; `Camber.HostOf` maps each surface to its host part by name; K from thin-airfoil flap theory and the chord fraction (F-22E fins ≈ 0.5, Ifrit flaps 0.46 / ailerons 0.57, Revoker inboard flaps 0.46). The FCS models include the same term. Camber raised the Ifrit's roll at 135 kt from 106 to 151°/s.

8.12 Asymmetric loads and damage (all six)

- §2.11; the asymmetric-load limit itself is §8.23. The pieces that transfer unchanged: asymmetric-load cap, yaw-health scaling, the yaw-target clamp, fuel balance/trim. The pieces that need per-jet thought: what "fin loss" means (Ifrit: a tail is pitch *and* yaw; Revoker: one fin), the VMC arm (engine spacing), single-engine jets (no thrust asymmetry to match).
- **Let the cap come down fast** after a hit (`AsymCapGapTighten`): at the old fixed 10°/s a full pull 2 s after a stab was shot off reached 25–43° AoA before the cap arrived.

- **Gate damage logic on the damage, not on a symptom:** the Compass's protections were gated on the lateral CG offset, which swung −0.28 ↔ 0.00 m with a hanging rudder (`AsymDamageGate` gates on a lost control surface instead). The F-22E's no-fin rules ran in straight flight because Unity's `Mathf.Sign(0)` is +1 (§19, F-22E 3.0.2): any "which side is the roll" test needs an explicit dead band.

8.13 Engine out / damaged engine (twin-engine jets)

- Thrust match before VMC; VMC aware of the engine-spacing arm; engine-out AoA cap (Ifrit: dynamic 22 → 18° as the yaw nears what can be compensated; F-22E: 30°); damaged-engine cap 50° (F-22E).
- **Fixed nozzles** (Compass, Ternion): a jet whose engines have no vectoring transform had its thrust moment skipped entirely, so with an engine out the live engine's yaw read 0. Take the thrust moment as it is (roll/yaw only, engine out only), and limit the throttle **predictively** once that yaw passes 70 % of the yaw authority at the predicted dynamic pressure (`VmcPredict`). Engine-out full-aft climbs from 220–300 km/h: departures → yaw held to the top of the climb.
- **Past the AoA where no surface can hold the asymmetric thrust** (Ternion): match the live engine to *zero* thrust above `EngineOutMatchAoA` 35°, latched with hysteresis on yaw saturation (`EngineOutZeroThrust` , 0.1.9–0.1.11), and learn the steady engine-out sideslip so a hands-off jet doesn't creep (`EngineOutSlipTrim` , 0.1.12; heading drift 3–6° → 0.8–2.4° in 20 s).

8.14 Aircraft without TVC (Compass, Ternion)

- `DisableTVC` (test switch) shows the behaviour on a TVC jet. For a jet with no vectoring at all, `TvcEffector` evaluates to zero authority. Things that broke on the real no-TVC jets, all fixed:
  - the inherited fixed `NoTvcAoACap*` 45° caps capped every limit above 45, MPO included (vectorable T/W is 0): off on a `noVecJet` (Ternion 0.1.1);
  - MPO's pitch-rate budget had a 40 × T/W term that is 0 without TVC, so full aft stopped at 55–58° (14 °/s at 215 km/h): `MpoNoVecFullRate` (Ternion 0.1.8);
  - `StickCancelsLimit` must not let the stick cancel the AoA limit on a jet that has no TVC to recover with (`StickBypass` , Compass 0.1.2);
  - the roll-coupling limit (the roll's inertial pitch-up paid out of the nose-down margin) must be on from 3° AoA whatever the stick (`CoupNoTvcAlways` , Compass 0.1.3), and with damage a roll governor on the stabs' nose-down margin (`StabRollGovernor` , 0.1.5): losing an outer wing in a 90°/s full-aft roll had taken the last nose-down (AoA 12 → 47°);
  - corkscrew entries built pitch on 30 % of the pitch authority because `LateralStabReserve` 0.7 reserved the rest for roll, a TVC-era rule (Ternion 0.1.2: 0).
- `T/W` (`tw`) still reads 0 on fixed-nozzle jets; several laws use it, so it was left alone and noted.

8.15 A relaxed-to-neutral airframe by scheduling the fore / aft body lift (F-22E X2 / X2e)

This is the F-22E's 3.2.0 / 3.2.1 airframe change: the largest aerodynamic change since 2.0. It is an **airframe** change made to the game (the FCS's own tables read the same chart builders, so plant and models stay consistent); no control law changed.

What Tom asked for (9 Nov): a simulated LERX vortex, no stab downwash modelling, a stab lift peak at 36°, and a margin relaxed to neutral. The goals behind it, from real-jet footage of a loaded roll at 36° AoA:

- **G1:** one stab near neutral riding its maximum lift, the other trailing-edge down into stall, good roll at low airspeed, TVC hardly used for pitch;
- **G2:** up to 36°, stabs and TVC only nudge the jet into a turn, then sit at or near neutral;
- **G3:** from 36 to 45°, useful nose-up *and* nose-down authority aerodynamically, without the stabs using most of their travel to hold the AoA.

What the study found (bench, calibrated plant; `claude/F22E_trimfree_airframe_study`, `..._lerx_vortex_airframe_prototype`, `..._stab_poststall_shape_study`, `..._loaded_roll_36_stabs`, `..._static_margin`, all 9 Nov):

- **The roll's pitch-up is inertial coupling,** $\dot{q} \approx (Izz - Ixx)/Iyy \cdot p \cdot r$: the same in both roll directions, needs p and r together, grows with roll rate² and sin 2α, independent of airspeed (+100 °/s² at a 180 °/s velocity roll, 15° AoA). INDI cancels it; the coupling roll limit (2.6.9) sizes the roll rate from the nose-down headroom it costs.
- **The package exactly as asked ("T": no downwash, stab peak 36, trim-free schedule) fails G3.** Full nose-down stab gives only −5 °/s² from 36 to 48° (the stab sees the full body AoA, Aryx's F22E_Wing normal force is flat ≈ 1.16–1.2 from 35 to 65° local, so a stab at its peak has nothing left, and past the peak its control reverses). It departed to 60° at idle; a push from 45° at idle or with TVC off never recovered.
- **G1 and G3 conflict literally at 36°:** a neutral stab at maximum lift has no nose-down left.
- **The explicit Polhamus LERX vortex** ($\Delta CL = Kv \cdot \sin^2\alpha \cdot \cos\alpha$) on the no-downwash base made the jet nose-up past 24°, untrimmable at 32–40°. CG shift tops out at ~0.18 m aft in the game (`Registry` moves at most 0.6 × the fore part's mass).
- **The post-peak shape of the stab curve** changes static moments but not the loaded roll as the FCS flies it: with thrust the allocator always unloads the second stab trailing-edge **up** (3–10× more roll per degree) rather than stalling it. So G1 is control-law work, not airframe work.

The solution, X2 (3.2.0): keep the 0.6 downwash, move the stab lift peak 25 → 36° (`StabilatorStallAoA`), and add an AoA-scheduled lift multiplier on the fuselage sections **ahead** of the CG (LERX, intakes, fore, cockpit, centre body) and **behind** it (engine bays, tail booms, tail), on top of the existing `BodyLiftFore` 0.9 / `BodyLiftAft` 1.8:

- fore and aft move **together**, so the lift at every AoA is unchanged;
- the schedule is **fitted** (`studies/x2_study_kit/scripts/fitsched.py`) so the stabs-neutral, TVC-0 pitching moment is zero from 4 to 48°, which makes the static margin ≈ 0 (±0.03 m) over 4–36°: "relaxed to neutral". This does the LERX vortex's job (more forward lift as AoA grows) in a controlled way;
- positive AoA only; linear between knots; faded back to 1 from 45 to 60° so post-stall is unchanged; a malformed string counts as off.
- Dev config "5. Airframe": `ForeLiftSchedule`, `AftLiftSchedule` (`"AoA:multiplier,..."`, applied at spawn). Empty strings and `StabilatorStallAoA` 25 give 3.1.9's airframe back.
- The FCS's stab logic reads the peak from the table (`se.stallAoA`), so constants tuned around 25° moved with it automatically (`stabStallShift` 8.75 → 0, `BranchScaleToPeak` factor 0.74 → 1.07, the stall windows 19 → 30 and 28 → 39).

What X2 cost, and the fix, X2e (3.2.1): X2 met G2 and G3 on the bench (holds at 20 / 30 / 45° with ≤ 0.8° of stab and ~1° of TVC; push-overs from 45° within 0.1 s of 3.1.9) but had two regressions:

1. **Low-throttle overshoot past the 45° cap** (54–56° instead of 45): the stab's small-angle lift slope is 0.69× (peak CL kept at 36 instead of 25), so the pull puts the stabs further trailing-edge up; and a trim-free airframe has no natural nose-down to help the brake.
2. **The 45–55° roll dip** Tom felt in flight (20:10 log): the coupling roll limit is $\sqrt{(2 \cdot 0.5 \cdot netC / (k_I \cdot |\sin 2\alpha|))}$, where netC is the full-stab nose-down plus the nozzles'. X2's 48° knot moved lift forward just as the stab downwash starts fading, so full nose-down stab at 48° fell to −18 °/s² (3.1.9: −41), the limit dropped to 24–54 °/s (3.1.9: 73–80), and the velocity roll at 45–55° fell to 0.48–0.82 of the max-AoA rate. Loosening the limit would have spent the stop margin, so the fix went into the airframe.

X2e is identical to X2 up to 40° and adds knots at 52 and 56° that give the nose-down back from 44° (stabs-neutral targets −6.5 / −18.4 / −31 / −30 °/s² at 44 / 48 / 52 / 56°, 85 m/s). Lift is unchanged at every AoA. Shipped defaults:

```
ForeLiftSchedule =
0:1.0000,4:1.0843,8:1.0814,12:1.0795,16:1.0738,20:1.0577,24:1.0466,28:1.0359,32:1.0319,36:1.0391,40:1.0607,
44:1.0334,48:0.9673,52:0.9270,56:0.5670
AftLiftSchedule  =
0:1.0000,4:0.8823,8:0.8862,12:0.8888,16:0.8984,20:0.9232,24:0.9402,28:0.9554,32:0.9616,36:0.9537,40:0.9287,
44:0.9596,48:1.0419,52:1.0889,56:1.9918
```

(the 56° point looks extreme, but the 45–60° fade has already scaled it to about ±20 % effective).

Bench (calibrated plant)	3.1.9	X2 (3.2.0)	X2e (3.2.1)
full nose-down stab at 85 m/s, 48 / 52 / 55° (°/s²)	−41 / −32 / −27	−18 / −21 / −22	−44 / −41 / −36
hold 20 / 30 / 45° at AB: mean stab	+4.5 / +5.9 / +9.4	+0.2 / +0.7 / +0.1	+0.2 / +0.7 / +4.2
push 45 → 20°: idle / TVC off / AB (s)	1.95 / 1.42 / 0.88	2.05 / 1.53 / 0.90	1.98 / 1.45 / 0.88
low-throttle pull + roll at 400 km/h: peak AoA (5-run median)	45.1 / 45.4	53.9 / 55.9	50.3 / 51.1
loaded roll at 36°, 400 AB / 250 AB / 300 idle (°/s)	68.5 / 50.9 / 28.8	65.6 / 46.9 / 26.4	65.7 / 46.9 / 26.4

In flight (X2d, the same as X2e up to 48°): the coupling limit was back to 70–82 °/s at AB and the 45–55° roll matched 3.1.9 at matched entry speed (130 kt: 57.7 vs 57.2 °/s) with sideslip 1.4–6.5° (3.1.9: 1.7–8.4). X2e's 52/56° knots target the remaining dip at 169 kt around 54°; **no X2e log has been analysed yet.**

Still open: the low-throttle overshoot is 50–51° (3.1.9: 45); the "hold 30 at idle" case settles at 26.8° because the full-aft latch (`protHold`) holds attitude until AoA < limit − 5 (releasing at the limit fixes it on the bench; a law change, not shipped); the bench's nose-down table reads 20–60 °/s² more nose-down than the game's in the coupling-limit entries (cause not found; a telemetry build logging netC's terms would find it); G1 (stalling the second stab in a loaded roll) is phase 2, control-law work, not started.

How to apply this to your jet: the schedule is a general tool for "make the airframe neutral without moving the CG past what the game allows". Fit it on a calibrated bench (`fitsched.py`, §9.2.4), keep lift unchanged at each AoA, fade it out before post-stall, and check what the airframe's natural nose-down was buying you before removing it: the roll's coupling limit, the low-thrust brake and the no-TVC fallback all spend it. Port only the schedule parts of `bench_patch/BodyLift.cs` (`Interp`, `Sched`, the multiply in `BuildCharts`, the strings in the chart key); the `Lerx*`, `VortexCl`, `AftBump` and `StabPost*` pieces are study-only.

8.16 Two pitch pairs: canards and an aft tail (Ternion)

- **Problem:** with two `Stabilator` pairs a cancelling roll split (canards one way, elevators the other) was nearly free for the allocator, so after a manoeuvre the pairs were left in opposing splits.
- **Solution:** `StabilatorRollCost` 0.0005 → 0.01 (0.1.6), per-pair trim costs (`ElevatorTrimCost`), canard-only attached / settled windows and rollout brake. The elevators stall at 30° local (`ElevatorStallAoA`) but keep the airfoil's own drag (0.1.4: moving the drag rise in with the peak raised their normal force past the stall). Cost: corkscrew entries reach 30° ~0.15–0.2 s later.

8.17 Single-axis canted nozzles (Su-30, `CantedTvc`)

- **Configuration:** each AL-31FP-style nozzle swivels about **one** axis (±15°, 65°/s) in a plane tilted 32° outboard. The pair's symmetric part is pitch; the differential is proverse roll **and** yaw, and each nozzle shares its one axis between pitch and the lateral job.
- **What it needed:** each nozzle as one effector axis; the canted differential counted as a yaw source everywhere (sideslip probe `SlipAuthTvc`, 0.1.1, after the probe read the authority halved by 4° of sideslip and cut the roll margin 6 → 2.8°); the velocity-roll weight raised at high AoA so the nozzles don't stay at full nose-up through a rolling pull; thrust-borne yaw not faded with IAS (`YawThrustShare`); the trade's capacity low-pass matched to the nozzles' 0.46 s transit (`TradeCapTau` 0.45); surfaces carry the pitch first at low/mid AoA (`SurfacesFirstPitch`, 0.1.8: nozzle common deflection at 5–45° 1.7–6.2 → −0.1–1.1°) so the nozzles are free

for the lateral job; and at low speed, roll about the nozzle axis itself (§2.9). In MPO past the AoA limit the roll axis blends to the axis the canted differential turns the jet about (~39° from body x).

8.18 A thrust line far from the CG, and taxi tipping (Su-30)

- **Problem:** the nozzles sit 0.49 m below the CG and the CG only 0.74 m ahead of the mains: full afterburner unloaded the nose wheel (nose up at 80 km/h, tail strike at 13.5°), and thrust changed pitch trim with throttle.
- **Tried first:** a closed-loop nose-wheel load hold through stabs, canards and nozzles (0.1.1–0.1.4), then a thrust feed-forward. Both were symptom fixes.
- **Solution:** move each nozzle's thrust force point, along the jet's vertical axis only, to the CG's height (`ThrustLineAtCg`, 0.1.5; the FCS's nozzle models use the same arm). Nose-wheel load at full AB with the stick neutral 26–32 kN (static ~30). The nose-wheel hold was deleted in 0.1.10.
- **CG for taxi vs margin:** CG 0.25 → 0.15 → 0.10 m aft with canards ×1.3 → 1.5 → 1.6 kept the low-AoA static slope within ~7 % while the static nose-wheel share went 9.6 → 12.2 % (0.2.1–0.2.2), and `FuelCgHold` stops fuel burn walking the CG ~30 cm aft.

8.19 When the airframe's nose-down beats all the pitch authority (Su-30 "pitch wall")

- **Symptom:** a full-aft pull from high speed stops pitching at 50–65° AoA until the jet bleeds speed.
- **Diagnosis method (reusable):** fit the game's own pitch acceleration (minus inertial coupling) over logs against AoA, each stab's deflection in bins, canard, pitch damping and nozzle × T/W, all scaled by $\bar{q}$ (R^2 0.94). Then compute the best-case pitch acceleration with everything at its best. Above ~400 km/h at 55°, 350 at 62° and 300 at 68°, the airframe's nose-down was larger than all the authority combined: mainly the airframe, not the FCS.
- **What the FCS can take:** the 5–15 °/s² it was wasting (nozzle split for roll, a stab crossing the worst-pitch middle of its travel when a roll starts, canards short of their peak): `PitchWallSurfacePeak` (0.1.9) holds surfaces at their raw pitch peaks in the wall. On the recalibrated bench it holds AoA better (dip 14.1 → 10.5°) but rolls about a third less bank from the wall at 460 km/h and up: a trade, Tom's call.
- **Lesson:** decide "airframe or FCS?" with a fit to game data before changing either; and see §10.7 for how an uncalibrated bench misled this one.

8.20 Strakes and slab tails on a stock trainer (Compass)

- The T/A-30's wing-root strakes flew the wing's own foil (stall 17°), adding a flat ~8 °/s² of nose-up above the stall. They now get a slender-strake vortex-lift curve with breakdown at 30–50° (`StrakeVortexLift`, `Strake.cs`, area ×3.4 via `LerxEffectiveness`); the tail slabs fly a swept-slab curve (stall 30°, peak CL 1.22) instead of the wing's 17° foil (`StabSlabFoil`). CG 0.15 m aft for relaxed stability (0.27 pinned both elevators nose-down in low-speed rolls from inertia coupling). The roll ceiling inherited from the F-22E held full-stick rolls to ~105 °/s with the ailerons at 6–7° (`RollRateGain*`, `RollAuthorityUse` raised: 142 → 247 °/s at 300 km/h, 1 g).

8.21 Blueprinter aircraft: names, hierarchy and scripts

- **Names:** the Su-30VXN names parts with a prefix (`LCanard_struct`, `Left_stab_struct`); `Names.Canon` maps them onto the `_L/_R` scheme every pairing, camber host, stall family and downwash rule keys on. The bench had the same mismatch: its `eqerr` poser matched none of the eight surfaces, so every early Su-30 calibration ran with surfaces at zero (§10.7).
- **Hierarchy:** the game hangs every part off one transform parent (`AttachInfo.parentPart`). The Ternion 1.0.1 blueprint makes the tail section a child of the engine, so a destroyed engine took the elevator, fin and rudder with it. `StructureFix` re-homes a part whose parent is an engine but which is jointed to the airframe, plus a prefix on `UnitPart.OnParentDetached` (`EngineLossKeepsTail`).

- **Scripts in other bodies:** the Su-30's `CanardSurface` AoA schedule (canard hinge leading edge down 1:1 with AoA to 30°) had never been switched off, because Blueprinter's split physics bodies put the script outside the `Aircraft` hierarchy (every log said "0 canard AoA-schedule script(s) found"). The scene's scripts are now searched by their aircraft field.
- **Mass re-layouts between versions:** Su-30VXN 1.0.1 added a 500 kg nose cone and re-massed eight parts; `Su30CgReference` gives them their 1.0.0 masses back (`UnitPart.ModifyMass`) so the FCS's tuning stays valid.
- **Stock scripts that write the same transform:** the Su-30 mod's `CanardSurface.LateUpdate` overwrites the leading-edge flap hinge the game's `HighLiftDevice` animates; anything that samples that transform must do it after every `LateUpdate` (the replay tool moved its recorder to `Application.onBeforeRender`).

8.22 Engine-out spin and automatic recovery (Ternion)

- See §2.11 item 12. The lesson from 0.1.12: gate a recovery device on the condition that defines the problem (**yaw saturated**, filtered > 0.5), not only on AoA and yaw rate; releasing an engine-out rolling pull at 50–55° had triggered it and shoved the nose to −8°. And make the recovery a rate target scheduled on AoA (−80 °/s at 75°+, 0 at 15°), not a constant −300 °/s².

8.23 Asymmetric loads: the coupled check

Problem. With stores hung on one side (a lateral CG offset of 10–40 cm), the AoA limit has to sit where the jet can still roll both ways, stop its roll and hold its pitch. The first answer (Ifrit 0.4.x, inherited by every jet) was a **one-axis check**: the load's roll and yaw demand against the surfaces' roll and yaw range, each surface frozen at its present deflection. In flight on 11 Nov it failed on every jet that carried a heavy load:

- **Su-30 0.4.1** (16–20 cm): "when I roll the AoA limit rises". The raw limit swung 12–75° with 42 and 68 jumps of over 10° in 0.5 s, and forced 4 and 7 nose-downs with full aft stick. Cause in the log: in a roll the elevators' symmetric part sat at +14° and the canards at +17°, and the load estimate fell from 30 to 19 deg/s² with them (fit over 1,991 ticks: −0.30 per degree of elevator, −0.26 per degree of canard, R^2 0.84). The check's answer moved with the stick.
- **Compass 0.3.1** (32–40 cm): the limit sat on its 12° floor, and on a 7° nose-down cap below ~62 m/s, while the jet held 25–30° in MPO with roll both ways.
- **Revoker 1.1.1** (27–34 cm): the limit followed throttle and speed, 17° at low thrust and 56° on afterburner, because the thrust × CG-offset yaw was in the load but the yaw nozzle was not in the authority, and a least-demand fallback returned 41–70° when no AoA passed.
- **Ternion 0.2.1** (the first one fixed): the limit collapsed to 7° or wobbled in pitch, from an extrapolated nose-down limit (`LiveAuthCap`) and a fixed 20 deg/s² nose-down margin.

Solution: the coupled check (`CoupledCap.cs` , Ternion 0.2.2, then the others). At each sweep AoA a small **linear programme** (LP: find the best mix of effector positions under linear limits) over every effector's moment points finds the most flight-path roll left each way with the load held, pitch held and no sideslip acceleration, with a design sideslip of 3° (`CplBetaDesign`). The pieces that made it work:

- **Count only what the allocator will really fly.** On the Ternion the first LP promised 18–60 deg/s² of roll the allocator never delivered: it used stalled elevator deflections and canards outside their attached-flow window (`CplAttachedOnly`). On the Su-30 the opposite: its allocator flies the stabs past their peak, so the LP's stab points follow the allocator's own `ClampStabToPeak` rule. `CplDebug` prints the LP's mix next to the real surfaces; repeat that check on every new jet.
- **Nozzles as LP columns** (Revoker 1.2.0, Su-30 0.5.0): each nozzle's moment grid is a column, and the thrust at the nozzle's present angle leaves the load. With the nozzles locked or the thrust low, the check runs without them (Revoker 1.3.0, `CplNoTvc`), and with a load on and the nozzles free at low thrust it takes the thrust at the nozzles' neutral plus their nose-down range (`CplNoTvNeutral` , found on the Ifrit, where the present nozzle angle fed back and flipped the cap 10 ↔ 34° at idle).

- **Thrust about the offset CG** is part of the load (`AsymThrustCgLoad`), faded in from 4 to 12 cm of offset so a symmetric jet gets exactly zero (on always, it changed every clean bench case by re-ordering equal-cost picks). It keeps the slow end honest: without it a slow pull reached 15° of sideslip under 60 m/s on the Compass.
- **The edge rule follows the curve's shape.** The demand curve is not monotonic on every jet (Su-30: a hump at 10–30°, a flat stretch at 40–50°, a cliff past 50–55°). A first-crossing search dropped the limit 51 → 12°; `FindTop` takes the lower edge of the over-threshold stretch that reaches the top of the envelope, with a latch. Look at the curve (`CplDbgT`) before choosing.
- **The share is per jet** (`AsymCoupledReserveMax`): the largest share of the roll range the load may take. Compass 0.75 (0.7 gave 11.6° of sideslip in a zoom to 30 m/s), Revoker 0.7 (0.8 with no TVC), Ifrit 0.75, Su-30 0.8; the Ternion, the first port, keeps `AsymLateralReserve` 0.35 capped at `AsymResMaxFrac` 0.8.
- **The nose-down limit reads the same LP** (its held-pitch figure), and its margin scales with dynamic pressure below 130 m/s IAS, judged at the predicted q, floor 6 deg/s² (`AsymPitchMarginIasRef`). Lifting the lateral cap without this exposed the old nose-down cap on the Revoker, which fell 23 → 12° under the jet in a pull: do both together.
- **Both signs of AoA, always** (`CplNegAlways`, Ternion 0.2.3): a cost-saving shortcut that solved the negative side only near zero AoA made the negative limit follow the flying (−18 in a pull, −40 in a push).
- **Cost:** one side solved per sideslip case (`side` field: 3 + 2 solves per point instead of 3 + 6), surface points from the per-tick grid in one kernel call (`CplGridCache`), the LP in Rust (`NativeCoupledCap`): the loaded sweep 37–52 % cheaper.

Results (bench; none flown yet): Su-30 51–53° at 16 cm and 46–51° at 31 cm with no jumps (was 12–75°); loaded reversals' AoA over the limit 31.3 → 1.1° mean. Compass 22–25° at 32 cm (was 12 / 7°). Revoker about 28° at 30 cm, steady with speed and throttle (was 12° fast, 40–70° slow on afterburner). Then the Su-30 asked for more: roll toward the load limited to what that side's reserve can stop in 0.8 s (`AsymStopTime`, bank after a release 47–117 → 27–44°), and the limit placed where AoA × usable roll is largest (`CplKnee`: about 45°, roll into the heavy side ×1.3–1.45). Tom: "I'd rather have my AoA a bit lower to get more roll. But don't make it pessimistic."

Why it is off on the F-22E (4.2.0, `AsymCoupledCheck` false): heavy loads gain (3,000 kg: 12–17° → 16–23°, steady), but at 1,000 kg the limit dipped 65 → 21 → 65° twice per pull below 55 m/s, and 1,500 kg roll spam at 600 km/h went to 50° AoA and 22–26° of sideslip. This jet's demand curve is flat near the threshold over a wide band, with a spike at 55° where the canted rudders' sideslip authority collapses. It needs its own campaign.

Open: Ifrit 12 cm loads (limit 63 → 31°, roll spam sideslip 3–5 → 6–8.5°); low-energy no-TVC cases on the Revoker and Ifrit (nozzles locked, slow over the top: roll-off with 9–19° of sideslip where the old 10–14° limits kept the jets faster); the allocator's flight-path-roll weight (1 against 8 for pitch and sideslip) leaves roll unmet that the LP counts, more so as speed falls.

8.24 Flying backwards: the mirrored nose axis

Problem (F-22E 4.2.0, 16:08 log, MPO tail slides): the lateral law commands a velocity-vector roll, which on the body is roll = rate × cos α and yaw = rate × sin α. Past 100° AoA a sense flip (`ReversedFlowRollSense`) reversed the whole command so the body roll followed the stick, but it reversed the yaw with it. Between 90 and ~135° the yaw is the bigger part, so the nose went against the stick: full left pedal at 100° gave yaw +25 °/s (nose right); full right stick at 134° gave yaw −23 °/s. At 100° the flip also reversed the command in one step with the stick held.

Tom's rule (12 Nov, for every jet): flying backwards, roll and yaw don't reverse. Left stick or left pedal = the nose rolls and yaws left toward the airframe's forward direction, on one axis blended by AoA, not two separate axes.

Solution (`ReversedFlowNoseAxis`, F-22E 4.3.0, then all six): the body-roll share is **|cos α|** instead of cos α; yaw stays sin α. Pure yaw at 90°, half and half at 135°, pure roll at 180°: forward flight mirrored about the wings (135° answers like 45°). |cos| is continuous through 90°, so nothing flips. The pace window, the capacity estimator and the TVC vector roll's start angle (Su-30) use the same axis. Negative AoA mirrors forward flight's negative side (like a push-roll). The sideslip hold stays on at speed (the airframe weathervanes and the hold brings the sideslip back; on the F-22E bench without it the jet hung at 27° of sideslip with every surface on its stop); on the F-22E it steps aside in a slow tail slide (below 20 m/s TAS) while the nose is being turned.

Result (bench, MPO tail slides, 1.5 s of input): roll and nose follow the input in every case on every jet. The cost is sideslip: peaks of 15–32° (were 2–9°), settling back within 0–7° by the end on most jets; the Compass at 186 km/h still had −21° 3 s after the release. Forward flight is bit-identical (|cos| = cos below 90°). Not flown.

8.25 A loaded roll reversal at one acceleration

Problem (F-22E 4.1.0, 10 Nov): "when I roll while holding pitch, then I reverse roll, the control surfaces stop the roll then they suddenly spiked in deflection to reverse the roll." In the 18:10 flight: 138–168 °/s² of braking, then 328–414 within 0.1 s of the zero crossing, the ailerons from 7–13° to their stops in four ticks.

Cause: two rules of the roll trajectory lived in two places. 1.7.4: a reversal flies one acceleration, "capped by what the yaw axis can follow", and the new roll starts at it. 2.1.6: with a load, a brake is limited to 0.4 × the yaw left after the load, in a block after the reversal branch, so a loaded reversal braked at the limit and started its new roll unlimited.

Fix 1 (4.1.1, "make inherent fix not on top"): the load's limit moved inside the reversal branch, so the acceleration carried into the new roll is the one the brake flew; nothing added. Bench: the step at the zero crossing 107 → 29 °/s² mean over 54 loaded roll-spam runs. Tom flew it: "still not fully fixed, it is just smoother".

Fix 2 (4.1.2): the 4.1.1 flight showed the remaining case: reversing **toward the heavy side**, the load and roll damping stop the old roll almost by themselves (ailerons at 4–6° for 0.35–0.5 s), then the new roll was built after the crossing (peak / brake 1.66–1.97). The 40 % stands for the rudders' travel: the load holds them on one side and they need 0.5–0.7 s to cross. So for a reversal toward the load, the acceleration is **one figure for brake and new roll**, within 60 % of the yaw left (the trajectory's own yaw figure), built over the time the rudders need to cross from where they are (measured positions, servo rate), and held through the zero crossing.

Ported to all five other jets (Ifrit / Revoker 1.0.1, Compass / Ternion 0.2.1, Su-30 0.3.8). **Still open:** it remains a roll-acceleration limit for the yaw's sake, which the governor rule says should not exist (§14); removing it needs the governor's rate cap to carry a loaded reversal first (bench with it deleted: sideslip 6.4 → 8.7° mean, 14.5° worst). Loaded roll spam sits at 6–11° of sideslip against a 4° margin. The bench's new loaded-reversal set (`bench/an/loadrev.sh`, 88 runs) is not in `suite.sh` yet.

8.26 Ground steering with the main brakes

Problem (Su-30, 11 Nov): "the turn stop is very slow"; "cannot drift on runway"; "why not make the diff brakes help in drifting" (no gates or timed logic).

What the game does (`LandingGear.FixedUpdate`, read from the IL): a main leg's brake is b = (braked ? brake : 0) [+ parking] + Clamp01(yaw × 0.3 × `differentialBrakeFactor`) for 3–30 m/s; `brakeStrength` follows b with a 0.2 s lag; the force is brake × μN along −velocity, total clamped to μN.

Solution: the main brakes become a second yaw effector for the ground INDI. The nose tyre takes the wanted yaw acceleration up to its grip; the excess goes to the brake on the right side, crediting each brake's lagged value next tick (`GroundIndiBrake`, full below 7.7 m/s, none above 12.9 m/s). A light nose tyre in a stop is steered on the contact-velocity angle (`GroundIndiLightNoseHold`). With the pedal in, the main on the pedal's side gets |pedal| × 0.5 (`GroundPedalBrake`), never below the stock term. Hooks: a prefix on `LandingGear` writes the leg's brake.

Gear data per jet (from `resources.assets` / Aryx's bundle): μ 0.7–1.0, nose lock 45° at 60 °/s; the F-22E has `differentialBrakeFactor` −1 on **both** mains and a 2.47 m track, so each unit of brake gives it less yaw than the Ifrit (3.94 m) for the same speed cost. Results (gyaw bench, no aero): release stops 7–17 kt 20 % shorter (10 kt: 1.67 → 1.33 s), full pedal sideslip 12.9 → 20.0° at 8 kt; 5–7 kt more speed lost in a stop. Ported to all six (F-22E 4.1.5, Ifrit 1.1.3, Revoker 1.3.1, Compass 0.4.2, Ternion 0.3.2). Not flown.

9. Adding the models

There are two different "models" in this project, and both have to be right:

- **The FCS's models (§9.1):** what the flight computer believes each effector does, built every tick from the live game state. These run *in the game*.
- **The bench's airframe model (§9.2):** an offline copy of the whole aircraft (parts, masses, airfoils, surfaces, engines) that the bench simulates so the real FCS code can be tested without the game.

9.1 The FCS's models

9.1.1 What a model is here

Each effector holds a **moment table**: for N deflections across its travel (the grid), the roll/pitch/yaw moment it would make right now (`M[i]`, a `Vector3` in the aero frame: x roll, y pitch, z yaw), plus `Mnow` (its moment at its actual deflection, from a full evaluation). The table is rebuilt every tick from the live flow, so it already includes dynamic pressure, local AoA, sideslip, stall and reversed flow. The allocator interpolates linearly between grid points.

INDI makes the *airframe* model unnecessary: everything that is not an effector shows up in the measured acceleration. So "adding a model" almost always means adding or fixing an **effector** model.

9.1.2 Adding a new kind of effector

1. Subclass `Effector` (see `IfritEffectors.cs`, `TvcYawEffector.cs`, `HingeFlapEffector.cs`, the Su-30's `CantedTvc.cs` for small examples):
   - in the constructor set `name`, `min` / `max` (travel, degrees), `lo` / `hi`, `regNorm` (the travel unit the costs are normalised by), `mu` (position cost), `lam` (move cost), and call `AllocGrid(N)` (13–21 points is typical);
   - in `Evaluate(ref EvalContext ctx)`: set `active`, `now` (actual position), `rate` (servo speed, deg/s), the reachable box `lo` / `hi` for this frame, fill `M[i]` for every grid point and `Mnow`. Use the game's own force equation for the part (see `SurfaceEffector.Evaluate` / `PartForce`), evaluated at the part's local flow, and convert world moments about the CG to the aero frame with `ToAero(rootInv, moment)`;
   - if the effector has a pair partner, set `partner`, `pairSign`, `symCost`, `antiCost`;
   - set `rollGroup` / `pitchGroup` if it should take part in fight costs.
2. Add it to the effector list in the `FcsController` constructor (`effs.Add(...)`), and keep a typed reference if the laws need it (`tvc`, `tvcd`, `splits`, ...).
3. **Output:** write its command into the game. Surfaces go through `Hooks.SurfacePostfix`; engines through the engine hooks; a part the game does not drive at all (a hinge) is moved by the FCS itself every tick (see `HingeFlapEffector` and `FlapSystem`). If the game also moves it, take ownership (the Revoker skips `HighLiftDevice.FixedUpdate` while engaged).
4. **Estimator group:** add it to `CapacityEstimator.Group` (§9.1.4).
5. **Everything that counts yaw or roll sources:** search the controller for loops over `surfaces` that should now include your effector (yaw-target clamp, yaw health, deceleron tiers, `yawEffs` on the Ifrit). The repeated bug in this project was a new effector that the allocator used but a limit or the estimator did not count (§13, F9).
6. **Telemetry:** add columns for its `now` / `cmd`.
7. **Bench:** the plant must move and model it too (§9.2.3), or the bench will fly a different aircraft.
8. **Threading:** the estimator and the reach plan solve on copies of every effector (`EffectorMirror`). Your class must copy every field its `Evaluate` /table reads in its copy method, and split `Evaluate` into a main-thread gather (anything that reads a Unity object) and a pure compute part, or switch the parallel paths off. The ports' clone-completeness check (bench `CloneCheck`, reflection over every field) is how the gaps were found.

9.1.3 Airfoils and the aero mirrors

- `AirfoilTable` samples the aircraft's own `Airfoil` (from `AircraftParameters.airfoils[part.airfoil]`) the way the game does. `FoilFor(part, prm)` picks it, through `StallStagger.TableFor` when stagger is on.
- **Every change the plugin makes to the game's aero must be mirrored in the FCS model** and the other way round: stall stagger (same remapped table), camber coupling (`host`, `hostFoil`, K), downwash at the stabs, lift reshaping. When the game and the model disagree, INDI hides it in symmetric flight and it shows up as creep, crossed surfaces or chatter under asymmetry (§13).

9.1.4 The capacity estimator's effector groups

`CapacityEstimator.Group(Effector e)` returns the group index used for the committed and probe terms:

- F-22E: 0 rudders, 1 stabilators, 2 other surfaces, **−1 for anything that is not a `SurfaceEffector`** (only its pitch-only TVC, so harmless there).
- Ifrit, Revoker, Compass, Ternion, Su-30: 0 rudders, 1 stabilators, 2 other surfaces, 3 any TVC (the Su-30's canted nozzles included), 4 splits, 5 hinged / anything else. **Nothing is dropped.**

If you start from the F-22E and your aircraft has any non-surface lateral effector, port the six-group version (copy the Ifrit's `CapacityEstimator.cs`). Gains (`GainZ` / `GainX` per group on the stock jets; `CapGainZ*` / `CapGainX*` on the F-22E) start at 1 and are fitted from logs: "achieved acceleration per table-modelled moment", identified on the F-22E with the bench's `fcsmom` mode and `bench/an/cap_gain.py` (F-22E bench only; port it if you need it).

9.1.5 Passive airframe probes

Some laws need the airframe's own moments at a *predicted* state, which INDI cannot measure in advance: the envelope sweep (`ComputeEnvelope`), the low-thrust cap (`AuthCapStep`), the asymmetric-load sweep (`AsymMomentAt`), the sideslip stiffness probe (`SlipMomentAt`), the hybrid INDI slopes (`NonSurfacePitch`), and the F-22E's passive telemetry (`AirframeMomentAt`). They sum the game's per-part forces at a hypothetical AoA/sideslip. They work on any aircraft because they iterate over the aircraft's own parts, but known limit: summing per-part charts at uniform flow under-reads the F-22E's sideslip stiffness 3–4× above 50° AoA. Treat them as relative, and validate against logs.

9.2 The bench's airframe model

The bench (`bench/`) compiles the **real** FCS sources (copied to `bench/fcs/` by the sync script, namespace changed to the bench's) against stub versions of Unity and the game (`bench/stubs/`), and flies them on a rigid-body **plant** (`bench/sim/Plant.cs`) that re-implements the game's force model (AeroJob / ControlSurfaceJob / engine thrust and nozzle slew, at 60 Hz). The plant is built from an **airframe JSON** extracted from the aircraft's Unity assets.

9.2.1 Extracting the airframe JSON

The JSON holds: the transform hierarchy, every part (mass, collider hulls, wing area, drag area, airfoil index, centre of lift, lift normal), every control surface (ranges, servo, split, brake), engines (thrust, vectoring, nozzles, speed curve), high-lift devices, tanks, gears and the airfoil curves.

- **Modded aircraft** (asset bundle inside the mod DLL, like Aryx's F-22E): cut the bundle out of the DLL at the `UnityFS` signature into `bundle.bin`, then (`bench/extract/`, UnityPy + scipy):
  1. `dump_mbs.py` → `mbs.json` (every MonoBehaviour by script class),
  2. `extract_mbs.py`, `extract2.py` → hierarchy, parts, surfaces, engines → `bench_airframe.json`,
  3. `extract_hulls.py` / `hulls.py` → convex-hull mass properties per collider (as PhysX computes them),
  4. `extract_damage.py <bundle.bin> <out.json>` → per-part hit points, joints and engine damage data (damage suite).

- **Stock aircraft** (in the game's `resources.assets`): `bench/calib/stock_extract.py <PrefabName> <out.json>` finds the prefab by name, follows its `AircraftParameters`, and resolves MonoBehaviours with a type tree generated from the game's `Managed/` DLLs. Edit the `D = ...` path at its top to your `NuclearOption_Data`. Collider meshes stream from `resources.assets.resS`, so stock jets keep box (AABB) mass properties unless you stage that file. `bench/extract/stock_damage.py` does the damage data.
- **Blueprinter aircraft** (the Ternion, the Su-30): the aircraft is a Blueprinter blueprint or a mod DLL with a bundle (the Su-30's `SolarSu30.dll` has its `UnityFS` bundle at offset 5928). `bench/extract/extract_ternion.py` / `extract_su30.py` plus `build_json.py`, `defs.py`, `dump.py` turn them into the same airframe JSON. Check the scripted behaviour while you are in there: the Su-30 extraction found stabs with `airflowChanneling` 0.6 (the game turns their inflow up to 34° toward the fuselage axis, so below ~34° body AoA the stabs give no AoA stiffness), canards on an AoA schedule, leading-edge flaps that are visual only, and a TVC model that sums pitch and roll in one channel clamped at ±20° (`claude/Su30VXN_airframe_extraction.md`).
- **Do not publish airframe files extracted from someone else's mod** without their permission (this kit leaves out the F-22E, Ternion and Su-30 files for that reason).

9.2.2 Point the bench at it

`BENCH_DATA=<path>` and, for the damage suite, `BENCH_DMG=<path>`. The wrappers default to `bench/data/airframe_cal.json` on the F-22E, `bench/extract/<jet>_airframe.json` + `bench/data/<jet>_damage.json` on the others (the Su-30 borrows the Ternion's damage file). `dotnet bench/bin/bench.dll info` prints mass, inertia, parts, surfaces and engines.

9.2.3 Teach the plant your aircraft

`bench/sim/Airframe.cs` has a few branches keyed to the reference jets (on the stock-jet benches: `ifrit` = the JSON has a stock FBW block, `revoker` = root named `Fighter1`): the surface order list, renames (the Ifrit's `elevator_L` → `Elevator_L` so the F-22E-style code matches), the rear fuselage part name used for CG trim, weapon bays, fuel/mass defaults (`sim/IfritTest.Defaults`: `FuelFrac`, `MassTarget`, `CgAftShift`), and which setup the plant applies (`ApplyIfritSetup` vs `ApplyRegistrySetup`, mirroring the plugin's `Registry.EnsureSetup`). Add a branch for your root name with your surface order and part names, and make the plant apply the same airframe changes your plugin applies (servo speeds, ranges, stagger, camber). New effector types need plant support too (e.g. hinged flaps: `rhinge` mode on the Revoker bench).

9.2.4 Calibrate against flight logs (equation error)

A bench is only useful once it agrees with the game. Method (see `bench/CALIBRATION*.md` on each reference):

1. Fly a varied log in game with `Telemetry` on (rolls, pulls, releases, push-overs, the falling leaf, at 2–3 speeds).
2. `bash bench.sh eqerr log=<log.csv> out=eq.csv` (or `dotnet bench/bin/bench.dll eqerr ...`): the plant is posed at every logged sample (TAS, AoA, sideslip, rates, density, attitude, every surface and nozzle at its logged position, thrust from the logged T/W) and its predicted angular accelerations and nz are compared with derivatives of the logged rates.
3. `calib/eqcmp.py` (RMS, lag, error maps), `calib/derivs_stock.py` (per-AoA-band derivatives game vs bench).
4. Fix root causes in code first (wrong tick rate, mass, thrust profile, a missing camber/stagger mirror, reference point), then fit a *few named* factors (`calib/fit.py`, `fit_leaf.py` for the high-AoA band): e.g. the Ifrit's single fitted factor was `CgAftShift` −0.05 m.
5. Hold out a log you didn't fit on. Report RMS per axis before/after.

Check the calibration tool before trusting the calibration. On the Su-30 (9 Nov) the first error table was a tool bug: `EqErr.Poser` matched log names (`elevator_L`) to blueprint names (`Left_stab_struct`) and all eight failed, so every "bench q̇" was the jet with all surfaces at zero, and `Plant.BuildCamber` had found no camber host for the same reason. With the poser fixed, the real error was a pitch moment growing with q̄; fitted as `FlapColInnerZ/OuterZ` 0 (the flap centre-of-lift offset, 1.3 m in the prefab, regressed to about −1.3 m in every

leave-one-log-out fit) and `ThrustLineDy` −0.115 m. Pitch equation error 21.9 → 6.2 °/s² (all samples), 12.5 → 2.4 above 40° AoA; closed-loop replays then followed the game down from 65 to 45° as it does. Conclusions drawn before that (the 0.1.5 "pitch-neutral" balance, the 0.1.8 wall tables, a 0.1.9 roll claim) were withdrawn (`CALIBRATION_su30.md` , `claude/Su30_bench_calibration_2026-11-09.md`). Still off: 15–40° AoA above ~550 km/h, 20–50 °/s² too nose-up.

Real numbers for scale: the Ifrit's first bench had roll RMS 16.5 deg/s² against the log; the root-cause fixes (60 Hz tick, real mass 25.5 t, profile thrust/drag, a camber host-name bug that made camber never fire on the bench) took it to 9.2, the one fitted factor to 9.6 roll / 10.0 pitch / 2.4 yaw. The F-22E's leaf calibration got yaw/roll error to 2.1/9.1 deg/s² held out at 40–70° AoA; its pitch through a fast 65 → 25° push is still 7–9 deg/s² short of nose-down.

10. The bench

10.1 Build

```
cd reference/<Jet>
bash bench.sh build      # sync the FCS sources into bench/fcs (namespace → bench's), regenerate the Plugin
stub
                         # from the jet's config file (tomcfg / tomifr / tomrev / tomcmp / tomtern /
tomsu30 .cfg),
                         # compile bench/bin/bench.dll
```

Always rebuild after editing the plugin source: the bench compiles a *synced copy*. "unknown key X" from a `Name=value` you just added means the sync was skipped. The config file the stub is generated from is the author's own config; replace it with your `BepInEx/config/*.cfg` to bench your settings. On the F-22E `bench.sh build` (= `regen.sh`) rewrites `bench/stubs/Plugin.cs` in dependency order, so expect that file to show as changed after a build.

The build also compiles the native core for Linux (`lib<Jet>_FcsCore.so`, `cargo build --release --offline`, no crates) and copies it into `bench/bin`. Without `cargo` the bench silently runs the C# twins (same numbers, slower); set `FCS_NATIVE_STATS=<file>` on a run to see which path ran (native solves, kernel calls, coupled-check LPs, fallbacks).

10.2 Single runs

- F-22E: `bash bench.sh one iasKmh=560 alt=1500 trimT=1 throttle=1 prof=<t:stick,...> rprof=<t:stick,...> csv=out.csv` (`prof` = pitch stick profile, `rprof` = roll stick, piecewise linear `time:value`).
- Ifrit, Revoker, Compass, Ternion, Su-30: `bash bench.sh fly ias=400 alt=1500 thr=1 T=6 pitch=0:0,0.2:1 roll=0:0,1.5:0,1.7:1 fcslog=out.csv` (`ifly` mode; `ias` in km/h). The Su-30 bench also has `takeoff` with `groll=` / `gyaw=`.
- Any `internal static` of `Plugin` (config), `FcsController`, `Allocator`, `SurfaceEffector`, `CapacityEstimator`, `Airframe`, `Plant` can be overridden on the command line as `Name=value`. This is how A/B tests are done.
- `info`, `effs` / `ieff` (effector authority maps), `iauth` / `rauth`, `stall`, `lift`, `em` (energy-manoeuvrability charts), `takeoff`, `fuel`, `ai` (the AI manoeuvre set), `matrix`. Read `bench/sim/Program.cs` for the full list.

10.3 Suites (regression tests)

`bash bench.sh suite <tag> [Name=value ...]` → `bench/out/out_<block>_<tag>.txt`; `bash bench.sh diff <tagA> <tagB>`.

Block	F-22E	stock-lineage jets	Reads
reversals / sweep	`rev`	`sw`	time to stop / reverse, peak sideslip, roll rate
corkscrews	`cork`	`cork`	roll profile, AoA/g overshoot, sideslip
regression set	`reg`	`reg`	rolling pulls, releases, push-overs (`betaPk` = peak sideslip)
utilisation invariant	`util`	—	does a held rate use the capacity it has (PASS/FAIL per AoA band)
damage	`dmg`	`dmg`	no fins / one fin / wing damage / engine out twins of the above

A suite takes about 10 minutes on 2 cores (cases are independent processes; more cores = faster). Per-tick CSVs land in `bench/out/<block>/<tag>/`.

Added on the F-22E since the first kit (F-22E bench, port them if you need them):

- **Curve-quality checks** (`bench/an/curveq.py`, 3.1.6), run on every manoeuvre since Tom asked for it. For each stick event on pitch and roll: onset (to 5 °/s of response), t50, rise, dip (worst drop below the running maximum), plateau (longest stall under 15 °/s²), secondary peaks, `cmdDip` (the law retreating from its own command), nozzle and stab reversals, and for stops the stop time, overshoot and rebounds. Gate: onset ≤ 0.15 s, dip ≤ 3 °/s, plateau ≤ 0.15 s, no secondary peaks, `cmdDip` ≤ 3 °/s, ≤ 1 rebound. A summary number can look fine while the curve has a notch; this is what catches it.
- **The low set** (`bench/an/lowset.py`): 48 scripted pushes from 60° and pulls from −45° at 55–135 kt with 0–15 °/s of rotation.
- **Push-over replays:** the 87 planned push-overs from the 5–6 Nov logs (and later 139 free replays), scored for sideslip, time to 20° AoA, nozzle and pitch-rate reversals, command spikes (`studies/push_scoring/`).
- **5-member ensembles** for roll-coupled cases: entry speed ±0.1 / 0.2 % (or a gain ×0.999…1.001). A ±0.1 % change in one gain moved one Ifrit case between 5.6° and 11.3° of sideslip, so a single run there is not evidence.

10.4 Replaying a flight log

`bash bench.sh replay <log.csv> <t0> <t1> <out.csv> [pinPitch=1] [Name=value]`: the plant is posed from the logged state at t0 and the logged stick inputs are played in, closed loop. The output carries bench and game columns side by side (`*_b` / `*_g`). `pinPitch=1` holds the pitch plane to the log (to study the lateral axis alone); `pinUntil` (F-22E) pins until a time then frees it. `bash bench.sh replaycmp` compares by column (stock jets). This is the most direct test of a reported fault: reproduce it on the bench first, then fix it.

10.5 Damage on the bench

`detach=<part>+<part>` flies with parts broken off (mass, fuel, aero removed, surfaces marked detached); the damage suite and `bench/sim/Damage.cs` apply hit-point damage the way the game does (`DAMAGE.md`).

10.6 The A/B rules

- **Identity:** a refactor or cleanup must give **byte-identical** suite output. Anything else is a law change.
- **Noise:** some cases flip between two outcomes with a 1 km/h change of entry speed (bistable releases, branch points). Before calling a difference real, re-run with several samples (e.g. altitude 1500/1510/1490/1520 m, or entry speed ±3–4 km/h) and compare distributions; ~0.5° of sideslip is typical null-perturbation noise.
- **"Without trade-offs"** means no suite case or replay metric worse beyond that noise.
- Report bench numbers as bench numbers; say "compiled / bench-only / flown".

10.7 What the bench cannot judge

It is one rigid body (the game uses ~37 jointed bodies on the F-22E, 34 on the Su-30), its derivative tables come from closed-loop logs (so they carry a bias), and some in-game effects never reproduced (an in-game stab-roll mismatch on the F-22E; the F-22E bench's coupling-limit nose-down table reads 20–60 °/s² more than the game's in 3.2.x entries). Neither stock bench can judge a high-AoA push-over law (Ifrit: +12 deg/s² nose-up bias at every AoA; Revoker: roll error 37 at 0–20°). Push-over pacing on the stock jets was flight-validated. Where the bench and the pilot disagree, the pilot has been right more often; when a flight shows a problem the bench doesn't, find the cause and upgrade the bench before fixing the law.

Bench bugs found on 9 Nov (BFM shootout; worked around in that run, **not fixed in the shipped benches**):

- the stock benches' `IfritTest.Defaults()` overrides settings the game applies: the Ifrit's drag relief zeroed, the Revoker's `StallStagger` off, the Ternion's LERX 1.0 instead of 1.13;
- the Ifrit and Revoker benches (`ApplyIfritSetup`) apply neither `ThrustScale` nor `DragScale`, and the Ternion path leaves out `DragScale`, so earlier **Buffed** bench numbers for those jets were really stock thrust

and drag;

- the F-22E and Ternion benches have no turbofan altitude lapse (the game's `Turbofan.SlowUpdate` multiplies altitude and speed curves); unchanged below 5 km on the F-22E, ~3.5 % high on the Ternion at 3 km;
- the Revoker bench has no airbrake (the extractor finds none);
- stepping `Plugin.AoAPos` mid-run no longer works as an energy-manoeuvrability method on the current laws (`authCapEff` slews). Fix them before using a stock bench for performance numbers.

10.8 Added since kit 2.0 (10–12 Nov, all six benches)

- **Native identity runs:** `bash bench.sh suite nat` (default: native), `... NativeAllocator=0` (all C#), and `... NativeAllocatorCheck=2` (every solve and every coupled-check LP done both ways and compared; the run counts differences). Zero differences and identical per-case CSVs is the pass mark for any change to the native core (§2.18, `native/README.md`).
- **Game-runtime checks** (`native/verify/`): `host.c` (a small Windows program that starts the game's own `mono-2.0-bdwgc.dll` under Wine with Unity's `-O=-float32`), `harness.cs` (F-22E: replays recorded solves through the plugin in modes `managed` / `auto` / `native` / `nativechk`, and `kernbench` times a kernel against its C# twin), `jetcheck.cs` (per jet: allocator self-test, coupled-check self-test, load from the cache folder), `multicheck.cs` (all six plugins in one process, each binding its own core), `csvcmp*.py` / `nsum.py` (compare CSVs / sum native stats). Su-30 extras: `flapcheck.cs`, `gearcheck.cs`, `cplcheck.cs` (the game-side hooks JIT-compile against `Assembly-CSharp`).
- **Asymmetric-load matrices:** `bench/an/asymmat.sh <tag>` (loads × speeds × pull/roll cases; `asymmat.py` tabulates the cap, AoA over the cap, sideslip, jumps); `asymnotvc.sh` (the same with the nozzles locked); Ternion `negload.sh` (the negative limit in pulls and pushes); Su-30 `stopmat.sh` (released-roll stops with a load).
- **Loaded reversals:** `bench/an/loadrev.sh <tag>` (88 runs: 34 single reversals toward and against the load, 54 roll-spam runs) with `loadrev.py` / `loadrev_cmp.py` / `lrcurve.py`. **The regression suite has no loaded reversal**: both 4.1.1 and 4.1.2 left all suite files identical. Run this set for any lateral-law change.
- **Flying backwards:** `bench/an/revflow_jet.py` (MPO tail slides, stick and pedal both ways at 115–145° AoA); F-22E `revflow.sh` / `revflow_an.py`.
- **Ground:** `bench/groundsim/` (Python yaw-plane models of the game's `LandingGear.FixedUpdate` read from the IL; `ildump.py`, `gsim2.py` exact tyre law, `gsindi.py`, `holdbench.py`) and the bench's `gyaw` take-off mode (pedal steps and release stops, §8.26).
- **Ternion calibration** (from a 0.2.1 flight log): the camber table the game really uses (`CamberGameTable`; roll error at 40–50° AoA 10.3 → 3.4), thrust yaw from the CG offset, stores found by part name. A heavy-load bench result is only as good as this calibration; the Compass bench still has none.

11. Tuning

11.1 Order

Tune from the inside out; each layer assumes the ones below it are right.

1. **The plant matches the game.** Surface ranges, servo speeds, TVC range, mass. (If the bench is off, calibrate it, §9.2.4.)
2. **Effector models match the game.** Signs, pairs, camber/stagger mirrors, `noPitch` / `linearPitch`, estimator groups. Symptom of a mismatch: creep with the stick released under any asymmetry, surfaces crossed at their stops, chatter.
3. **Rate loops** (`PitchRateGain` 5, `RollRateGain` 6, `YawRateGain` 2.5, `SideslipGain` 1.3): rarely need changing. INDI makes them nearly airframe-independent.
4. **Envelope:** AoA and g limits, `MaxPitchRate`, `NoTvcAoACap*`, `FinLossAoACap`, engine-out caps. Find limits on the bench first (e.g. the F-22E's −50° negative limit came from an inverted-corkscrew matrix: −55/−60/−65 departed).
5. **Lateral feel:** `RollRateGainLowIAS/HighIAS`, `MaxRollRate`, `MaxYawRate`, `LowIasYawBoost*`, `CoupledAxisAoA`, `PedalSideslip`.
6. **Governor margins** (player file, section 6): `RollStopSideslip`, `RollPaceRateBlend`, `PullRollTrade`, `PushoverSideslip`.
7. **Per-jet fits:** push-plan constants, estimator gains.
8. **Allocator costs** last, and only with a reason from a log: `TvcMoveCost`, `LateralStabReserve`, `YawAxisPriority`, `ReleasedStickPitchPriority`, `RollFightCost`, `RudderToeCost`, `StabParkEscape`, `Settled*`.

11.2 What the main knobs do

Knob (file)	Default F-22E / Ifrit / Revoker	Raise it →	Lower it →
`StickSmoothing` (player)	0.06 s	softer, laggier stick	twitchier
`MaxPitchRate` (dev)	60 / 48 / 48 °/s	faster nose at full stick	slower
`RollStopSideslip` (player)	4 / 6 / 6 °	faster stops/reversals, more sideslip	crisper coordination, slower rolls near stops
`RollPaceRateBlend` (player)	0.408 / 0.4 / 0.65	less steady-rate cap: faster mid-AoA roll, more reversal sideslip, wobblier nose	more cap: slower roll, less sideslip
`PullRollTrade` (player)	0.7	rolling pulls give up more roll to keep pitch	more pitch given up to keep roll
`PushoverSideslip` (player)	−1 (= `RollStopSideslip`) / 8 / 8 °	faster push-overs from high AoA, more sideslip at the bottom	slower, tighter
`CoupledAxisAoA` (dev)	36°	pedals stay a sideslip command to higher AoA	earlier single-axis behaviour
`PedalSideslip` (dev)	15°	more pedal authority	less
`LateralStabReserve` (dev)	0.7	stabs kept for roll/yaw with lateral stick in	stabs back to pitch
`ActuatorDamping` (dev)	0.06 s	smoother servo starts/stops	snappier (at 65° AoA the stab start lag is about 60 ms of it; 0.02 s is the low-speed proposal, not built)

Knob (file)	Default F-22E / Ifrit / Revoker	Raise it →	Lower it →
`PushOverMinRate` (dev, F-22E)	23 °/s	faster planned push-overs at very low speed, more sideslip	0 = the plan's own rate (3.1.3)
`ForeLiftSchedule` / `AftLiftSchedule` (dev, F-22E)	X2e strings	(a fitted pair: refit, don't hand-edit, §8.15)	empty = 3.1.9 airframe
`TradeCapTau` (dev)	0.05 s (Su-30 0.45)	trade ignores short capacity dips (match the transit of your yaw effector)	follows every dip: pitch-target hitches

Tom's lateral-governor values are shipped as the defaults of each jet. Don't change someone's tuning values without asking them; if you change a default, migrate existing config files deliberately (the code has examples: search `migrated once`).

11.3 Method

- **Judge the curve, not only the peak** (Tom, 6 Nov: "instant pitch response from stick input, an actual smooth pitch rate curve, no spike no hitch no sudden slow down, no chatter, no hesitation"): run the curve-quality checks (§10.3) on every manoeuvre you change, and look at the time histories of the motivating events.
- One change at a time; bench suite before and after (`bench.sh suite a` / `suite b X=…` / `diff a b`), with samples for noise.
- Replay the logged events that motivated the change (§10.4) and report them in a table: before → after.
- Fly with `Telemetry` on; compare the log with the bench replay of the same window.
- Keep a release note per version: what changed, why, bench numbers, flown or not (see any `release-notes/` for the format).

11.4 Fitting the push plan to your aircraft

The push plan's yaw shed rate is a fit to *your* jet's logged push-overs:

1. Fly 6–10 push-overs from 60–70° AoA at 150–250 kt, released and held.
2. From the log, for each push measure the yaw-rate shed (deg/s²) and the IAS.
3. Fit shed = K0 + K·IAS^1.5 (IAS in m/s) and set `PushPlanK0`, `PushPlanK` (the F-22E fit was 0 / 0.028 over 93 pushes; the Ifrit 9.6 / 0.0144, rms 3.7 deg/s²; the Revoker a constant 40). Report the rms.
4. Set `PushoverSideslip` explicitly; its −1 default follows `RollStopSideslip`, which may be wrong for a push.

11.5 Fitting the estimator gains

Gains are "achieved angular acceleration per table-modelled moment" per effector group. On the F-22E they came from the bench `fcsmom` mode (the FCS's modelled moment at logged states) regressed against the logged accelerations (`bench/an/cap_gain.py`; target: composite within 1.5× of the realised peaks in reversals). Logs flown with the lateral loops active carry a closed-loop bias; use varied manoeuvres.

PART III

Reference

12. Troubleshooting

12.1 It doesn't attach

Symptom	Look at	Likely cause
No "attached" line in `LogOutput.log`	`Registry.IsF22`, `Hooks.FilterPrefix`	aircraft match wrong; `Enabled` false; host doesn't run/allow the FCS (multiplayer); not `LocalSim`
"waiting to attach (attempt N)" repeating	`FcsController` constructor	fewer surfaces than `expected N`, engines not found, parts not spawned yet; gives up after 1500 attempts
"could not attach; stock flight controls stay active"	the exception text	a null part, a surface without an `AeroPart`, a reflection field (`Acc`) missing after a game update
"FCS fault, reverting this aircraft to stock flight controls"	the stack trace	an exception inside `Tick`; the FCS fails safe to stock FBW for that aircraft
Airframe setup "incomplete, will retry"	`Registry.EnsureSetup` completeness check	surface/engine count condition not met
Works offline, stock in multiplayer	`MpPolicy`	host has `HostAllowFcs` false or no plugin

12.2 It flies badly

Symptom	Likely cause	Where / what to do
A surface pinned at its stop, others crossed against it, stick centred	model/game mismatch (camber/stagger mirror missing, wrong sign), stall plateau, fight cost off	check the effector's modelled vs live moment (telemetry `anchR/anchY`, `nuR/nuY`, `unR/unY`); §13 F4, F5
Slow roll or yaw creep with the stick released after damage / asymmetric stores / engine out	table vs live mismatch at the surface position; released-stick fight cost holding the trim	`AnchorLateral` (on); `nuR` / `nuY` should sit near 0; the stock jets' open fight-cost item (Revoker 0.8.2 notes)
Surfaces chatter at saturation	the target is out of reach and the solver jumps between near-equal solutions	clamp the target to the reach (as the yaw target is); §13 F3
Nose shoves down / pitch dip after releasing a roll	pitch traded for roll by the allocator after release	`ReleasedStickPitchPriority`, the reach pass; §13 F2
Weak roll and slow reversals at low speed	regularisation not scaled with $\bar{q}$	`LowQAllocatorRelief`; §13 F1
Roll rate uneven at low IAS / low AoA	a consumer reading a *measured* rate where it means a *command*	the trade's ask gate (`qSweepAsk`); §13 F10
Rolls much slower than before after a port	the capacity read is missing an effector group (reads ~0) → rate cap on its floor	`CapacityEstimator.Group`, `ce*` telemetry columns; §13 F9
Big sideslip in roll stops/reversals	`RollStopSideslip` / `RollPaceRateBlend` too loose; yaw capacity over-read	compare `capZeP/N` with the realised yaw acceleration in the log
Push-over from high AoA hitchy or delayed	push plan constants not fitted for this jet; pitch waiting on the yaw	§11.4; `ppOn`, `ppQ`, `qCmd` columns
AoA overshoots the limit	capture braking can't see the real nose-down authority (no thrust, wrong TVC model)	`AoACaptureThrustBrake`, the low-thrust cap (`authCap` column)
AoA runs away at idle / no TVC	low-thrust cap not engaging, or no-TVC caps too high	`AuthCapStep`, `NoTvcAoACap*`
Nose wobble after a snap roll	released-stop yaw brake overshooting	F-22E 2.31.0 approach

Symptom	Likely cause	Where / what to do
Sideslip departure in an inverted corkscrew	airframe limit: canted fins deep post-stall	lower `AoALimitNegative` (F-22E: −50)
Airbrake opens with brake + countermeasures on the same bind, `ThrottleIndependentAirbrake` off	the FCS must return the brake fully to vanilla when off	check the airbrake hooks honour the switch
FPS drop with two FCS mods loaded	each re-registered the aero charts	fixed in all jets (chart ownership, Revoker 0.4.3 / F-22E 2.16.1); keep that code
Push-over from very low airspeed feels frozen for a second	the push plan's IAS fit planned a very slow rate	the push-over floor (`PushOverMinRate` , §2.10); `ppFloor` column
Push-over smooth at first, then spikes / dips / goes slow near the bottom	hand-offs between the push plan and the sweep caps	the 3.1.6–3.1.7 fixes (§19.1); run the curve-quality checks (§10.3)
Roll dips at 45–55° AoA on a relaxed airframe	the coupling roll limit is paid from the airframe's nose-down, which the airframe change removed	give the nose-down back in the airframe (F-22E X2e, §8.15); don't loosen the limit
AoA overshoots the cap at low throttle after an airframe change	lower stab lift slope + no natural nose-down to help the brake	§8.15 (still open on the F-22E: 50–51° against 45)
Hitch in the pitch target in rolling pulls on a TVC jet	the trade follows short dips of the yaw capacity while the yaw effector crosses its travel	`TradeCapTau` matched to that effector's transit time (Su-30 0.45 s)
A jet with no TVC can't reach its AoA limit, or MPO stops short	TVC-era caps / budgets that read 0 without nozzles	§8.14
A tail section disappears when an engine is destroyed (Blueprinter jet)	the part's transform parent is the engine	`StructureFix` (§8.21)
Jet pitches up on the take-off roll with the stick neutral	thrust line well below the CG	`ThrustLineAtCg` (§8.18), not a closed-loop nose-wheel hold
Jet tips onto its tail while taxiing	CG too close to the mains	move the CG forward and restore the static margin with foreplane / LERX area (§8.18)
With a one-sided load the AoA limit moves with the stick, throttle or speed	a one-axis check with surfaces frozen where they are, or a load term without its effector	the coupled check (§8.23); `cplU` / `cplRp` / `cplRn` columns
With a load the limit sits on its floor where the jet could fly higher	first-crossing search through a hump in the demand curve	`FindTop` (§8.23); plot the curve with `CplDbgT`
Loaded roll reversal: the roll stops, then the surfaces spike	the load's brake limit applied outside the reversal branch	§8.25
Flying backwards, the nose goes against the stick or pedal	the old reversed-flow sense flip	`ReversedFlowNoseAxis` (§8.24)
Ground turn stops slowly, or the jet won't pivot / drift	nose tyre at its grip, mains not used	`GroundIndiBrake` , `GroundPedalBrake` (§8.26)
3 ms hitches every few seconds with telemetry on	the CSV row's garbage (17 KB per tick)	telemetry off (default); §2.16

12.3 It builds badly

- **Different md5 than expected:** the assembly name comes from the **output file name**; build to exactly `<Namespace with _>_FCS.dll` . Whether `Properties/` and `System.Runtime.CompilerServices/` are included depends on the source dir given to `build.sh` : since kit 3.0 **all six releases are built from the plugin folder** (§15.1). From `.` the result differs and the native core is not embedded.
- **The plugin builds but `-resource:` fails:** `build.sh` embeds `native/prebuilt/<Lib>.dll` ; the clone tool renames it with the library (§7.2). Keep the prebuilt core next to the source.

- **Bench "unknown key X":** you edited the plugin but did not `bench.sh build` (the bench compiles a synced copy).
- **Bench compile error after adding a file:** add it to the sync script's file list (`bench/sync.sh` / `isync.sh` / `rsync.sh`) or stub what it needs in `bench/stubs/`.
- **`/tmp/Plugin.stub.bak` / hard-coded `/root/w/...` paths in analysis scripts:** several scripts in `bench/an`, `bench/calib`, `bench/scripts` were written for one machine; the main `bench.sh` commands use relative paths and `BENCH_DATA`.

12.4 Verify what is installed

Before reading any log: check `BepInEx/LogOutput.log` for the plugin's version line, and compare the installed DLL's md5 with the release. (On the author's machine a jet ran an old build for days while the notes said otherwise; on 6–7 Nov a mod manager modpack put F-22E 3.0.3 back twice and removed the Compass FCS.)

12.5 Problems that are not the FCS

Symptom	What it really was
Fuel burns much faster than the profile says (1.5–1.75× stock)	another mod's prefix on `Aircraft.UseFuel` drawing a second time; fixed by the two-prefix hook (§2.17). Use a diagnostic that logs the patch owners on the method
Control surfaces look choppy at low speed, "like lower fps"	the game draws surfaces once per physics tick (~59 Hz) with no interpolation; fast slews at low speed make it visible. Not FCS CPU (§19.1)
A split-second before the stabs move when rolling at 65° AoA	mostly servo speed and 65° of travel from a stop; the FCS's command smoothing is ~10 % of it (§19.1)
The game crashes 15–50 s into a mission	a Direct3D 12 device hang (driver / injected upscaler / `-force-d3d12`), not a plugin (§19.7)
A fix "didn't work"	an old DLL installed by a mod manager (§12.4)

12.6 The native core

`LogOutput.log` says	Meaning / what to do
the core's ABI number and "self-test 200 / 200"	loaded and checked; the Rust paths are in use
a self-test failure, or "falling back to C#"	the core is off for that part (allocator, kernels or coupled LP); the jet flies the C# twins: same laws, more CPU. Report it with the line
the cache file was rewritten	the cached copy in `BepInEx\cache\<Lib>\<sha16>\` did not match the embedded one (corrupted or replaced) and was written again; harmless
no line about the core at all	an old plugin (before the one-DLL releases), or a separate `<Lib>.dll` from an older install shadowing nothing: delete stray core DLLs from the plugin folder

`NativeAllocator` 0 forces all C# (to rule the core out); `NativeAllocatorCheck` 2 solves both ways and logs differences (expect small ones in the game: §2.18). Mod managers: on 11 Nov every FCS mod except the F-22E was listed in `disabledPlugins` and the Ternion still ran 0.2.0 from a nested folder; check `BepInEx\plugins` for nested copies and the manager's disabled list.

13. Failure modes we have hit

Most hard bugs were not in the control laws but in how the allocator's costs and the governor's inputs interacted with the plant. Each was found in flight telemetry, reproduced on the bench and fixed at the cause.

F1. Weak roll, slow reversals at 117 kt

- *Cause:* regularisation in travel units while the error term scales with $\bar{q}^2$
- *Fix:* scale regularisation by $(\bar{q}/\bar{q}_{ref})^2$, floor 0.02 (`LowQAllocatorRelief`)

F2. −37°/s nose shove after a release during a pitch arrest

- *Cause:* the solver accepted a pitch miss for roll; one stab went to its stop, 0.2 s away from where pitch needed it
- *Fix:* released-stick pitch priority; later the reach pass + two-horizon pick

F3. Chatter under saturation

- *Cause:* unreachable yaw target: two near-equal solutions
- *Fix:* clamp the target to what can be reached

F4. A stab stuck at −30° for 15 s at 31–35° local AoA

- *Cause:* on the stall plateau the modelled moment barely changes with deflection, so moving back "shows no gain"
- *Fix:* stall windows: keep each stab on the attached side (`StabParkEscape`, `SettledWindow`), search beyond one frame's reach

F5. Crossed sets: wing and rudders at their stops against the stabs, stick centred

- *Cause:* INDI cancels airframe-model error but not *effectiveness*-model error; the game's stab split made 60–80 % of the modelled roll, and the allocator corrected with that same over-rated effector every cycle
- *Fix:* fight cost with released stick; later the live-node anchoring; a plant mismatch knob (stab lift ×0.8) reproduced it

F6. The fight cost fought roll stops, priced the nose-down split, froze the stabs with fins gone

- *Cause:* a cost term's side effects
- *Fix:* reference sign from the roll target; wait for pitch to settle; count only the stab *split*'s roll

F7. VMC limiter never acted with both fins gone

- *Cause:* saturation judged against a target already clamped to the reach
- *Fix:* count the clamped-away demand as saturation

F8. Nozzles swinging ±4° in low-AoA rolls with q held

- *Cause:* roll deflections on nonlinear curves make pitch moment; the allocator cancelled it with the cheapest effector (correct, but unwanted)
- *Fix:* `TvcNeutralHold` moves the steady part onto the stabs

F9. After the first governor port the Ifrit's and Revoker's rolls were much slower everywhere (falling-leaf yaw 75 → 40–45°/s)

- *Cause:* the capacity read counted rudders only; the Ifrit has none and both jets' trailing edges are their high-AoA yaw axis
- *Fix:* count every effector, on the sideslip axis (six groups), rudW from the probes

F10. Uneven roll at low IAS

- *Cause:* a consumer reading a measured rate as a command

- *Fix:* the ask gate

F11. Slow creep with stick released under asymmetric trim

- *Cause:* table vs live mismatch at the surface's position acted as a standing pilot input
- *Fix:* live point as a node of the roll/yaw table (`AnchorLateral`)

F12. Yaw read as sideslip, wobble

- *Cause:* air data at the nose rigidbody, 2.5 m ahead of the CG
- *Fix:* air data at the CG

F13. Released push-over hitch, many "fixes" made it worse

- *Cause:* special-case devices stacked on each other, each pacing to a different read
- *Fix:* one governor on one capacity; a planned push-over

Added since the first kit (5–10 Nov):

F14. No-fin rules acting in straight flight: roll twitch, 48° sideslip departure, roll creep

- *Cause:* "which side is the roll" from `Mathf.Sign(0)` = +1; a release rule reading a banked turn's yaw as rotation at 4° AoA
- *Fix:* gate each rule on the state it is for (a roll being stopped; above 15° AoA with real rotation)

F15. A protection outrun by the pilot after a hit

- *Cause:* the asymmetric cap tightened at a fixed 10 °/s after its target dropped 50°
- *Fix:* tighten rate scaled by the gap

F16. A protection gated on a symptom that swings

- *Cause:* the Compass's damage protections keyed on the lateral CG offset, which swung −0.28 ↔ 0 m with a hanging rudder
- *Fix:* gate on the damage itself (a lost control surface)

F17. A switch at a threshold the state hovers around

- *Cause:* the push governor's sweep cap switched on/off at 5 °/s of velocity roll while the jet rotated at 4–7 °/s: command spikes and hesitant TVC
- *Fix:* fade in over a band (5 → 15 °/s); the same for the far-side cap (20 → 40)

F18. Hand-offs between two controllers that each read differently

- *Cause:* push plan → governor at 10° AoA came back at a 6 °/s floor; caps read for the other AoA side for one tick; a cap taking back rate the push already had
- *Fix:* keep the handed-over side open, read caps for the side they were sized for, never retreat from reached rate in a held push (§19.1, 3.1.7)

F19. A guard that turned a floor into slow-then-fast

- *Cause:* the 3.1.4 sideslip guard faded the push-over floor to zero mid-push
- *Fix:* remove the guard; a plain floor (Tom: some more sideslip is fine)

F20. Two mods' prefixes both doing the work

- *Cause:* HarmonyX runs every prefix even after one returns false; two fuel prefixes at the same priority both drew
- *Fix:* scale at `Priority.First`, draw at `Priority.Last` only if nobody drew (`__runOriginal`)

F21. TVC-era rules on a jet without TVC

- *Cause:* fixed no-TVC caps, a 40 × T/W term in MPO's budget, `LateralStabReserve` 0.7, the stick cancelling the AoA limit — each assumed nozzles that aren't there
- *Fix:* off or zero on a `noVecJet` (§8.14)

F22. An effector left out of a sum it belongs in

- *Cause:* the Su-30's sideslip probe summed surfaces only, so with the nozzles carrying the lateral authority past 60° it read the authority halved by 4° of sideslip; the Compass's fixed nozzles were skipped by the VMC model (yaw read 0)
- *Fix:* count every effector (rule 3 of §14, again)

F23. A low-pass sized for another jet's effector

- *Cause:* the trade's capacity filter (0.05 s, the F-22E's rudder transit) followed every capacity dip while the Su-30's nozzles crossed their 0.46 s travel: pitch target 43 → 23 → 60 °/s
- *Fix:* 0.45 s on that jet

F24. An airframe change that removed something a limit was spending

- *Cause:* the F-22E's X2 airframe removed the natural nose-down at 45–55° that the coupling roll limit pays the roll's pitch-up from; roll there fell to 0.48–0.82 of the max-AoA rate
- *Fix:* put the nose-down back in the airframe (X2e), not loosen the limit

F25. A worker copy missing a setting

- *Cause:* the estimator / reach-plan worker copies didn't take the Ifrit's per-tick allocator settings; results differed
- *Fix:* copy every setting; a reflection check over every field (`CloneCheck`)

F26. Bench conclusions from a tool that silently did nothing

- *Cause:* the Su-30 `eqerr` poser matched no surface name, so the "game vs bench" table compared the game with an unposed jet; three versions' conclusions rested on it
- *Fix:* fixed the poser, recalibrated, withdrew the conclusions (§9.2.4)

F27. A part's script outside the aircraft

- *Cause:* Blueprinter's split bodies put the Su-30's canard-schedule scripts outside the `Aircraft` hierarchy, so "switch it off" found 0 of them for eight versions
- *Fix:* search the scene, matched by the script's aircraft field; log the count (it said 0 all along)

F28. A recovery device that fired on the wrong condition

- *Cause:* the Ternion's spin recovery keyed on AoA and yaw rate only; an engine-out rolling-pull release at 50–55° met both
- *Fix:* require the condition that defines a spin here (yaw saturated)

F29. A limit computed with the surfaces frozen where they are

- *Cause:* the one-axis asymmetric-load check read each surface's range from its present deflection, so the pilot's own pitch and roll moved the answer (Su-30: limit 12–75° with the stick; fit R^2 0.84 against elevator and canard position)
- *Fix:* the coupled LP over every effector's whole range (§8.23)

F30. A load term without its effector

- *Cause:* the Revoker's thrust × CG-offset yaw was in the load, the yaw nozzle not in the authority, so the limit followed throttle
- *Fix:* nozzles as LP columns; whatever makes a load term must also be counted on the authority side

F31. A model promising what the allocator never flies

- *Cause:* the Ternion's first LP counted stalled surfaces and canards outside their attached window: 18–60 deg/s² of roll that never came. The opposite on the Su-30, whose allocator flies stabs past their peak
- *Fix:* the LP uses the allocator's own rules (`CplAttachedOnly`, `ClampStabToPeak`); compare the LP's mix with the flown one

F32. A first-crossing search on a non-monotonic curve

- *Cause:* a hump in the demand curve at low AoA put the limit on its floor (Su-30 51 → 12°, Ifrit at 37 m/s)
- *Fix:* `FindTop`; plot the curve before choosing an edge rule

F33. A rule applied after the branch it belongs to

- *Cause:* the loaded brake limit sat in a block after the reversal branch, so the brake was limited and the new roll was not (F-22E 4.1.0)
- *Fix:* the limit moved inside the branch (§8.25); look for "applied twice / applied after" when a curve has a step

F34. A sign flip that reversed more than it meant to

- *Cause:* the reversed-flow sense flip reversed the yaw part with the roll part, and switched in one step at 100°
- *Fix:* the mirrored axis $|\cos\alpha|$ (§8.24): continuous, nothing flips

F35. A cost-saving shortcut that made a limit follow the flying

- *Cause:* the Ternion solved the negative side only near zero AoA; the negative limit read −18° in a pull and −40° in a push
- *Fix:* solve both signs always (`CplNegAlways`); a shortcut must be proven identical, not "close"

F36. The controller's own output fed back into its limit

- *Cause:* the Ifrit's check took the nozzles' present angle (which the controller sets) as given: the cap flipped 10 ↔ 34° at idle
- *Fix:* `CplNoTvNeutral`: free nozzles counted from neutral with their range

F37. A config revision bound after the migration that clears unbound keys

- *Cause:* Su-30 `Su30ConfigRevision` bound after `MigrateOldConfig`: every launch dropped it, so "moved once" steps never ran
- *Fix:* bind revision keys first (0.3.4)

F38. Native code used before its self-test, or a self-test that fails only on the game's runtime

- *Cause:* the F-22E tables fell back to C# on the first tick while the self-test ran (4.1.3); the Ternion's `CplNear` test produced NaN on Mono (`-O=-float32`) and switched the core off
- *Fix:* self-test before the first use; run the self-tests on the game's Mono (`native/verify/`), not only on the bench

F39. A diagnostic that costs more than what it measures

- *Cause:* the telemetry row made 17 KB of garbage per tick and every 3 ms hitch
- *Fix:* telemetry off by default, togglable in flight; measure with the switches flipped one by one in one flight

Lesson (from the paper): every cost term in a nonlinear allocator needs consistent units with the error term, an explicit statement of which state it must not act in, and a test with a deliberately wrong model. Several failures were **bistable**: a case landed on either branch depending on a 3 kt change of entry speed, so release sets are flown at several speeds.

14. Design rules

These are the rules the author set and the code has been held to. Follow them in a port; an AI working on this code must (see `AGENTS.md`).

1. **Measured state over schedules.** Limits and authorities come from the live airframe (surface models at local AoA, live thrust, measured accelerations), not offline tables. Keep gain scheduling to a minimum. Deliberate feel tuning (e.g. low-speed lateral balance) is allowed and labelled as such.
2. **No on-top fixes.** Don't add a special-case device for a symptom; replace devices by modelling what they cover. No new gates or "if this manoeuvre then…" logic. If you release an axis, the jet must try to stop that axis.
3. **Count everything on the sideslip axis, at every AoA.** Every effector and every passive term (dihedral effect, weathercock stiffness, forebody side force, damping) affects yaw and sideslip stiffness. Score effectors by $Mz \cdot \cos\alpha - Mx \cdot \sin\alpha$ at the actual AoA. A number quoted at one AoA does not describe the jet.
4. **The governor slows the commanded rate only,** never the roll acceleration or deceleration. At low–mid AoA slow the velocity-vector roll; at high AoA slow the pitch. Sideslip from gravity pulling the jet sideways is the only acceptable failure.
5. **A positive pitch command with the stick full forward is a hitch,** whatever the pace behind it. "Smaller" does not ship.
6. **Rate regressions must be caught by the bench,** on a newly ported airframe too.
7. **Identity means byte-identical** bench output. Refactors and cleanups must be identical; anything else is a law change.
8. **Never overstate verification.** Say compiled / bench-only / flown, with numbers.
9. **Don't change someone's tuning values unasked.**
10. **Versioning:** minor bump for a law change or an airframe change, patch for a fix, telemetry or cleanup.
11. **Judge the curve** (6 Nov): instant response, smooth rate, no spike, hitch, sudden slow-down, chatter or hesitation, on every manoeuvre, not only the one being fixed. Use the curve-quality checks (§10.3).
12. **Roll-coupled bench cases are chaotic:** use 5-member ensembles before calling a regression.
13. **Threading never changes the flying:** anything moved off the main thread must be byte-identical with its switch on and off; no Unity object off the main thread; the main thread always takes part.
14. **Decide "airframe or FCS?" with data** (a fit to the game's own accelerations) before changing either, and say which one a fix is in (§8.15, §8.19).
15. **The native core is the C# twin, bit for bit on the bench:** same expressions, same order, same float/double conversions, no fused multiply-add, no maths library except `atan2` written as C# does. A new field means an ABI bump on both sides. The C# stays the reference and the fallback (§2.18).
16. **A model inside the FCS counts only what the allocator will really fly** (the coupled check's LP: §8.23, F31).
17. **A reversal is one acceleration** from the stick flip to the new rate; any limit on it lives inside the reversal branch (§8.25).
18. **Flying backwards, stick and pedals point the nose,** on one axis blended by AoA ($|\cos\alpha|$ roll share, §8.24). Tom's rule, 12 Nov, for every jet.
19. **Measure performance in the game,** switch by switch, before claiming a saving (bench µs on .NET are not Mono µs).

15. Building, versioning and releasing

15.1 Build the game DLL

```
export NO_MANAGED="<Nuclear Option>/NuclearOption_Data/Managed"   # or edit the default path at the top of
build.sh
export NO_BEPINEX_CORE="<Nuclear Option>/BepInEx/core"
cd reference
(cd F22E-FCS-4.3.0    && bash build.sh Aryx_F22E_StrikeRaptor.FCS Aryx_F22E_StrikeRaptor_FCS.dll)
(cd Ifrit-FCS-1.3.0   && bash build.sh Ifrit_KR67.FCS Ifrit_KR67_FCS.dll)
(cd Revoker-FCS-1.4.1 && bash build.sh Revoker_FS12.FCS Revoker_FS12_FCS.dll)
(cd Compass-FCS-0.5.0 && bash build.sh Compass_TA30.FCS Compass_TA30_FCS.dll)
(cd Ternion-FCS-0.4.0 && bash build.sh Ternion_FS3.FCS Ternion_FS3_FCS.dll)
(cd Su30-FCS-0.6.0    && bash build.sh Su30_VXN.FCS Su30_VXN_FCS.dll)
```

All six are built from **the plugin folder** (kit 2.0's "." builds for four of them are gone). `build.sh` embeds `native/prebuilt/<Lib>.dll` as a resource (`-resource:$CORE,<Lib>.dll`); that prebuilt core is the one the release carries. `build.sh` calls the .NET SDK's Roslyn `csc` directly (C# 9, deterministic, against the game's Mono assemblies). With the same inputs it reproduces the release DLLs byte for byte (§17). The Dev DLL is compiled from `DevUnlock/FcsDevPlugin.cs` directly (`build.sh` excludes `*/DevUnlock/*` , so `build.sh DevUnlock` gives an empty 3.5 kB assembly).

15.2 Install and release

- Install to `BepInEx/plugins/<folder>/<dll>` with `meta.json` ; keep the previous DLL as `<dll>.<prevversion>.bak` ; check the md5 after copying (a write can fail while other files in the same copy succeed).
- Release folder per version: the main zip (sorting first), the optional Dev zip, `Source.*.zip` with the bench (sorting last), the notes, `meta.json` .
- **NOMM / NOMNOM:** the registry requires the multiplayer host checks for mods that change a vanilla flight model (§2.14). Describe the mod in plain language for players. Players who use a modpack must update the version there (§6.1).
- If the game is running, writing the DLL fails; close the game or retry the copy.

15.3 Build the native core

```
cd reference/<Jet>
bash native/build.sh linux     # the bench's lib<Lib>.so (bench.sh build / regen.sh does this for you)
bash native/build.sh           # the game's <Lib>.dll -> native/fcs_core/target/x86_64-pc-windows-
gnu/release/
cp native/fcs_core/target/x86_64-pc-windows-gnu/release/<Lib>.dll native/prebuilt/   # then rebuild the
plugin (§15.1)
```

- **Needs:** Rust with the `x86_64-pc-windows-gnu` target and mingw-w64's gcc as the linker (Ubuntu: `apt install gcc-mingw-w64-x86-64`). No crates are downloaded (`--offline`).
- **With rustup** and the Windows target installed: `PLAIN=1 bash native/build.sh` (F-22E; on the other five the script's last line is the build-std form, run the plain `cargo build --release --target x86_64-pc-windows-gnu` with `CARGO_TARGET_X86_64_PC_WINDOWS_GNU_LINKER=x86_64-w64-mingw32-gcc`).
- **Without the prebuilt Windows std** (how the shipped cores were built): Ubuntu's `rustc-1.97` + `rust-1.97-src` , the standard library compiled from source (`RUSTC_BOOTSTRAP=1` , `-Zbuild-std=std,panic_unwind`), `RUST_SRC` pointing at a copy of `library/` with std's `windows-link` dependency put back (Ubuntu's source package prunes it; the script's comment has the line). `RUSTBIN` , `RUST_SRC` and `MINGW_GCC` are environment overrides; the defaults are the author's paths.

- Two builds of the same source differ in a few header bytes (the linker's time stamp): **ship the build you tested**, and bump the ABI number on both sides for any change to a shared struct.
- **Kit verification note:** for kit 3.0 the Linux `.so` was rebuilt for all six (rustup cargo 1.97.0) and every bench ran native, but the Windows core could **not** be rebuilt in the kit's environment (no Windows target for that toolchain; the apt Rust was 1.91, whose build-std failed). The kit therefore relies on the prebuilt cores, which are exactly the ones inside the release DLLs (the plugin md5s match, §17).

16. Telemetry (flight logs)

`<Jet>_FCS_<date>_<time>.csv` in the BepInEx folder, one row per physics tick while `Telemetry` is on. The Revoker's has ~260 columns plus 4 per surface (`<name>_cmd` , `_now` , `_aoaNow` , `_aoaCmd`). The groups:

Group	Columns (examples)
state	`t, mode, ias, tas, rho, alpha, beta, nz, tw, p, q, r, phi, theta, omegaV, massKg`
pilot	`sp, sr, sy` (shaped stick), `qCmd, qT, pCmd, rCmd, alphaCmd, nCmd`
authority	`aUp, aDn, aRoll, aYaw, pMax, rMax, gAvail, yawSat, yawHealth`
envelope / damage	`prot, unload, authCap, asymCap*, asymU, engOut, vmcLim, fuelTrimCm, rudderAuth`
allocator	`axWr, axWp, axWy` (axis weights), `pitchPrio, fightK, fightGate, tvcAnchor, tvcBudget, latReserve`
INDI / anchoring	`nuR, nuY` (rate-loop demand), `anchR, anchY` (table error at the live point), `unR, unY` (unmet), `gUnmetX/Z`
governor	`pullTrade, rateCap, paceLead, paceHold, paceCapZP/ZN/XP/XN, capSwp, capShd, aYawShed, ppOn, ppQ, ppReach` , estimator `ce*`
air data reference	`alphaRoot, betaRoot, airRefCg, alphaCg, betaCg`
effectors	`tvcCmd, tvcNow, tvcdCmd, tvcdNow, tvcyCmd, tvcyNow, splitL, splitR` , Su-30 `nozL, nozR` , per surface `_cmd/_now/_aoaNow/_aoaCmd`
push-over (F-22E 3.1.4+)	`ppFloor` (the floor in force)
timers (3.0.4–3.1.3, all jets since 9 Nov)	`fcsUs, estWaitUs, estThr, tRebUs, tAllocUs, tKickUs, tRowUs, tLawUs, tAsymUs, tAsymMomUs, tSlipUs, tSweepUs, tEnvUs, tMassUs, tFlapUs, tGndUs, tVmcUs` (µs; after the fixed block, before the per-surface columns)
Su-30 extras	`mpoAxisW, mpoAxisDeg, cantVvW, aYawTvc, yawThrustShare, rudShare, rateBlendEff, surfFirst, canPeakLo/Hi, gcReach, gyroQLim, capTauDn/Up, capBrkDn/Up, landFlap, leafBoost`
Ternion extras	`spinRec, matchF, fightHintOff`
tick segments (F-22E 4.1.3, then all)	`sgJoin, sgMass, sgAir, sgFlap, sgSurf, sgTab, sgPre, sgAsym, sgMode, sgGnd, sgLaw, sgApply, sgRow` (µs, sum to the tick), `heapB` (bytes allocated), `gcRan` , `swMask` (9 digits: Telemetry, NativeAllocator, NativeSweep, NativeTables, SweepGridCache, ParallelTables, ParallelSweep, ParallelAllocator, EstimatorThreaded)
coupled check (all six)	`cplU` (load share of the roll range in force), `cplRp` / `cplRn` (flight-path roll left each way), `cplDn` (held-pitch nose-down), `pitchMarg` (its margin); Su-30 `cplRef` , `stopRateP/N` (the stop-time roll limits), `softPitch`
ground brakes (all six)	`gsBrkL/R` (brake written to each main), `gsBsL/R` (the stock term), `gsLn` (light-nose hold)
flying backwards (F-22E)	`revSlipHold` (the sideslip hold's share in a slow tail slide)
archived	`psAX..psNr` , `cePassXP..cePassSN` (the retired passive-step probe) write 0 so columns don't shift

With the stick centred and nothing happening, `nuR` / `nuY` should sit near 0; a standing `unR` means the allocator is leaving roll unmade (look for a cost holding a trim).

17. Verification status

Checked when this kit was assembled (12 Nov 2026):

Check	F-22E 4.3.0	Ifrit 1.3.0	Revoker 1.4.1	Compass 0.5.0	Ternion 0.4.0	Su-30 0.6.0
Plugin rebuilt from this folder with `build.sh`, md5 equal to the shipped release DLL (native core embedded)	✅ `56cc5c3f…`	✅ `ba48d6a6…`	✅ `b03819c2…`	✅ `794d7859…`	✅ `1e397de2…`	✅ `16010c75…`
Bench compiles, with the Linux native core (`lib<Lib>.so`)	✅	✅	✅	✅	✅	✅
Bench flies a scripted case from this folder, native paths used (`FCS_NATIVE_STATS`: 0 fallbacks)	✅ with the private airframe file (§5)	✅ private file	✅ private file	✅ private file	✅ private file	✅ private file
Cloned by `tools/new_airframe.py` under a new identity → game DLL (new core name embedded) and bench (new `.so`) both build	✅	✅	✅	✅	✅	✅
Clone flies the same bench case as the reference (all columns but wall-clock timers identical)	✅ summary line	—	—	✅ 602 / 602 rows	—	✅ 602 / 602 rows
Windows native core rebuilt from source in the kit's environment	❌ (§15.3: prebuilt core used)	❌	❌	❌	❌	❌
Native core vs C# twin, both-ways check (from the release notes)	✅ 427,582 solves, 0 diffs	✅	✅	✅ 467,226 LP calls	✅	✅ 526,376 LP calls
Plugin loaded on the game's own Mono under Wine (release notes)	✅ 4.2.0 (4.3.0: plain C# change, not re-run)	✅ 1.2.0 (1.3.0 not re-run)	✅ 1.3.0	✅ 0.4.1	✅ 0.3.1	✅ 0.5.6
In-game behaviour of this release flown and passed by the author (Tom, 12 Nov)	✅	✅	✅	✅	✅	✅

What that means for you: the source here *is* exactly what the six releases run, and their in-game behaviour has been flown and passed by the author. **The bench is an approximation and may not be accurate:** where a bench number and the game disagree, the game is the reference. The bench is good for direction and A-vs-B regressions, never for absolute numbers (§10.7). The Compass bench has no flight-log calibration; the Ternion's has one log (§10.8).

18. Glossary

Term	Plain English
AoA (α)	angle of attack: the angle between the nose and the oncoming air, in the vertical plane
Sideslip (β)	the same angle sideways: how much the air comes at the jet from the side
IAS / TAS	indicated airspeed (what the wings "feel", scales with air density) / true airspeed
q̄ (dynamic pressure)	½ρV², how hard the air pushes; surface forces scale with it
nz / g	load factor: lift as a multiple of the jet's weight
Effector	anything the FCS can move to make a moment: a control surface, a nozzle, a split brake
Moment	a twisting force about an axis (roll, pitch, yaw)
Allocator	the optimiser that decides how far to move each effector to get the requested moment
INDI	incremental nonlinear dynamic inversion: measure the jet's current angular acceleration and ask the effectors only for the difference needed, so unmodelled airframe effects cancel out
Plate	NO's aerodynamic element: a flat surface with an airfoil table, area and centre of lift, one per part
Local AoA	the angle of attack one surface actually sees (from its own velocity), not the body's
Stall / reversal	past its lift peak a surface makes less force per degree (stall); past ~120° the air arrives over the trailing edge (reversed flow)
Velocity-vector roll	rolling around the direction of flight rather than the nose; at high AoA this is mostly a yaw
Stability axis	an axis tilted by the AoA so it points along the flight path
Coupled axis	above ~36° AoA, stick roll and pedal yaw become one lateral axis
Rate command	the stick sets how fast to rotate; let go and the rotation stops
G command	the stick sets the load factor
MPO	manual pitch override (Stability Assist off): plain rate command, no AoA limit
Envelope / protection	the limits (AoA, g, rates) and the logic that keeps the jet inside them
Capture	braking the approach to a limit so it is not overshot
Capacity	how much angular acceleration an axis can still produce toward a side right now
Governor	logic that slows a command when an axis can't keep up
Trade	in a rolling pull, splitting a yaw shortfall between less roll and less pitch
Push-over	pushing the nose down from high AoA toward zero or negative
Push plan	the push-over's pitch rate planned once at the start from the yaw the jet can shed
rudW	how much of the yaw can be added without spending roll
TVC	thrust vectoring: steering the engine nozzle to make pitch/roll/yaw moment
Stabilator / canard / ruddervator / elevon / flaperon / deceleron	all-moving tail / foreplane ahead of the wing / a canted tail doing pitch and yaw / a delta's combined elevator-aileron / a combined flap-aileron / a split surface used as a drag brake for yaw
Camber coupling	a deflected flap also changes the lift of the fixed wing ahead of it (real physics NO leaves out; the plugin adds it)
Staggered stall	giving surfaces different lift-peak angles so they don't all stall at once
VMC	minimum control speed with an engine out: below it the rudders can't hold the thrust asymmetry

Term	Plain English
Crossed set	surfaces deflected against each other, cancelling, while the jet looks trimmed
Bench / plant	the offline simulator / the simulated aircraft inside it
Equation error	comparing the bench's predicted accelerations with logged ones at the same logged state
Byte-identical	the bench output files are exactly the same, bit for bit
Static margin / neutral point	how far the point where extra lift acts (neutral point) sits behind the CG; positive = the jet points into the wind by itself, zero = "relaxed to neutral"
Inertial (gyroscopic) coupling	the pitch (or roll) a rotation about two other axes makes on its own, e.g. $\dot{q} \approx (Izz - Ixx)/Iyy \cdot p \cdot r$ in a velocity roll at AoA
Coupling roll limit	the cap on roll rate at high AoA set by how much nose-down is left to absorb that coupling
Fore / aft lift schedule	AoA-dependent multipliers on the body lift ahead of and behind the CG, fitted so lift is unchanged and the pitch moment is zero (F-22E X2 / X2e)
Downwash	the wing's wake turning the air down at the tail, so the tail sees less AoA than the body
Polhamus vortex lift	the extra lift a sharp leading-edge extension (LERX, strake) makes from its vortex, growing with $\sin^2\alpha$
Push-over floor	the slowest nose-down rate a planned push-over is allowed (F-22E 23 °/s)
Curve-quality checks	bench measures of a response's shape: onset, dips, plateaus, secondary peaks, command retreat
Ensemble	the same bench case run several times with tiny changes (±0.1 %), so chaotic outcomes aren't mistaken for real differences
Worker / helper thread	another CPU core doing part of the tick's maths; the main thread is the game's own
Canted single-axis nozzle	a nozzle that swivels about one axis tilted outboard, so one movement makes both pitch and lateral moment
Native core	the Rust library (`<Jet>_FcsCore`) that runs the allocator's solve, the surface-table kernels and the coupled check's LP; carried inside the plugin DLL
ABI number	a version stamp both sides of the C# ↔ Rust call check at load, so a plugin never calls a core with a different data layout
C# twin / fallback	the original C# code kept beside each native routine: the reference for tests and what runs if the core is missing or fails
`-O0=-float32` (float mode)	the Mono option Unity uses: C# `float` maths is done in double, so the game's C# and the bench's differ in the last bits
LP (linear programme)	an optimisation with a linear goal and linear limits; here: the best effector mix for roll each way with the load and pitch held
Coupled check	the asymmetric-load AoA limit from that LP: where the weaker roll side is down to the allowed share (§8.23)
FindTop	the edge rule that takes the lower edge of the over-threshold stretch reaching the top of the AoA range
Mirrored nose axis	flying backwards, roll share
Differential brake	braking one main wheel more than the other to yaw the jet on the ground
Blueprinter	the tool other modders use to build aircraft for NO; its jets can name and parent parts differently from stock

19. Problem and solution log

Everything found and done since the first kit: §19.1–19.7 cover kit 1 (5 Nov 2026: F-22E 3.0.1, Ifrit 0.16.0, Revoker 0.8.2) to kit 2.0 (10 Nov); §19.8–19.14 cover kit 2.0 to this edition (12 Nov). One entry per problem: what was seen, what caused it, what was done, and how far it is verified. "Bench" means the offline simulator only; "flown" means Tom flew it in game. Quotes are Tom's. The full notes for each version are in the jet's `release-notes/` or `CHANGELOG.md`; findings documents are named where they exist.

How to read the status words: **compiled** (builds against the game), **bench** (tested offline), **flown** (tested in game), **open** (known and not fixed).

19.1 F-22E Strike Raptor (3.0.1 → 3.2.1)

3.0.2 · No fins: roll twitch and sideslip departure in straight flight, and roll creep (5 Nov)

- *Problem:* with both fins and the right stab shot off, stick centred, straight and accelerating: the yaw command flipped +1.4 ↔ −2.0 °/s every 0.1 s, the yaw-making flaperons swung ±15–25°, roll ±7 °/s at 1.5 Hz; then the yaw command held at the measured yaw rate minus 2 °/s and sideslip went 3 → 48°. A second log crept in bank −17 → −37° at 4° AoA.
- *Cause:* the 2.20.0 no-fin stop asked for "the yaw you have, minus 2" whenever it thought a roll was being stopped. Its "which side is the roll" fell back to `Mathf.Sign(0)`, which is **+1** in Unity, so with no roll at all it saw a right roll every tick, and it ignored the near-zero-AoA rule. The roll creep came from the 2.18.0 release coordination (`r / sin AoA` with sin AoA floored at sin 15°) reading a banked turn's own yaw rate as a rotation at 4° AoA and feeding itself.
- *Solution:* `PaceStopNoFinGate` (only for a roll really being stopped, only where the window is on); `ReleaseCoordGate` (only above ~15° AoA, only with more than 5 °/s of rotation after the turn's yaw is taken out).
- *Result:* straight no-fin flight peak sideslip 53° → 0°; bank drift after a bank-and-release −11…+98° → 0…+10° over 18 s; clean-jet suite identical. *Status:* bench. A departure at the end of the second log (gear down, 7.5 g, full reversal with the left aileron and right stab gone) was a real damage limit, not changed.

3.0.3 · The asymmetric-load cap came down too slowly after a part was shot off (5 Nov)

- *Problem:* right stab shot off, full aft 2 s later: AoA 25° at 3.4 g, 78° of bank against full opposite aileron, sideslip −19°.
- *Cause:* the cap's target dropped 65 → 12.6° at once, but the cap followed at its fixed 10 °/s tighten rate.
- *Solution:* `AsymCapGapTighten`: tighten rate × gap / 3°, 1–4× (also in Compass 0.1.9).
- *Result:* AoA peak 26–44° → 22–27°; no-fin + right-stab sideslip 26.1° → 1.5°. *Status:* bench. Not changed: no fins at negative AoA has only ~13 °/s² of yaw either way (a damage limit).

3.0.4 → 3.1.3 · CPU cost per tick (5–6 Nov)

- *Problem:* ~1180 µs per tick in the air in game; Tom: "can u start working on making the fcs mods multi-threaded".
- *Cause, measured step by step with new timers:* the estimator's six probe solves (~200–250 µs), the table rebuild (~100 µs), the asymmetric-load AoA sweep (~468 µs per tick in game, the largest single item, 3 points × ~250 µs on 62 % of ticks), the envelope refresh (~355 µs on each refresh tick, reading `rb.velocity` 25 × parts times), the CSV formatting.
- *Solution:* §2.16, one piece per version: estimator on a worker (3.0.4), table rebuild and reach plan on several cores and the shared pool rules (3.1.0), a "rest" split of the timer (3.1.1), the sweep gathered once and run in parallel (3.1.2), the envelope gathered once and run in parallel plus a cached camber base and force-hook timers (3.1.3). The telemetry-only `PassiveStep` probe was archived (3.1.0; its CSV columns write 0 so log columns don't shift).

- *Result:* main-thread time per tick on the bench 0.72 → 0.43–0.50 ms; every version byte-identical to 3.0.3 (suite 153/153 files, 52/52 replays, the game CSV 2316/2316 rows). *Status:* bench; the in-game timer has the final word.

3.1.0 · "Your FCS runs even after I eject" (relayed from Discord)

- *Cause:* after ejection `aircraft.Player` is null, so the hook treated the empty jet as an AI jet and kept flying it.
- *Solution:* once `aircraft.HasEjected()`, disengage and hand back to the stock FBW. Ported to every jet on 9 Nov. *Status:* game hook, not bench-testable; not flown.

6 Nov · Low-speed surfaces look choppy, roll input feels delayed (findings, nothing changed)

- *Answer:* real, three separate causes, none new. (1) The game draws each control surface once per physics tick (~59 Hz) with no smoothing while the airframe is interpolated every frame, and at low speed the stabs slew flat out 15–21 % of the time. Not an FCS cost (main-thread time is *lower* at low speed). (2) The roll command is ramped at the available roll acceleration (25–35 °/s² at 65° AoA), a 1.6.2 design that keeps the allocator from asking for more than the surfaces can deliver. (3) The stab command path (10 ms pre-filter + 0.06 s acceleration limit) plus 65° of travel at 104 °/s from a stop.
- *Proposal, not built:* schedule `ActuatorDamping` by dynamic pressure (0.06 s at high q, ~0.02 s below ~100 m/s). (`claude/F22E_low_speed_surface_lag_findings.md`)

3.1.4 · "At the end, the push-over pitch didn't respond for a moment" (6 Nov 22:22 log)

- *Problem:* a released-roll push from 64° at 92 kt held a 9.1 °/s nose rate for 1.3 s with full forward stick (peak sideslip only 1.1° against a 4° budget). The slowest of 84 planned pushes.
- *Cause:* the push plan sized itself from its IAS yaw-shed fit: 0.028 · 47.3^1.5 = 9.1 °/s², T0 2.34 s, planned rate 9.1 °/s; and the lateral effectors were in transit (a stab sent to 80° local AoA on the roll release, then yanked across its whole travel by an instant stall window mid-push). (`claude/F22E_pushover_hold_findings_2026-11-06.md`)
- *Solution:* "set a minimum push over rate so I could push the nose down... I don't care if sideslip is 12–15 degs": `PushOverMinRate` 20 °/s floor, faded out by predicted sideslip from 12 to 15°. *Result:* the 22:22 push 1.98 → 1.51 s to 20° AoA, sideslip 4.3 → 7.7°; 68 of 86 replayed pushes byte-identical. *Status:* bench.

3.1.5 · "The last push-over was kinda rough... it went as slow as around 16 °/s" (23:24 log)

- *Cause:* at 27.6 s the predicted sideslip passed 12° and the 3.1.4 guard faded the floor to zero: slow-then-fast, which is the roughness. The 20° of sideslip in that push was there with the floor off too (the rudders held the corkscrew yaw).
- *Solution:* "no guard, just a plan floor of 23 °/s": guard removed, floor 23 (forced once, `AirframeRevision` 25). *Status:* bench.

3.1.6 · "It took a long moment for the pitch to kick in... then the TVC looked very hesitant... smooth at first then it spikes"

- *Cause:* the push governor's sweep cap (not the plan) was gated by a hard 5 °/s velocity-roll switch, so a trivial rotation hovering around 5 °/s switched it on and off mid-push; at 63–78 kt its yaw-shed read was 1–4 °/s², so the cap sat near its 6 °/s floor; and its `|α|` term collapsed to 6 near zero AoA, stopping the nose short on pushes and on pulls from negative AoA (a pull from −41° got command 0 for 0.8 s). (`claude/F22E_sweepcap_findings_2026-11-06.md`)
- *Solution:* the cap fades in from 5 to 15 °/s of velocity roll (`PushGovFade`), and never holds the nose's *body* rate under 23 °/s scaled by the push (`SweepFloorPushMin`; an AoA-rate floor left the nose at −2 °/s because the flight path itself was falling at ~20 °/s); neither with fins damaged. Tom also asked for the curve to be checked on every manoeuvre from then on: the curve-quality checks and the low set were written (§10.3).
- *Result:* low set dips > 3 °/s 11 → 2, worst dip 25.3 → 10.7 °/s; two fixes found by the new checks during the work (no-fin push-overs departed until the fin exclusion was added; an unscaled floor loosened a released

corkscrew recovery). *Status:* bench. Not fixed then: roll onset 0.38–0.50 s at 65° and very low speed; roll stops with rebounds.

3.1.7 · "At the beginning of every push-over the TVC would hesitate or chatter… towards the end… only a few °/s" (00:23 log)

- *Cause:* six hand-offs between the push plan and the sweep caps: the plan ending at 10° handed over to a 6 °/s floor; caps read for the wrong AoA side for one tick at zero; the pace closing on a zero target; the far-side grow cap switching in whole; the governor taking back rate the push already had (the "spike or dip" and the "frozen" part); the plan's bottom ramp cut at the crossing.
- *Solution:* "if it is real excess yaw, then just go… some more sideslip is ok": seven switches (§2.10): `PushPlanDoneOpen`, `SweepSideKept`, `PushGovPaceYaw`, `PushGovFadeGrow`, `PushNoRetreat`, `SweepZeroBlend`, `PushPlanThrough`.
- *Result:* 87 planned replays: nozzle reversals 280 → 235, command spikes 14 → 4, sideslip never worse by more than 1°. Two side effects found and fixed in the work (`PushNoRetreat` on the far side reached 26°; `PushPlanThrough` letting the plan own pitch past zero). *Status:* bench.

3.1.8 · "The best push-over performance so far, but still a little bumpy, especially the first few" (01:23 log)

- *Cause:* the plan jumped to its rate, then the yaw shed faster than the IAS model expected and the plan crept up at `PushPlanRise` 15 °/s² *after* the onset; the nozzles braked onto the plan, then pushed again.
- *Solution:* `PushPlanRise` 15 → 40 °/s², so the rise happens inside the onset. *Tried and left off:* `PushPlanRamp` (one constant-acceleration ramp: sideslip worst 24.4 → 35.8°) and `PitchTrajLead` (more nozzle reversals). *Status:* bench. Open: the jet runs ~3 °/s past a held planned rate; the stab command steps between grid points while rolling.

3.1.9 · "Taking off cost me 1 % fuel, now it is 4 % for the Revoker"

- *Cause:* found with a diagnostic plugin (`FuelAudit`): the Aryx Weaponry Expansion's own `Aircraft.UseFuel` prefix (external tanks first, then skip the original) and the FCS's fuel prefix ran at the same priority, and HarmonyX runs every prefix even after one returns false, so both drew. Burn 1.65× stock on the stock jets (1.75× Ternion), 1.5× on the F-22E. (`claude/Fuel_overburn_findings_2026-11-07.md`)
- *Solution:* two prefixes (§2.17), on every jet (F-22E 3.1.9, Ifrit 0.17.1, Revoker 0.8.3, Compass 0.1.12, Ternion 0.1.7).
- *Result:* tested on the game's HarmonyX 2.9 in every load order: F-22E 0.75×, Revoker 0.65×, others 1.0×. Wrong leads retracted on the way (afterburner use / a detents mod; a voice-warning mod with a read-only postfix). Also found: the mod manager had redeployed F-22E 3.0.3 over 3.1.x (§6.1). *Status:* compiled and hook-tested; not flown.

9 Nov · Airframe studies (bench only)

- Static margin of the current build (NP 0.12–0.21 m behind the CG at low AoA, about neutral at 20–28°); the roll's pitch-up is inertial coupling, not aero; a loaded roll at 36° compared with real-jet footage; concave post-peak stab curves (no effect on the loaded roll as flown); a LERX-vortex / no-downwash / stab-36 / neutral-margin prototype (H3 ran away to 68–69° at idle; H4 cut roll 20–33 %); the trim-free schedule study (cur / N / T / X2). All summarised in §8.15.

3.2.0 · Airframe X2: relaxed to neutral, stab lift peak 36° (9 Nov)

- *Asked:* "the simulated LERX vortex, getting rid of downwash modelling, 36 AoA as max lift curve for stabs, and the relaxed to neutral margin"; goals G1–G3 (§8.15).
- *Solution:* the package as asked (T) failed G3 on the bench, so X2 was built: downwash kept, stab peak 36, fitted fore/aft body-lift schedule. `AirframeRevision` 26 migrates old Dev files. *Result:* G2 and G3 met on the bench.
- *Two regressions reported in the notes before shipping:* low-throttle overshoot to 54–56° past the 45° cap, and lower roll at moderate AoA (−17…−27 % at 11–35° in some cases; each of the two parts costs roll on its

own). *Tried, not shipped:* `StabilatorCLScale` 1.4 (fixed the entries, but the idle 45° hold ran to 60+), X2b (halved the overshoot; not the handed-off target), releasing `protHold` at the limit (fixes only the 26.8° hold case). *Status:* bench; then flown via the Dev cfg.

3.2.1 · Airframe X2e: the 45–55° roll dip (9 Nov)

- *Problem:* "the velocity roll seems a bit slower at the higher end of mid AoA... the roll dip at 45–55 is slower, are there any way to boost it... without generating more sideslip, stopping margin".
- *Cause:* the coupling roll limit pays the roll's inertial pitch-up out of the nose-down headroom; X2's 48° knot had removed most of the airframe's nose-down at 45–55° (−18 against −41 °/s² at 48°).
- *Solution:* knots added at 52 and 56° that give the nose-down back from 44° (X2e), lift unchanged; `AirframeRevision` 27. The limit itself was not loosened (that would spend the stop margin).
- *Result:* full nose-down stab at 48° −18 → −44 °/s²; the X2d flight (same up to 48°) matched 3.1.9's 45–55° roll at matched entry speed with less sideslip. *Status:* bench, X2d flown, X2e flown but no log analysed. **Open:** low-throttle overshoot 50–51°; the 26.8° idle-hold case; the bench-vs-game nose-down table gap; G1 (phase 2, control law).

19.2 KR-67 Ifrit (0.16.0 → 1.0.0)

0.17.0 · "See if the push-overs can be smoother" (7 Nov 09:16 log)

- *Problem:* on 5 of 9 planned pushes the command dropped from −48 to −10…−28 °/s for 0.1–0.3 s near the bottom; one push held back to −25…−15 °/s near zero AoA. Onset was fine (0.07–0.15 s).
- *Cause:* the same five hand-offs fixed on the F-22E in 3.1.7.
- *Solution:* six of the F-22E switches ported and on (`PushPlanDoneOpen`, `SweepSideKept`, `PushGovFade`, `PushGovFadeGrow`, `PushGovPaceYaw`, `PushPlanThrough`). The F-22E's "just go" parts (`PushNoRetreat`, the 23 °/s floor, `SweepZeroBlend`) are ported but **off**: they cost sideslip in the regression ensemble, and the Ifrit's rule is "no more sideslip than 0.15.0".
- *Result:* replays: command retreat 241 → 110 °/s total, spikes 3 → 0, nozzle reversals 24 → 16, time to 10° unchanged; 5-member ensemble worst case 12.1 → 11.6°. *Status:* bench.

0.17.1 · Fuel over-burn with the Aryx Weaponry Expansion — as F-22E 3.1.9. *Status:* compiled, hook-tested.

1.0.0 · The FCS on several cores (9 Nov)

- *Solution:* the F-22E 3.0.4–3.1.3 work ported (§2.16), plus the ejection fix.
- *Problem found in the port:* the first bench test differed (estimator columns, then pitch and roll commands). *Cause:* the Ifrit's allocator has four per-tick settings the F-22E's doesn't (slip-frame weighting on / sin α / cos α, and the settled flag of the fight-cost null-space solve), and the worker copies didn't take them. *Fix:* copied; the reach job also hands back its stage-1 residual. `PassiveStep` ran every tick while logging but nothing read it: removed.
- *Result:* suite 159/159 identical (all on and all off), replays 4/4, main-thread time 1.9 → 0.9–1.1 ms. *Status:* bench.

19.3 FS-12 Revoker (0.8.2 → 1.0.0)

0.8.3 · Fuel over-burn — as F-22E 3.1.9 (this is the jet the 4 % takeoff was seen on: 10.7 kg/s at full afterburner instead of 4.2).

1.0.0 · The FCS on several cores (9 Nov)

- As the Ifrit, plus one Revoker-specific problem: the hinged inboard elevons (`HingeFlapEffector`) read the game's transforms and rigidbodies in their flow evaluation, which can't run off the main thread. *Fix:* those reads happen once per sweep tick on the main thread (`Snap`) and the sweep points use the snapshot (`EvalFlowSnap`, same expressions on the same values).
- *Result:* suite 159/159 identical, main-thread time 2.3 → 0.9–1.0 ms. *Status:* bench.

19.4 T/A-30 Compass (new: 0.1.0 → 0.2.0, 5–9 Nov)

A stock trainer with no TVC, forked from Revoker 0.8.2. Almost every problem was a TVC-era assumption meeting a jet without it.

Version	Problem (log)	Cause	Solution
0.1.0	—	new mod	Revoker laws on the T/A-30: AoA ±30, flaps as allocator effectors, CG 0.27 m aft, camber, tail downwash 0.45
0.1.1	AoA limits and pitch at high AoA wrong; low-speed rolls pinned both elevators nose-down	strakes flew the wing's 17° foil; CG 0.27 too far aft (roll inertia coupling)	strake Polhamus vortex lift (`Strake.cs`), CG 0.20, elevators ±30, AoA +35/−30
0.1.2	strake lift kept rising past the 35° limit; stick could cancel the limit	breakdown too late; `StickCancelsLimit` assumes TVC to recover	breakdown 30–50°; `StickBypass` only with pitch TVC; CG 0.15
0.1.3	both elevators at the nose-down stop in a 100 °/s roll at 70 m/s, AoA 10 → 46° (17:41 log)	slabs on the wing foil; the roll's inertial pitch-up had no limit without TVC	`StabSlabFoil` (stall 30°), `CoupNoTvcAlways` , LERX 3.4
0.1.4	full-stick rolls ~105 °/s at 300 km/h with ailerons at 6–7° (18:12 log)	the F-22E roll reference and steady-roll estimate	roll-rate gains / `RollAuthorityUse` raised: 142 → 247 °/s (1 g bench)
0.1.5	departure after losing the right outer wing in a 90 °/s full-aft roll (18:52)	the roll's pitch-up took the last nose-down	`StabRollGovernor` ; `FlapRollBias`
0.1.6	outer wing gone, push to −8 then full aft through zero at 30 °/s, AoA 12 → 47 (19:04)	capture leaned on authority it didn't have; the stabs went to holding the lost wing's roll	`AsymCapBrake` ; stab roll split priced at the cap (`AsymStabAntiCost` 0.1)
0.1.7	outer wing, fin and stab gone: the cap's model read 25–29° of room with every pitch surface out of nose-down at 12° (19:18)	the cap's model misses nose-up the damaged airframe really has	`AsymPitchBias` : measure it in flight and take it off the margins; cap floor 7°. Bench damaged departures 23 → 1 of 133
0.1.8	departures at 193 and 217–223 s (20:25)	protections gated on the lateral CG offset, which swung −0.28 ↔ 0 m with a hanging rudder	`AsymDamageGate` (gate on a lost surface); negative-AoA mirror; `PushPlanStopsRoll` off ("a held roll is no longer stopped through a push-over"). 27 → 2 of 133
0.1.9	outer wing shot off 0.9 s before full aft: 6 of 8 depart	cap too slow (as F-22E 3.0.3)	`AsymCapGapTighten` : 0 of 8. A 180 s departure was the speed bleeding to 45 m/s at idle, below this damage's ~100 m/s minimum
0.1.10	inner flaps moved with the pitch stick	ground-law pitch throw and free common deflection	`FlapPitchLock` (common held to schedule, differential free for roll; released when a slab is lost)
0.1.11	engine out: only the last-resort VMC cut acted, after the sideslip ran away	fixed nozzles have no vectoring transform, so the FCS skipped their thrust entirely (yaw read 0)	thrust moment taken as it is + `VmcPredict` (throttle back at 70 % of yaw authority): departures → held; sideslip 53/35 → 0.6/1.9°
0.1.12	the Compass FCS wasn't installed at all	the mod manager had removed it	reinstalled; fuel hook fix
0.2.0	CPU	—	threading port (§2.16), byte-identical

19.5 FS-3 Ternion (new: 0.1.0 → 0.2.0, 7–9 Nov)

A Blueprinter jet with canards *and* aft elevators and no TVC, forked from Compass 0.1.11.

Version	Problem	Cause	Solution
0.1.0	—	new mod	two `Stabilator` pairs (`isCanard`), Revoker staggered trailing-edge stall, camber, no downwash, AoA ±40
0.1.1	AoA limits above 45 didn't fly, MPO included	inherited fixed no-TVC caps always on (vectorable T/W is 0)	off on a `noVecJet`
0.1.2	corkscrew entries slow to build pitch	`LateralStabReserve` 0.7 reserved 70 % of pitch authority for roll (a TVC-era rule)	0; elevators stall at 30° ×1.2; canards ×1.1; LERX ×1.13 to keep the −0.29 m margin; rollout brake with elevators up
0.1.3	elevators', rudders' and flaperons' travel taken in flight; nose-down dip when a corkscrew is let go	the stock split airbrake and split yaw were still active	both off while the FCS flies; F-22E trailing-edge brake set; `HighAoAReleaseTau` 0.3 s; AoA 50
0.1.4	elevators' normal force rose past the stall	0.1.2–0.1.3 moved the drag rise in with the peak	keep the airfoil's own drag (`ElevatorStallDragShift` false)
0.1.5	limit raised on request	—	AoA 55 (holds to ~165–180 km/h)
0.1.6	canards and elevators left in opposing roll splits after a manoeuvre	a cancelling split nearly free with two pitch pairs	`StabilatorRollCost` 0.0005 → 0.01
0.1.7	1.75× fuel burn with the weapon pack	as F-22E 3.1.9	two-prefix fuel hook
0.1.8	MPO full aft stopped at 55–58° (14 °/s at 215 km/h)	MPO's pitch budget had a 40 × T/W term, 0 without TVC	`MpoNoVecFullRate`: 47–48 °/s through every AoA
0.1.9	a roll/yaw kick every 2–3 s with a roll surface shot off; engine-out above the match AoA; no recovery from spins	the roll-fight cost used a mirror partner that was gone; no zero-thrust match; nothing for spins	`FightHintUnpaired` (the groups' own net roll); `EngineOutZeroThrust`; `SpinRecovery`
0.1.10	**a destroyed engine took that side's elevator, fin and rudder with it**; full pulls with a wing's surfaces shot off departed	the blueprint makes `wing_1c` a transform child of the engine; the asymmetric cap had no measured authority	`StructureFix` + `UnitPart.OnParentDetached` prefix; `LiveAuthCap`
0.1.11	the engine-out match switched on/off at 40–50°	shared with the AoA cap key, no hysteresis	`EngineOutMatchAoA` 35, latched on yaw saturation; `RollPaceRateBlend` 0.45
0.1.12	releasing an engine-out rolling pull at 50–55° triggered spin recovery and shoved the nose to −8°; hands-off engine-out jet crept 1–2 °/s	recovery gated on AoA and yaw rate only; steady engine-out sideslip fought by the hold	recovery only with the yaw saturated, as a rate target scheduled on AoA; `EngineOutSlipTrim`: drift −6.0/−3.3/−3.0 → −2.4/−1.2/−0.8° in 20 s
0.2.0	CPU	—	threading port (§2.16), byte-identical

19.6 Su-30VXN Gandiva (new: 0.1.0 → 0.2.3, 7–9 Nov)

kyleisred's Blueprinter Su-30 with canted single-axis nozzles, forked from Ternion 0.1.8. Most of its history is about the airframe and the bench rather than the laws.

Version	Problem	Cause	Solution
0.1.0	—	new mod	`CantedTvc`, the F-22E stab curve and downwash, canards pitch-only, `Names.Canon`, AoA +70/−45, MPO about the nozzle axis past the limit

Version	Problem	Cause	Solution
0.1.1	high-AoA roll/yaw slow, nozzles only doing pitch; tail drops onto the runway on the take-off roll	the sideslip probe left the nozzles out (margin cut 6 → 2.8°); velocity-roll weight too low; thrust line 0.5 m below the CG	`SlipAuthTvc` ; `HighAoAVvRollWeight` ×4; a nose-wheel load hold (later replaced)
0.1.2	velocity roll flat at 23–26 °/s from 110 to 400 km/h	the roll governor's rate cap counted the rudders in full (Revoker's 75–85° fade) while the nozzles carried the yaw	cap scaled by the rudders' share (`HighAoARollRateCapFloor`)
0.1.3	jet went past the limit to ~114°	the sideslip-margin model faded out at a fixed 70°, below the 75° limit	fade from the limit instead; LERX 1.15, canards 1.2
0.1.4	rolling-pull hitch at mid AoA; take-off nose bouncy; flaperons not rolling first	trade read the yaw capacity through 0.05 s while the nozzles take 0.46 s to cross	`TradeCapTau` 0.45; thrust feed-forward (later replaced); `FlaperonRollPriority` ; `FallingLeafYawBoost`
0.1.5	pitch-up on taxi/take-off; loses speed "comically fast" with AoA	thrust line 0.49 m below the CG; drag 1.5–2× the Ifrit/Revoker from 30° up	`ThrustLineAtCg` ; `HighAoADragShape` (solved with pitch moment unchanged)
0.1.6	high-AoA yaw worse than the Ifrit	the yaw floor's IAS-decay anticipation also faded the nozzles' thrust-borne share	`YawThrustShare`
0.1.7	asked: pitch-neutral airframe, 75°	—	AoA 75 adopted; the neutral airframe benched and **not** adopted (overshoots to 83–92°, twice the low-speed release sideslip)
0.1.8	TVC spends deflection on pitch; corkscrew stops at ~55° then restarts	the mod's own canard AoA schedule had never been switched off (script outside the `Aircraft` hierarchy)	scene search for the scripts; `SurfacesFirstPitch` ; canard peak window
0.1.9	pitch wall at 50–65°; rough falling leaf at 75	`HighAoACanardPitchFirst` acted as a relay at ~1.3 Hz	relay off; `PitchWallSurfacePeak` ; `GCaptureReachable` . Bench claims later withdrawn (below)
(bench)	bench disagreed with the game at high AoA	`eqerr` poser matched no surface; no camber hosts; flap centre-of-lift offset	poser fixed, recalibrated (§9.2.4); the wall shown to be mainly the airframe (§8.19)
0.1.10	—	Tom's dev values after flying	CG 0.25 aft as default; the nose-wheel hold deleted
0.1.11	AoA past the limit; inverted corkscrews stop with no airspeed then tumble	gyroscopic roll at low speed; no mode for "too slow for sideslip"	`GyroBudget` ; `LowSpeedTvcRoll`
0.1.12–0.1.14	stability margin vs taxi	—	aft body lift ×0.5, LERX 1.5, CG 0.30 → 0.25 with canards 1.3; Buffed thrust ×1.12 / fuel ×0.65
0.1.15	"stop on the AoA limit right on time without being too slow or too fast"	modelled braking blend with hand factors	`CaptureMeasured`
0.1.16	TVC roll should engage only when the airframe is "like a brick" in sideslip	IAS-gated band	gate on the probe's measured `slipStiff`

Version	Problem	Cause	Solution
0.2.0	corkscrew stop waiting on sideslip; rough push-overs; hitch in rolling pulls above ~300 kt; Su-30VXN 1.0.1 moved the CG	—	`StopIgnoresSlipOutsideMargin`, `GCaptureMeasured`, `SoftAoALimit`, `PitchTrajMeasuredEnd`, threading port, `Su30CgReference`; AoA +70/−65
0.2.1–0.2.2	tips onto its tail taxiing (nose to 11° in a slow full-pedal turn); fuel burn moves the CG	CG too far aft for the gear; stock CG walks ~30 cm aft from full to empty	CG 0.15 → 0.10 with canards 1.5 → 1.6 (margin kept within ~7 %); `FuelCgHold`
0.2.3	on the ground the nozzles only followed the pitch stick	—	`GroundTvcRollYaw`

19.7 Across all jets

- **Mod manager rollbacks:** a NOSMR/NOMM modpack redeployed F-22E 3.0.3 over 3.1.x and removed the Compass FCS. Check the version line in `LogOutput.log` before reading any log (§6.1, §12.4).
- **Game crashes on 31 Oct–6 Nov were not the FCS:** every one was a Direct3D 12 device hang (DXGI `887a0006`) on an NVIDIA driver with an experimental OptiScaler build injected as `dxgi.dll` and `-force-d3d12`; one happened before any FCS jet had spawned. (`claude/Game_crash_investigation_2026-11-06.md`)
- **BFM shootout (9 Nov, bench):** the six jets compared for guns and high-off-boresight fights, rate ↔ slow transitions and supersonic performance. The Ifrit is the best all-rounder; the Revoker the stock-profile post-stall specialist; the Su-30 "gets stuck slow" (can't accelerate while holding 12 °/s from 135 kt); the F-22E the supercruise and high-altitude jet. The run also found the stock-bench bugs listed in §10.7. (`claude/BFM_shootout_2026-11-09.md`)
- **Replay recording of the Su-30's leading-edge flaps** chattered because two scripts write the same transform in `LateUpdate` in an undefined order; the separate replay tool now samples in `Application.onBeforeRender` (§8.21).
- **Version numbers:** Ifrit and Revoker went to 1.0.0 and Compass / Ternion / Su-30 to 0.2 at Tom's request with the threading port; the F-22E went 3.1 → 3.2 for the airframe change (a minor bump for an airframe change, as for a law change).

19.8 Kit 2.0 → kit 3.0: F-22E Strike Raptor (3.2.1 → 4.3.0, 10–12 Nov)

4.0.0 · "Rewrite in Rust, keep the exact same flight behaviour" (10 Nov)

- *Problem:* Tom asked for the 3.2.1 FCS in Rust where possible, flying exactly the same.
- *Finding:* Unity starts the game's Mono with `-O=-float32`: every C# `float` expression is worked out in double and rounded only when stored, passed or returned (found in `UnityPlayer.dll`, confirmed on the game's own `mono-2.0-bdwgc.dll`: `(16777216f + 1f) - 16777216f` gives 1). So no compiled code can match the C# bit for bit **in the game**, and the C# in the game does not match the bench either. Over 26,700 recorded solves, Mono vs Rust: 7.1 % identical, 68.4 % within 0.001°, 24.2 % equal-cost alternatives, 0.3 % different.
- *Solution:* the allocator in Rust, bit-identical to the C# **on the bench** (where both round to float), with C# twins, self-tests and fallback (§2.18). Off in the game by default (`NativeAllocator` 1). About 8× faster per solve.
- *Status:* bench identical; run on the game's Mono under Wine.

4.0.1 · The sweep at half its cost; a Configuration Manager error (10 Nov)

- *Problem:* the asymmetric-load sweep ran on 61 % of ticks, ~300 µs on a sweep tick (31 % of the tick). Separately, the new integer settings drew a free text box, and emptying it while typing logged `Failed to draw setting NativeAllocator - System.FormatException`.

- *Solution:* each deflection's geometry worked out once per sweep tick, not once per AoA point (`SweepGridCache`: 211 → 98 µs on the bench, same numbers); `AcceptableValueList<int>(0, 1, 2)` turns the box into a dropdown. Skipping the sweep while symmetric, or not waiting for it, were both ruled out: the sweep's answer is needed every tick it changes.

4.1.0 · Rust on by default, plus the kernels (10 Nov)

- `NativeAllocator` 2 / `NativeAllocatorCheck` 0 by default (forced once, `AirframeRevision` 28); the sweep's inner loop and the surface tables' per-deflection maths moved to Rust kernels (4.6–4.7× on Mono). Bench byte-identical to 3.2.1.

4.1.1 / 4.1.2 · A loaded roll reversal stopped, then spiked (10 Nov)

- §8.25. *Status:* 4.1.2 flown 22:50; still a roll-acceleration limit for the yaw's sake (open).

4.1.3 · Where does the tick go in the game? (10 Nov)

- *Solution:* the tick timer splits the whole FCS tick into 13 segments, reports heap growth and garbage collections, logs slow ticks with their breakdown and names the switches in force (§16).
- *Result (flown 23:48, switches off one by one):* everything on 470 µs per tick (median 444, p95 640) against 1,862 µs all C# on one thread; threads saved ~240 µs; the CSV row was most of the garbage (17 KB → 1 KB per tick) and every 3 ms hitch, so telemetry is off by default and the setting now takes effect in flight.

4.1.4 · One DLL, and a first-tick fallback (11 Nov)

- *Problem:* two DLLs to install (plugin + core); and on the first tick the tables fell back to C# while the core's self-test ran.
- *Solution:* the core embedded as a resource in the plugin, written to `BepInEx\cache\<Lib>\<sha16>\` and loaded by full path (a corrupted cached file is detected and replaced; outside a BepInEx folder the temp folder is used); the self-test runs before the first tick's tables. All five FCS plugins loaded in one process, each bound its own core.

4.1.5 · Ground braking (11 Nov) — §8.26. This jet had **no** stock differential braking (`differentialBrakeFactor` −1 on both mains: left pedal braked both, right pedal neither). Bench: release stops 15–20 % shorter (10 kt 1.38 → 1.12 s); 3 s of full pedal from 15 kt: peak sideslip 2.6 → 9.3°, speed 19.9 → 9.5 kt.

4.2.0 · The coupled check ported, shipped off (12 Nov) — §8.23 ("why it is off"). With the defaults, identical to 4.1.5 (231 / 231 + 22 / 22 files).

4.3.0 · Flying backwards, the nose went against the stick (12 Nov) — §8.24.

19.9 KR-67 Ifrit (1.0.0 → 1.3.0)

- **1.0.1** loaded reversal port (§8.25). **1.1.0** performance port (Rust allocator, sweep cache, kernels, tick segments). **1.1.1** one DLL.
- **1.1.2 · MPO roll held back by sideslip (11 Nov, 12:35 log).** In MPO with `SeparateRollYaw` the body roll was limited by the sideslip protection, which a pilot who chose separate axes does not want. `MpoFreeRoll` (player, on): the roll stick's body roll is limited only by the roll axis (surfaces + roll nozzles). Bench replay only.
- **1.1.3** ground braking (§8.26): release stops 14–20 % shorter (10 kt 1.37 → 1.10 s); pedal drift peak sideslip 6.3 → 10.1°.
- **1.2.0 · Coupled check (12 Nov).** Share 0.75. Two Ifrit findings went to every jet: the demand curve has a hump near 10° at very low speed (8 cm, 37 m/s: 1.11 at 10°, 0.25 at 65°), so the first-crossing search pushed the nose down through it (`FindTop` fixes it); and the present nozzle angle (the controller's own output) fed back into the load at idle and flipped the cap 10 ↔ 34° every 1.5 s (`CplNoTvNeutral`). `AsymPitchNoGainSkip`: no pitch cap when the held-pitch margin is short already at the lowest AoA. Open: 12 cm loads (limit 63 → 31°).

- **1.3.0** flying backwards (§8.24); MPO SeparateRollYaw unchanged; 150 / 159 suite files identical (the 9 are past-90° damage cases).

19.10 FS-12 Revoker (1.0.0 → 1.4.1)

- **1.0.1** loaded reversal; **1.1.0** performance port; **1.1.1** one DLL.
- **1.2.0 · The limit followed throttle and speed (11 Nov).** The thrust × CG-offset yaw was in the load, the yaw nozzle not in the authority, and a least-demand fallback returned 41–70° when no AoA passed. Coupled check with both nozzles as columns, share 0.7: about 28° at 30 cm, steady with speed and throttle (was 12° fast, 40–70° slow on afterburner).
- **1.3.0 · Nozzles locked fell back to the old check (12 Nov).** `CplNoTvc`: the same LP without the nozzles, the nose-down cap from its held-pitch figure, its own share 0.8 (at 0.7 a locked-nozzle pull at 500–550 km/h ran 6–9° past with 13° of sideslip; at 0.8 within 2°, ≤ 1.4°). Plus the native LP / grid cache / one-side solve (sweep 510 → 299 µs). Open: low-energy no-TVC cases.
- **1.3.1** ground braking (release stops 8–15 % shorter). **1.4.0** flying backwards: in Tom's SeparateRollYaw MPO the stick is the velocity-vector roll, so it changes there; that mode holds sideslip only with the stick in, so it stays after a release (29.9 / 24°) as the pedal cases already did. **1.4.1** `MaxPitchRate` 48 → 60 (config only, not benched).

19.11 T/A-30 Compass (0.2.0 → 0.5.0)

- **0.2.1** loaded reversal; **0.3.0** performance port; **0.3.1** one DLL.
- **0.4.0 · 12° and 7° limits where the jet held 25–30° (11 Nov).** Coupled check (share 0.75; 0.7 gave 11.6° of sideslip in a zoom to 30 m/s), the nose-down limit from the same LP with a q-scaled margin (the 7° cap under ~62 m/s is gone), thrust about the offset CG in the load (limit to its 12° floor under ~100 kt on afterburner), negative limit −30° (was −14 to −27°). 22–25° at 32 cm.
- **0.4.1** native LP + grid cache + one-side solve: 413 / 413 identical; 467,226 LP calls compared both ways, all identical; sweep −41 %. **0.4.2** ground braking (15–19 % shorter). **0.5.0** flying backwards: the worst sideslip cost of the six (peaks 20–32°, −21° still 3 s after a 186 km/h release).

19.12 FS-3 Ternion (0.2.0 → 0.4.0)

- **0.2.1** loaded reversal; also **bench calibration** from Tom's 0.2.1 log (`claude/Ternion_0.2.1_log_and_bench_calibration`): the camber table the game uses (`CamberGameTable`; roll error at 40–50° AoA 10.3 → 3.4), thrust yaw from the CG offset, stores found by part name. Calibrate before you trust a heavy-load bench result.
- **0.2.2 · Heavy load: 7° drops and pitch wobble (11 Nov).** The first coupled check (§8.23): LP with attached-flow surfaces only, nose-down margin scaled with airspeed; `LiveAuthCap` (an extrapolated nose-down limit) off. A first attempt counted stalled surfaces and promised 18–60 deg/s² of roll that never came (`claude/Ternion_0.2.2_attempt`).
- **0.2.3 · The negative limit followed the flying.** A cost-saving shortcut solved the negative side only near zero AoA: −18 to −20° in a pull, −40° in a push, widening over 2.5 s. `CplNegAlways`.
- **0.3.0** performance port + the LP in Rust + one DLL. **0.3.1** one-side solve; the `CplNear` self-test returned NaN on Mono and switched the core off (fixed in all six). **0.3.2** ground braking (11–15 % shorter). **0.4.0** flying backwards.

19.13 Su-30VXN Gandiva (0.2.3 → 0.6.0)

- **0.2.4 (10 Nov)** no-TVC caps unclamped from 45 (default 50); the pitch wall's peak pin ends 2° short of the limit **in force** (with TVC off it held the canards at −34° up to 81° AoA against a 50 cap: replay 60.5 → 52.9°); push plan shed refit to this jet's logs; `PushoverSideslip` 15 / `PushoverSideslipNoTvc` 12; `PushOverMinRate` 23; Buffed thrust ×1.15.

- **0.2.5 · The AoA wall at 56–58° (pitch rate 70 → 0 for 1.3 s).** `HighAoAPitchBump` (`PitchBump.cs`): a pure nose-up couple on the body sections ahead of / behind the CG, 53–70° AoA, no net lift or drag; best nose-up at 55–65° filled from 13–33 to 62–71 deg/s². An airframe fix, flagged as such (§2.13).
- **0.3.0** AoA limit 75; MPO release stop in the nozzle-axis rotation (`MpoAxisReleaseStop`: yaw under 10 °/s 0.6–3.5 s sooner); pedals drive the nozzle differential (`AxisPedalRoll`); `LowSpeedSlipTvc` experiment. **0.3.1–0.3.3** `LowSpeedSlipTvc` rebuilt twice (stability-axis yaw, an authority check that gives up when the effectors are not turning it); `PaceTasMin` 40 → 20 m/s (every lateral protection was idle at 25–40 m/s). Finding: in the low-speed corkscrew every yaw effector was saturated; the fix needs more roll with less yaw, the one combination canted nozzles cannot make.
- **0.3.4 · Config revision lost every launch.** `Su30ConfigRevision` was bound **after** `MigrateOldConfig`, which clears unbound keys and saves, so every "moved once" step was skipped. Bind revisions first (§13). Also `MpoRelPastLimit`: the release stop whenever MPO flies past the limit.
- **0.3.5 · Full left stick rolled right for 24 s.** The push governor's band was refreshed only inside its block but applied every tick from stale values; cleared each tick now. `GyroRollTrade`: the gyroscopic budget also limits the roll (corkscrew peak sideslip 37.8 → 24.2°). **0.3.6** `LsSmooth` (fades instead of switches). **0.3.7–0.3.10** the soft AoA-limit hand-over: `SoftMpoStick`, `SoftMpoFromMpo` ("MPO then assist on: no recovery"), `SoftPitchHold` (past the limit the AoA law does not push the nose down by itself).
- **0.3.8** loaded reversal; **0.4.0** performance port; **0.4.1** one DLL.
- **0.5.0 · The limit swung 12–75° with the stick (11 Nov).** Coupled check (§8.23) with nozzles as columns and `ClampStabToPeak` stab points; `FindTop`. **0.5.1** `AsymStopTime` 0.8 s (bank after a release 47–117 → 27–44°; roll into the heavy side slower). **0.5.2** `CplKnee` / `AsymAoAWeight` (limit ~45° for 30–50 % faster roll into the heavy side).
- **0.5.3 / 0.5.4** ground braking, origin of §8.26. **0.5.5 · Flap droop tied to the gear lever.** `FlaperonFlapSchedule`: the flaperons follow the F-22E flap schedule at every gear state (1 g droop 20 / 14 / 10 / 2° at 119 / 140 / 162 / 216 kt).
- **0.5.6** native LP + grid cache (sweep −37 %). **0.5.7** `MaxPitchRate` 60 (0.7 stick at 450 km/h: 29 → 64° AoA); `StopHardOnReversal` ("stop the roll or yaw even if the sideslip goes past the margin": falling-leaf yaw stopped 2.4 s sooner, peak sideslip 30 → 37°).
- **0.6.0** flying backwards; here the TVC vector roll's start angle is mirrored too (at 135–150° left stick used to turn it right).

19.14 Across all jets (10–12 Nov)

- **The performance port** (11 Nov): the F-22E's Rust allocator, sweep cache, kernels and tick segments ported to the other five, then the coupled check's native LP, grid cache and one-side solve to all six (12 Nov, "bring all of them to the same level"). Each port was checked by bench identity (per-case CSVs) and `NativeAllocatorCheck` 2 both-ways runs, then on the game's Mono under Wine, each jet alone and all six in one process.
- **Mod manager state** (11 Nov): every FCS mod except the F-22E was listed in `disabledPlugins`, and the Ternion in game was still 0.2.0 in a nested folder. Its `meta.json` records still list older versions, and a modpack redeploy can put them back (§12.4).
- **Flight status:** the per-version notes above record what was benched or flown at the time. The releases in this kit (F-22E 4.3.0, Ifrit 1.3.0, Revoker 1.4.1, Compass 0.5.0, Ternion 0.4.0, Su-30 0.6.0) have since been flown and passed in game by the author (12 Nov 2026).

20. History and further reading

- `docs/papers/Measured-State_FCS_paper_(Oct2026).pdf`: the architecture, the allocator failure catalogue, damage tolerance and the first Ifrit port (F-22E 1.4.0–2.2.23). The governor has since been rebuilt; read §2.10 here for the current one.
- `docs/papers/F-22E_FCS_How_the_Control_Laws_Work_(Sep2026).pdf`: the control laws explained for players (early version).
- `docs/design-history/RefGov_port_Ifrit_Revoker_handoff.md`: the reference-capacity governor (F-22E 3.0.0) piece by piece, and how it was ported to two very different airframes. The best single document on porting.
- `docs/design-history/FCS_performance_handoff.md`: CPU cost per tick (~0.85–0.95 ms median on the bench for the F-22E, mostly allocator solves and the estimator's probes) and the plan to move the estimator to a worker thread.
- `docs/design-history/F22E_FCS_handoff_2.31.0.md`: the F-22E state at 2.31.0.
- `reference/<Jet>/native/README.md`: the native core's rules, build and checks (F-22E's is the fullest).
- `docs/design-history/F22E_airframe_X2_handoff_2026-11-09.md`: the build hand-off for the F-22E's X2 airframe (goals, decision, acceptance targets, phase 2), with the study kit in `reference/F22E-FCS-4.3.0/studies/x2_study_kit/`.
- Documents named `claude/...md` in this manual are the author's project notes (6–12 Nov findings); they are not in the kit, and what they found is summarised in §8, §10.7–10.8 and §19.
- Each reference tree's `release-notes/` and its `README.md` / `bench/CALIBRATION*.md` / `bench/DAMAGE.md`.

Lineage: SimplePlanes F-22 FCS (Funky Trees) → F-22E FCS 1.4 (21 Sep 2026) → 2.0 (stagger, damage) → 2.1 (G + rate law) → 2.2 (rolling-pull governor, decelerons) → 2.4 (multiplayer policy, ground) → 2.5–2.7 (governor rework) → 2.27–2.44 (one reference-capacity governor, push plan, reach pass) → 3.0 → 3.0.4–3.1.3 (several cores) → 3.1.4–3.1.8 (push-over floor and hand-offs) → 3.1.9 (fuel hook) → 3.2.0–3.2.1 (relaxed-to-neutral airframe X2 / X2e) → 4.0–4.1 (native core, one DLL, loaded reversal, ground brakes) → 4.2.0 (coupled check, off) → 4.3.0 (flying backwards). Ifrit 0.1 (port of 2.2.5) → 0.9 (3.0.0 governor) → 0.16 → 0.17 (push-over hand-offs) → 1.0.0 (several cores) → 1.3.0. Revoker 0.1 (28 Oct) → 0.6 (3.0.0 governor) → 0.8.3 → 1.0.0 → 1.4.1. Compass 0.1.0 (5 Nov, fork of Revoker 0.8.2) → 0.1.12 → 0.2.0 → 0.5.0. Ternion 0.1.0 (7 Nov, fork of Compass 0.1.11) → 0.1.12 → 0.2.0 → 0.4.0. Su-30 0.1.0 (7 Nov, fork of Ternion 0.1.8) → 0.2.3 (10 Nov) → 0.6.0 (12 Nov).

Kit editions: 1.0 (5 Nov 2026: F-22E 3.0.1, Ifrit 0.16.0, Revoker 0.8.2); 2.0 (10 Nov 2026: six jets, §19.1–19.7); 3.0 (12 Nov 2026: F-22E 4.3.0, Ifrit 1.3.0, Revoker 1.4.1, Compass 0.5.0, Ternion 0.4.0, Su-30 0.6.0; native core, coupled check, loaded reversals, flying backwards, ground brakes; §19.8–19.14); 3.1 (12 Nov 2026: Part IV, §21–24).

PART IV

Background and practice

21. Control theory primer for this FCS

This part is for modders coming from the stock fly-by-wire who have not met flight control theory. It covers only what you need to read this kit's code and README. Each idea gets the equation, a plain-English reading, and what it means in Nuclear Option. Symbols: body axes x forward, y right, z down; roll p, pitch q, yaw r (body rates, deg/s or rad/s).

21.1 Angles and air data

The jet's velocity in body axes is (u, v, w), with speed $V = \sqrt{u^2 + v^2 + w^2}$.

$$\alpha = \arctan\frac{w}{u} \qquad \beta = \arcsin\frac{v}{V}$$

- **AoA (α)**: how far the nose is above the flight path, in the jet's own vertical plane. Lift, stall and most limits are functions of α.
- **Sideslip (β)**: how far the air comes from the side. A fighter wants β near zero: sideslip costs lift, adds drag and, past the fins' authority, starts a departure.
- **Dynamic pressure** $\bar{q} = \frac{1}{2}\rho V^2$: how hard the air pushes. Every aerodynamic force and moment is $\bar{q} \times$ an area $\times$ a coefficient, so **control power scales with** $\bar{q}$. In Nuclear Option a fighter flies from about 50 to 400 m/s, so $\bar{q}$ (and with it what one degree of stab does) changes by about $(400/50)^2 = 64\times$ at the same altitude.

These are measured at the **CG** in this FCS (README §2.4): the game's own AoA reading is taken at the root part, which differs in a fast rotation.

21.2 The rigid body

The jet's rotation obeys Euler's equations:

$$I\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (I\boldsymbol{\omega}) = \mathbf{M}$$

$\boldsymbol{\omega} = (p, q, r)$ are the body rates, I the inertia tensor, $\mathbf{M}$ the sum of every moment (aero, thrust, gear). In plain words: **moment = inertia × angular acceleration, plus a gyroscopic term**. That second term is **inertial coupling**. For the pitch axis:

$$\dot{q} = \frac{M_y}{I_{yy}} + \frac{I_{zz} - I_{xx}}{I_{yy}}\, p\, r$$

A fighter is long and heavy along its fuselage, so $I_{zz} \gg I_{xx}$, and a fast roll at high AoA (where roll and yaw rates are both large, §21.6) makes a big nose-up moment on its own. The FCS has to keep nose-down authority in hand to absorb it: that is the **coupling roll limit** (README §8.15). Su-30 example: yaw inertia about 600,000 kg·m² against 27 t of mass.

21.3 Static stability and the static margin

The **neutral point** is where extra lift from more AoA acts. The **static margin** is how far the CG sits ahead of it, as a fraction of the wing chord $\bar{c}$:

$$\mathrm{SM} = \frac{x_{np} - x_{cg}}{\bar{c}} \qquad C_{m_\alpha} \approx -C_{L_\alpha}\,\mathrm{SM}$$

- SM > 0 (CG ahead): a gust that raises the nose makes a nose-down moment. The jet points into the wind on its own: **stable**.
- SM = 0: **neutral**. The jet holds whatever AoA it is put at.
- SM < 0: **unstable**. It diverges unless a controller holds it.

The pitch "spring" C_{m_α} grows with the margin. The short-period frequency (how fast the nose settles) grows roughly with $\sqrt{\bar{q}\cdot\mathrm{SM}}$.

Why stock-FBW modders put the CG far forward: the stock FBW has no real AoA protection (it only scales the pitch-rate target down past 25–31°, §1.1), so a big margin's natural nose-down does the protecting. The cost:

- the tail must push down to hold any AoA (**trim drag**, less lift);
- the stabs spend travel just holding the nose up, so less is left for pitch rate and for roll at high AoA;
- the maximum AoA ends up set by tail authority, not by a limit you chose.

With a real FCS the margin can come down toward neutral, because the controller provides the stability (README §8.15: the F-22E's airframe X2e is relaxed to neutral). This FCS still works on a stable airframe, but it can't create nose-up authority that isn't there.

21.4 Why one fixed-gain PID struggles

A PID controller sets its output from the error e = target − measured:

$$u = K_p\, e + K_i \int e\, dt + K_d \frac{de}{dt}$$

The loop's real gain is K_p × how much moment one unit of u makes, and that is proportional to $\bar{q}$. So with fixed gains:

- tuned at high speed, the jet is sluggish slow (loop gain 64× too low at the bottom of the range);
- tuned slow, it oscillates fast.

The stock FBW divides its output by $\bar{q}/\bar{q}_{\text{corner}}$ above the corner speed (`remapFactor`, §1.1). That is a one-number **gain schedule** (gains changed with flight condition). It doesn't account for AoA, stall, a surface at its stop or a missing tail. Other PID problems that show up in this game:

- **Saturation and windup:** when a surface hits its stop, the integrator keeps growing, then overshoots when it comes off.
- **Coupled axes:** a PID per axis ignores that at high AoA roll and yaw are one motion (§21.6) and that a canted tail makes pitch and yaw together.
- **Fixed mixing:** each surface moves by stick × a fixed range. Nothing redistributes when one surface stalls or is shot off.

21.5 What the stick commands

- **Rate command:** stick = rotation rate. Let go and the rotation stops. Natural at low speed and for roll.
- **G command (load factor n_z):** stick = g. In a turn $n_z \approx Vq/g_0$, so at high speed a small pitch rate is a lot of g, and g is what the pilot (and the airframe) cares about.
- **AoA command:** stick = AoA. Natural near the stall.

This FCS's pitch law blends them (`GAndPitchRate`, README §2.8): g where full g is reachable, rate where it isn't, and AoA / g limits on top.

21.6 Velocity-vector roll and sideslip kinematics

Rolling about the **nose** (body x-axis) at high AoA does not roll the flight path; it swings the nose sideways around the velocity. The flight path rolls about the **velocity vector**. In body rates, a velocity-vector roll at rate $\dot{\mu}$ is:

$$p = \dot{\mu}\cos\alpha \qquad\qquad r = \dot{\mu}\sin\alpha$$

At 45° AoA that is equal parts body roll and body yaw; at 70° it is mostly yaw. The sideslip then changes as:

$$\dot{\beta} \approx p\sin\alpha - r\cos\alpha + \frac{g_0}{V}\cos\theta\sin\phi + \frac{Y}{mV}$$

(the last two terms are gravity in a bank and the side force). Put a pure body roll ($r = 0$) into it: $\dot{\beta} = p\sin\alpha$. At 30° AoA a 100 °/s body roll builds **50 °/s of sideslip**. That is why "rolling hard while pulling" departs with the stock FBW, and why this FCS commands velocity-vector roll and spends its yaw authority to make r follow (README

§2.9). The yaw the roll needs is also why the roll rate is limited at high AoA by **yaw** capacity, not roll capacity (the governor, §2.10).

Flying backwards (AoA past 90°), the kit mirrors the axis: the roll share is $|\cos\alpha|$, so 135° answers like 45° (README §8.24).

21.7 INDI: incremental nonlinear dynamic inversion

The model-based way to make a jet follow a wanted angular acceleration $\boldsymbol{\nu}$ would be to compute every moment on the jet and invert that model. Every model error goes straight into the control. **INDI** only models the part it changes:

$$\mathbf{M}_{\text{req}} = \mathbf{M}_{\text{now}} + I\,(\,\boldsymbol{\nu} - \dot{\boldsymbol{\omega}}_f\,)$$

- $\dot{\boldsymbol{\omega}}_f$ is the **measured** angular acceleration (filtered).
- $\mathbf{M}_{\text{now}}$ is what the effectors make now (from their models).
- The bracket is "how much more acceleration do I need than I'm getting". Only that **increment** is asked of the effectors.

Everything you didn't model (fuselage lift, damping, gear, damage, a mod's odd airframe) is already inside the measured acceleration, so it cancels. What must be right are the **effector models**: what each surface makes at its local AoA across its travel (README §2.7, §9.1). In practice this removes per-axis gain tuning. The main knob is the filter time constant (`IndiTau` 0.04 s here): too short and noise gets through, too long and the loop lags.

The outer loops are simple: the wanted acceleration comes from the rate error, e.g. $\nu_q = \left(q_{\text{cmd}} - q\right)/\tau$.

21.8 Control allocation

INDI gives a wanted moment $\mathbf{m}$ (three numbers). The jet has 6–14 effectors. **Allocation** picks their positions $\mathbf{u}$:

$$\min_{\mathbf{u}} \;\; \|\,W(B\,(\,\mathbf{u}\,) - \mathbf{m})\,\|^2 + \text{costs}\,(\,\mathbf{u}\,) \quad \text{subject to} \quad \max\,(\,u_{\min},\, u_{\text{now}} - \dot{u}_{\max}\Delta t\,) \le \mathbf{u} \le \min\,(\,u_{\max},\, u_{\text{now}} + \dot{u}_{\max}\Delta t\,)$$

- $B\,(\,\mathbf{u}\,)$: the moment the effectors make at positions $\mathbf{u}$ (nonlinear: surfaces stall, reverse, interact).
- W: axis weights (which axis wins when not everything is reachable: here pitch and sideslip outrank roll).
- The bounds are the travel limits **and** how far each servo can move this tick (100 °/s servos on all six jets, README §3).
- The costs keep the solution sensible: small deflections, no surfaces fighting each other, nozzles near trim.

Here it is a box-constrained Gauss-Newton solve (README §2.6). What allocation buys: a stalled or missing surface is simply worth less in B, and the solve uses the others. That is where damage tolerance comes from.

The **coupled asymmetric-load check** (README §8.23) is a small **linear programme** (a linear goal under linear limits) built on the same effector tables: "with the load held, what roll is left each way?"

21.9 Actuators, filters and discrete time

- The FCS runs once per physics tick (about 60 Hz in this game, README §12.5). Everything is discrete: a first-order low-pass filter is $y \leftarrow y + \frac{\Delta t}{\tau}\,(\,x - y\,)$.
- Surfaces move at a limited rate (100 °/s by default). A full stab swing of 60° takes 0.6 s. Rate limits are often what bounds a reversal, not the deflection limits (the loaded-reversal problem, README §8.25, was rudders taking 0.5–0.7 s to cross).
- Every filter and every rate limit adds **lag** (phase delay). Lag in a feedback loop is what turns a tight loop into an oscillating one. The FCS predicts where effectors will be (the allocator's reach pass, two-horizon pick) instead of reacting to where they are.

21.10 Envelope protection and capture

A limit has to be **approached**, not hit. The AoA rate must be brought to zero by the time the AoA reaches the limit. With the nose-down acceleration available $\ddot{\alpha}_{\max}$, the stopping distance is:

$$\Delta\alpha_{\text{stop}} = \frac{\dot{\alpha}^2}{2\,\ddot{\alpha}_{\max}}$$

So the FCS starts braking at limit − $\Delta\alpha_{\text{stop}}$ (**kinematic capture**). The hard part is knowing $\ddot{\alpha}_{\max}$ in the current state: thrust, nozzle angle, stall, load, damage. That is why this FCS measures it (capacity estimator, sweeps) instead of using a table, and why a wrong authority read shows up as an overshoot.

21.11 Measured state over schedules

A **gain schedule** stores tuned numbers per flight condition (speed, AoA, altitude) and interpolates. It works on a known, undamaged aircraft. This FCS reads authority and limits from the **live** state instead: surface models at their actual local AoA, live thrust, measured accelerations (design rule 1, README §14). The payoff is that one law works across six very different airframes, with damage and stores, without retuning tables per jet. The price is complexity: every model has to be right, and the bench has to check it.

21.12 Further reading

- B. L. Stevens, F. L. Lewis, E. N. Johnson, *Aircraft Control and Simulation* (Wiley): rigid-body equations, stability, classical and modern flight control.
- R. C. Nelson, *Flight Stability and Automatic Control* (McGraw-Hill): static and dynamic stability, the gentlest start.
- S. Sieberling, Q. P. Chu, J. A. Mulder, "Robust Flight Control Using Incremental Nonlinear Dynamic Inversion and Angular Acceleration Prediction", *Journal of Guidance, Control, and Dynamics* 33(6), 2010: INDI.
- W. Durham, K. A. Bordignon, R. Beck, *Aircraft Control Allocation* (Wiley, 2017): control allocation.
- `docs/papers/Measured-State_FCS_paper_(Oct2026).pdf` in this kit: how these ideas are applied here.

22. Workflow and tools

22.1 The loop every change went through

About 700 builds (Sept–Nov 2026), 210 release notes across the six jets, each through the same loop:

1. **Report.** The pilot flies and describes what felt wrong, in his words, with the flight log (`Telemetry` CSV, one row per tick, 260+ columns) and `LogOutput.log`.
2. **Check what was installed.** The version line in `LogOutput.log` and the DLL's md5 against the release. Mod managers rolled DLLs back more than once (README §12.4).
3. **Read the log at the reported moment.** Commands, measured rates, every surface's command and position, the limits in force, the capacity reads. Find which number did something it shouldn't.
4. **Reproduce on the bench.** Replay the log from the logged state with the logged stick inputs (`bench.sh replay`). If the bench can't reproduce it, find out why and fix the bench first (README §10.7).
5. **Find the cause, not the symptom.** Which model is wrong, or which device owns this behaviour. No gates and no special cases (design rule 2).
6. **Change it behind a switch,** so old and new run from one build.
7. **A/B on the bench:** the regression suite, the targeted sets for that area (loaded reversals, asymmetric matrices, tail slides, ground), ensembles for chaotic cases, curve-quality checks. Refactors must be byte-identical (README §10.6).
8. **Write the release note:** what, why, the pilot's words, before → after tables, status (compiled / benched / flown).
9. **Build, install** with a `.bak` of the old DLL, read the md5 back.
10. **Fly it.** The next report starts the loop again.

Then, when it works on one jet, **port it** to the others: same law, each jet's own effectors and constants, each jet's bench, each jet's release.

22.2 Tools

Job	Tool used
Read the game's code	the game's `Assembly-CSharp.dll` decompiled (ILSpy / dnSpy); `bench/groundsim/ildump.py` (dnfile + dncil) dumps one method's IL with names resolved
Patch the game	BepInEx 5 + HarmonyX (prefix / postfix / transpiler), Configuration Manager for settings
Build the plugin	.NET SDK's Roslyn `csc` against the game's Mono assemblies (`build.sh`), C# 9, deterministic
Native core	Rust (`x86_64-pc-windows-gnu`) + mingw-w64 gcc; Linux `.so` for the bench (README §15.3)
Check on the game's runtime	Wine + the game's own `mono-2.0-bdwgc.dll` (`native/verify/`)
Aircraft data	UnityPy for asset bundles (mod DLLs carry a `UnityFS` bundle; stock prefabs are in `resources.assets`); the bench's `extract*` scripts
Simulate	the bench (`dotnet bench.dll` , .NET 8), the ground models in `bench/groundsim/` (Python)
Analyse logs and bench output	Python with pandas / numpy / scipy; ~40 scripts in each `bench/an/`
Manual	pandoc + Playwright Chromium (`tools/build_manual_pdf.py`)

22.3 Why an LLM agent was the better fit for this project

The kit was developed with an LLM agent: a model with a shell, so it could run the bench, read logs and build, rather than a chat window. The reasons it fit this work better than doing it by hand:

- **Iteration speed is the whole game.** The loop in §22.1 is mostly reading, writing and running: a log of ~3,600 rows per minute of flight × 260+ columns, a replay, a suite (~10 min), a diff, a note. An agent does a full turn in tens of minutes. By hand, with a day job, one turn is an evening. 700 builds at an evening each is two years. Many fixes needed three tries before the right cause was found, so turn count matters more than any single turn.
- **Six codebases kept in step.** 26,000–34,000 lines of C# per jet, each with its own effectors and constants. A law change is ported five more times, each benched. On 12 Nov the flying-backwards law went from the F-22E to the other five jets, each benched and released, in one session. The 11 Nov performance port moved the Rust allocator, kernels and tick timers to five jets in a day.
- **Mechanical work that has to be exact.** The Rust core is ~2,000 lines transcribed from C# expression for expression, bit-identical on the bench. That kind of transcription is slow and error-prone for a person, and fast for a model. The identity checks then prove it, independently of who wrote it.
- **Throwaway tools on demand.** Each question gets its own analysis script: a fit of the load estimate against elevator position (R^2 0.84), an asymmetric-load matrix, a loaded-reversal set of 88 runs. About 40 such scripts per jet. By hand you'd write a few and reuse them past their fit.
- **Breadth in one place.** One change can touch control theory, aerodynamics, Unity internals, Harmony, IL, Rust, Wine, asset extraction and statistics. An agent carries all of them; a person usually has two or three and looks the rest up.
- **Documentation as a by-product.** Every build has a release note with numbers and status, which is what made this kit (and §19) possible.

What it did **not** replace: the pilot. Every law decision came from flying, and the agent was wrong in documented ways (README §19): it misread a flight report (Su-30 0.3.7, reverted in 0.3.10), gave optimistic performance estimates before measuring (F-22E 4.1.0 notes), and drew conclusions from a bench tool that silently did nothing (§13 F26). The bench identity checks and the next flight are what caught those, not the agent.

23. Doing it by hand: caveats

Porting from this kit without an LLM is realistic; developing new laws at this pace is not. What to expect, and how to keep it manageable:

- **Time.** Rough estimates, not tested with a human porter: a jet close to one of the six references (same kinds of effectors) flies sanely after days of work and reaches reference quality after weeks of flying and logs. A new effector type (canted nozzles, an unusual surface) means understanding the allocator and the capacity estimator: weeks to months.
- **Size.** `FcsController.cs` is 11,400 lines on the F-22E, with 326 config bindings. Read `Tick` top to bottom once (README §2.3 is the map, `AGENTS.md` §5 the "where is it" table) before changing anything. Most of the switches are history; keep the defaults of your reference jet and change only what `PORT_TODO.md` lists on the first port.
- **Forget the Rust core at first.** Every native routine has a C# twin; `NativeAllocator = 0` runs all C#, with the same flying on the bench and only more CPU (README §2.18). **Trap:** if you edit the surface aero code (`SurfaceEffector.Eval` and its relatives, `Camber`, `AirfoilTable`), the native kernels and their load-time self-test twin no longer match your C#, and the self-test **still passes** (it compares the kernel to its twin, not to your new code). Turn the kernels off (`NativeTables=false`, `NativeSweep=false`) or keep the Rust in sync (`native/README.md`).
- **One jet at a time.** A fix you make on your jet stays on your jet. Don't try to keep a port in step with all six references; take the reference's newer release notes as a list of things to consider.
- **The bench is required, not optional.** Without it, every test is a flight. Build the bench model (README §9.2), check it against one flight log before trusting it, and reuse `bench/an/` scripts rather than writing new ones.
- **Wall time.** A suite is ~10 min on 2 cores; chaotic cases need 5-run ensembles; one law change is an evening of runs. Batch the runs and read the results the next day.
- **Theory.** If §21 is new to you, read it and one of the books in §21.12 first. Without the velocity-vector roll and the static margin, the port checklist can be followed but what comes out can't be debugged.
- **Your configuration may be new.** VTOL / STOVL, swing wings, tailless and other layouts in §24.10 have no reference jet; porting one means writing laws, not following a checklist.
- **Keep a log of your own.** What you changed, the before → after numbers, and whether it was flown. §19 exists because every build had one.

24. Airframe aerodynamics: recommendations for modders

The FCS can only use the authority the airframe gives it. Most of the hard problems in §8 and §19 were partly airframe problems. This section collects what was learned about shaping a Nuclear Option airframe so a real FCS can fly it well. Every recommendation names the jet and the evidence. These are lessons from six airframes, not universal targets.

24.1 What the game models, and what it leaves out

- **One plate per part.** Each part is a flat aerodynamic element with an airfoil table (lift and drag against local AoA), an area and a centre of lift. Lift and moment come from summing the plates (README §18 "Plate").
- **A control surface is its own plate.** Deflecting a flap changes only the flap's own force. On a real wing the fixed part ahead carries part of that extra force (**camber coupling**). The game leaves it out, so ailerons and rudders are weak: adding it raised the Ifrit's roll at 135 kt from 106 to 151 °/s (README §8.11).
- **Surfaces that share an airfoil stall together,** and leave a hole in roll authority at that AoA (README §8.10).
- **No tail downwash by default.** The tail sees the full body AoA, so an all-moving stab reaches its lift peak early and has nothing left for nose-down past it. In the F-22E X2 study, taking the plugin's downwash out (with the stab peak at 36°) left only −5 °/s² of full nose-down stab from 36 to 48°.
- **Strakes and LERX fly the wing's foil.** No vortex lift: the Compass's strakes stalled at 17° like the wing (README §8.20).

The kit's aero mirrors (`Camber`, `StallStagger`, `Downwash`, `BodyLift`, `Strake`) add these to the game and to the FCS's models together. Any airframe change you make must go into both, or the FCS's tables are wrong (README §9).

24.2 Pitch: aim for a small margin, not a big one

- **Don't use a forward CG as your AoA protection.** That's the stock-FBW habit (§21.3). It costs trim drag, stab travel and roll authority at high AoA, and the FCS already protects the AoA.
- **Relaxed toward neutral over the usable AoA.** The F-22E X2e has a static margin of ±0.03 m from 4 to 36° (README §8.15). The Compass runs its CG 0.15 m aft. Further (0.27 m) pinned both elevators nose-down in low-speed rolls, because the roll's inertial pitch-up then had nothing to push against. The game moves the CG at most ~0.18 m aft (`Registry`, 0.6 × the fore part's mass).
- **Keep natural nose-down past the AoA limit.** Three things spend the airframe's nose-down at high AoA:
  - the coupling roll limit, sized from the nose-down headroom (§21.2);
  - the brake at low thrust, when TVC has no authority;
  - the no-TVC fallback.

  The F-22E X2 removed it at 45–55°: full nose-down stab at 48° went −41 → −18 °/s², the coupling limit fell to 24–54 °/s and the velocity roll at 45–55° dropped to about half. X2e put it back (−44 / −41 / −36 °/s² at 48 / 52 / 55°).
- **But not more nose-down than the controls can beat.** The Su-30's "pitch wall": above ~400 km/h at 55°, the airframe's nose-down exceeded every surface and nozzle combined, and pulls stopped at 50–65° (README §8.19). The fix was the airframe, a nose-up couple at 53–70° (`HighAoAPitchBump`): best nose-up at 55–65° went from 13–33 to 62–71 °/s².
- **The test for both:** at each AoA and a few speeds, compute the **best-case** nose-up and nose-down acceleration with every effector at its best. Both must stay clearly non-zero through the envelope you want to fly. The bench's `effs` / `iauth` maps give these (README §10.2).

24.3 Change the moment without changing the lift

When the pitch curve needs reshaping, move lift between the parts ahead of and behind the CG, so the **total lift and drag are unchanged** and only the moment moves:

- F-22E X2 / X2e: AoA-scheduled multipliers on the fore and aft body sections, fitted (`fitsched.py`) so the stabs-neutral pitching moment is zero over 4–48° (README §8.15);
- Su-30 `HighAoAPitchBump`: +K on the sections ahead, the same total less behind. On the bench the net lift and drag changed by less than 0.0001 over the whole pose grid.

This keeps the jet's turn performance and energy exactly where they were, and confines the change to handling. Fade every such change out before post-stall (X2e: 45 → 60°), so spins and tail slides stay as the game made them.

24.4 Control surfaces

- **Stagger the stall.** Give surface families their own lift peaks so pairs don't all stall at one AoA (`StallStagger`, README §8.10). Keep the peak lift (keep-peak-CL mode); the keep-slope mode lost 15–30 % of roll. Staggering alone does not add roll; it improves low-AoA stability and removes the hole.
- **All-moving tails want their own curve.** A slab flies a swept-slab curve (Compass: stall 30°, peak CL 1.22), not the wing's 17° foil. Raising a stab's peak (F-22E 25 → 36°) keeps it working at high AoA, but its small-angle slope falls (0.69× there) and it needs more travel for the same pitch.
- **Add camber coupling** for flaps, ailerons and rudders (K from the chord fraction, 0.46–0.57 on these jets).
- **Canards are ahead of the CG,** so their stall acts the other way in pitch from a tail's. Give them asymmetric travel and keep them in their attached-flow window at high AoA (Revoker, README §8.6).
- **Canted fins make inverse roll at high AoA** with the rudder. Count them as sideslip effectors there (F-22E, README §8.2).
- **Servo speed matters as much as travel.** All six run 100 °/s (Aryx's F-22E shipped 70). A reversal is often bounded by how long the rudders take to cross (0.5–0.7 s with a load, README §8.25), not by how far they go.

24.5 Yaw authority sets the high-AoA roll rate

A velocity-vector roll needs body yaw rate $r = \mu \sin \alpha$ (§21.6). At high AoA the roll rate you get is set by the **yaw** sources: fins, differential tails, split brakes, TVC. If you want a jet that rolls at 60° AoA, give it yaw there:

- the Ifrit's roll TVC is about 48 % of its sideslip authority at 150 km/h and 65°, and the only part that doesn't fade with speed (README §8.4);
- the F-22E's canted rudders lose sideslip authority sharply at 55°, which is also where its asymmetric-load demand curve spikes (README §8.23).

Fins blanked by the fuselage at high AoA are the usual limit. Canted fins, ventral fins, yaw or differential TVC and split brakes are the airframe answers.

24.6 Strakes and LERX: placement decides the sign

A leading-edge extension's vortex adds lift that grows with AoA (Polhamus: $\Delta C_L = K_v \sin^2 \alpha \cos \alpha$). It sits **ahead** of the CG, so it is also a growing nose-up moment:

- on the Compass, a slender-strake curve with breakdown at 30–50° (`StrakeVortexLift`, area ×3.4) fixed a flat stalled strake;
- on the F-22E, an explicit vortex on top of the existing body lift made the jet nose-up past 24° and untrimmable at 32–40°.

Add vortex lift only with a pitch check across the whole AoA range (§24.2), and expect to trade it against the CG.

24.7 Thrust and TVC

- **Put the thrust line through the CG's height,** or trim changes with throttle and full afterburner can lift the nose wheel. The Su-30's nozzles sat 0.49 m below the CG: nose up at 80 km/h, tail strike at 13.5°. `ThrustLineAtCg` fixed it at the source; a nose-wheel hold loop was a symptom fix (README §8.18).
- **TVC disappears with thrust.** At idle, or above the vectoring speed, the airframe must recover on its own surfaces. Design the no-TVC case first (README §8.14).
- **Canted single-axis nozzles** make roll and yaw together and can't make one without the other (Su-30, README §8.17). That is fine for a velocity roll at high AoA, and a limit when the job is "more roll, less yaw".

24.8 Ground and mass

- **CG vs mains.** The Su-30's CG was 0.74 m ahead of the mains and the static nose-wheel share 9.6 %: it tipped onto its tail taxiing. Moving the CG forward and restoring the margin with foreplane area (canards ×1.3 → 1.6) brought it to 12.2 % with the low-AoA pitch slope within ~7 % (README §8.18). For comparison the F-22E and Ifrit ground models carry 18 % and 24 % of the weight on the nose.
- **Track width** sets how much yaw a main brake gives: the F-22E's 2.47 m track gives less than the Ifrit's 3.94 m. `differentialBrakeFactor` must be −1 / +1 on the mains for any stock differential braking (the F-22E has −1 / −1, README §8.26).
- **Fuel moves the CG.** On the Su-30 burning fuel walked the CG about 30 cm aft (`FuelCgHold`). Check where the tanks are against the CG, and plan for the margin full and empty.

24.9 How to make an airframe change

1. **Decide "airframe or FCS?" with data.** Fit the game's own pitch (or roll, yaw) acceleration from flight logs against AoA, deflections and $\bar{q}$ (Su-30: R^2 0.94), then compute the best case. If nothing can beat the airframe, fix the airframe (README §8.19, design rule 14).
2. **Change one thing,** keeping lift and drag unchanged where you can (§24.3).
3. **Mirror it** in `Registry.EnsureSetup`, in the FCS's effector tables and in the bench plant (README §9).
4. **Check what the old airframe was buying you** (§24.2: the nose-down that limits spend).
5. **Version it as a minor bump** and say in the note that it is an airframe change, not a law change.

24.10 Configurations not done yet

The six reference jets are all conventional take-off jet fighters with fixed wing geometry. Configurations that **none of them covers** yet, and what each would need beyond this kit (expectations, not tested):

- **VTOL / STOVL** (lift fans, swivelling lift nozzles, reaction-control puffers): below wing-borne speed there is almost no aerodynamic authority, so attitude comes from thrust alone. It needs hover and transition laws (attitude and velocity command instead of g and AoA), effector models for lift and puffer jets, and a blend through the transition. The measured-state limits here assume forward flight.
- **Swing wings / variable geometry:** the sweep changes the neutral point, the CG (if the wing mass moves), the lift curve and every wing surface's moment arm in flight. The effector tables and the aero mirrors would have to be rebuilt for the sweep in force, and the sweep schedule itself becomes a law.
- **Tailless and flying-wing designs** (no vertical tail): yaw from split drag rudders and differential thrust only. The F-22E's finless damage handling (split flaperons, README §8.9) is the nearest piece, built as a failure mode, not a design point.
- **Forward-swept wings, three-surface layouts with large canards, or a canard as the main pitch surface:** partly covered by the Revoker and Ternion, but not with a strongly unstable layout.
- **Propeller aircraft:** torque, P-factor and slipstream over the tail are not modelled.
- **Thrust reversers, independent 3D nozzles, or more than two engines:** the TVC effectors and the engine-out logic assume what the six jets have.

If you port to one of these, expect new effector types, new failure modes and new laws, not just a port. Please share what you find: it is how this solution library grew.